

Multithreading in C# with Examples

In this article, I will discuss **Multithreading in C#** with Examples. Multithreading is one of the most important concepts in C# that you must understand as a developer. In this and a few upcoming articles, I will cover all the concepts of C# Multithreading with examples. As part of this article, I will cover the following pointers.

1. **What is Multitasking?**
2. **How Does the Operating System Execute Multiple Applications at a time?**
3. **What is a Process?**
4. **What is a Thread?**
5. **Example to Understand Threading in C#.**
6. **Understanding the Thread Class in .NET Framework.**
7. **What are the Drawbacks of Single-Threaded Applications?**
8. **How to Overcome the Drawbacks of Single-Threaded Applications Using Multithreading?**
9. **What is Multithreading in C#?**
10. **What is Thread Class in C#?**

What is Multitasking?

Before understanding the concept of Multithreading in C#, let us first understand multitasking. As the name says, multitasking means performing multiple tasks simultaneously. Windows Operating System is a multitasking operating system. It means it has the ability to run multiple applications at the same time. For example, on my machine, I can open the Google Chrome Browser, Microsoft Word Document, Notepad, VLC Media Player, Visual Studio, etc., at the same time. This is possible because, on my machine, I have installed the Windows operating system, which is a multitasking operating system.

So, Multitasking refers to the concurrent execution of multiple tasks or processes on a computer system. It allows a computer to appear as though it is performing multiple tasks simultaneously, even though it typically has a single central processing unit (CPU). Multitasking is a fundamental feature of modern operating systems and plays an important role in improving computers' efficiency, responsiveness, and usability.

How Does the Operating System Execute Multiple Applications at a time?

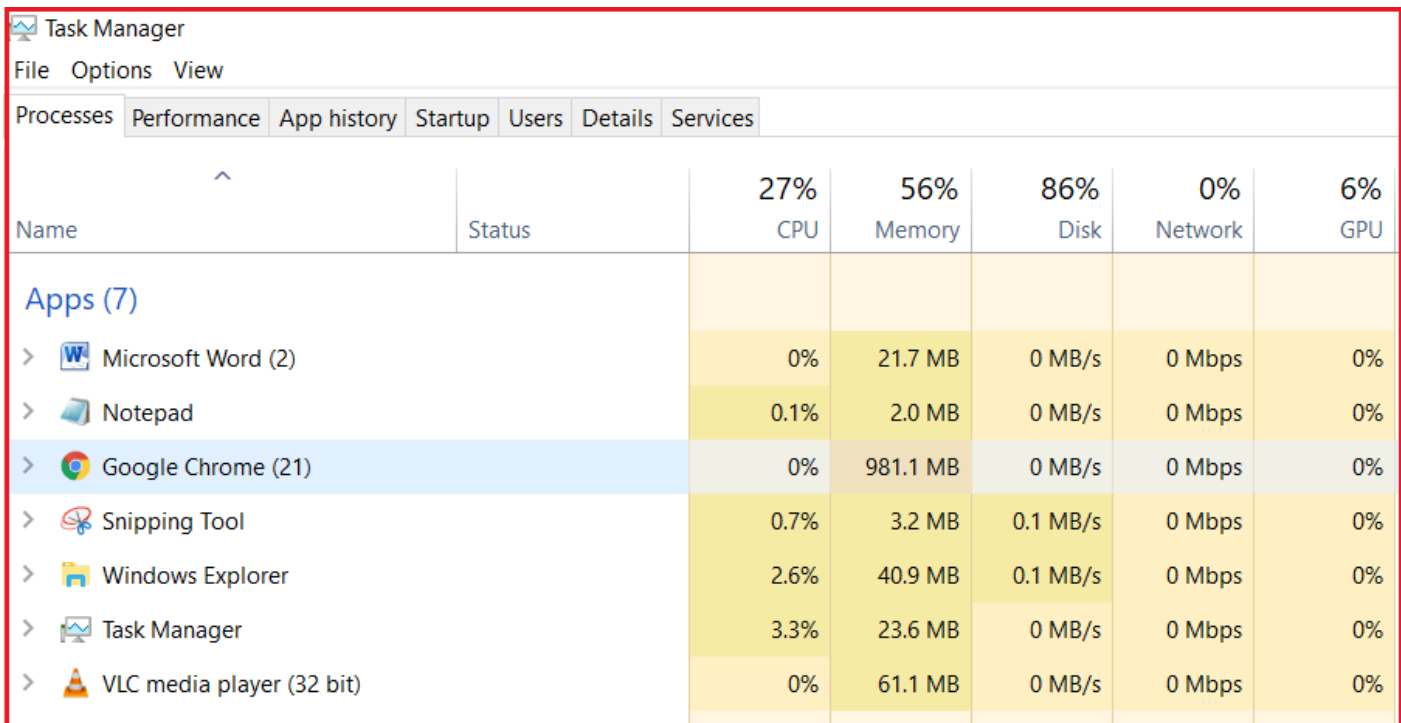
The operating system uses processes internally to execute multiple applications (Google Chrome Browser, Microsoft Word Document, Notepad, VLC Media Player, Visual Studio, etc.) simultaneously.

What is a Process?

A process is a part of the operating system (or a component under the operating system) responsible for executing the program or application. So, to execute each and every program or application, there will be a process.

In an operating system (OS), a process is a fundamental concept representing a program in execution. It is the smallest unit of work in a computer system's execution. Each process has its own memory space, which includes the program code, data, and a stack for managing function calls and local variables. The OS manages processes and has its own resources, such as CPU time, file handles, and system state information.

The processes executing the programs or applications using the Task Manager. Just right-click on the Taskbar and click the Task Manager option to open the Task Manager window. From that window, click on the “Processes” button as shown below.



		27%	56%	86%	0%	6%
		CPU	Memory	Disk	Network	GPU
Name		Status				
Apps (7)						
>	Microsoft Word (2)	0%	21.7 MB	0 MB/s	0 Mbps	0%
>	Notepad	0.1%	2.0 MB	0 MB/s	0 Mbps	0%
>	Google Chrome (21)	0%	981.1 MB	0 MB/s	0 Mbps	0%
>	Snipping Tool	0.7%	3.2 MB	0.1 MB/s	0 Mbps	0%
>	Windows Explorer	2.6%	40.9 MB	0.1 MB/s	0 Mbps	0%
>	Task Manager	3.3%	23.6 MB	0 MB/s	0 Mbps	0%
>	VLC media player (32 bit)	0%	61.1 MB	0 MB/s	0 Mbps	0%

As you can see in the above image, each application is executed by one corresponding process. Along the same line, multiple processes are running in the background, known as the background processes. These background processes are known as Windows Services, and the Operating System runs many Windows services in the background.

So, we have an operating system, and under the operating system, we have processes that run our applications. So, the point that you need to remember is under the process, an application runs. To run the code of an application, the process internally uses a concept called Thread.

What is Thread?

Generally, a Thread is a lightweight process. In simple words, we can say that a Thread is a unit of a process that is responsible for executing the application code. So, every program or application has some logic or code, and to execute that logic or code, Thread comes into the picture.

In an operating system (OS), a thread is a smaller unit of execution within a process. Threads are sometimes called “lightweight processes” because they share the same memory space as the parent process, including its code, data, and resources. However, threads have their own execution context, including a program counter, registers, and stack. Threads within the same process can run concurrently, allowing for parallel execution of tasks.

By default, every process has at least one thread that is responsible for executing the application code, and that thread is called Main Thread. So, every application by default is a single-threaded application.

Note: All the threading-related classes in C# belong to **System.Threading** namespace.

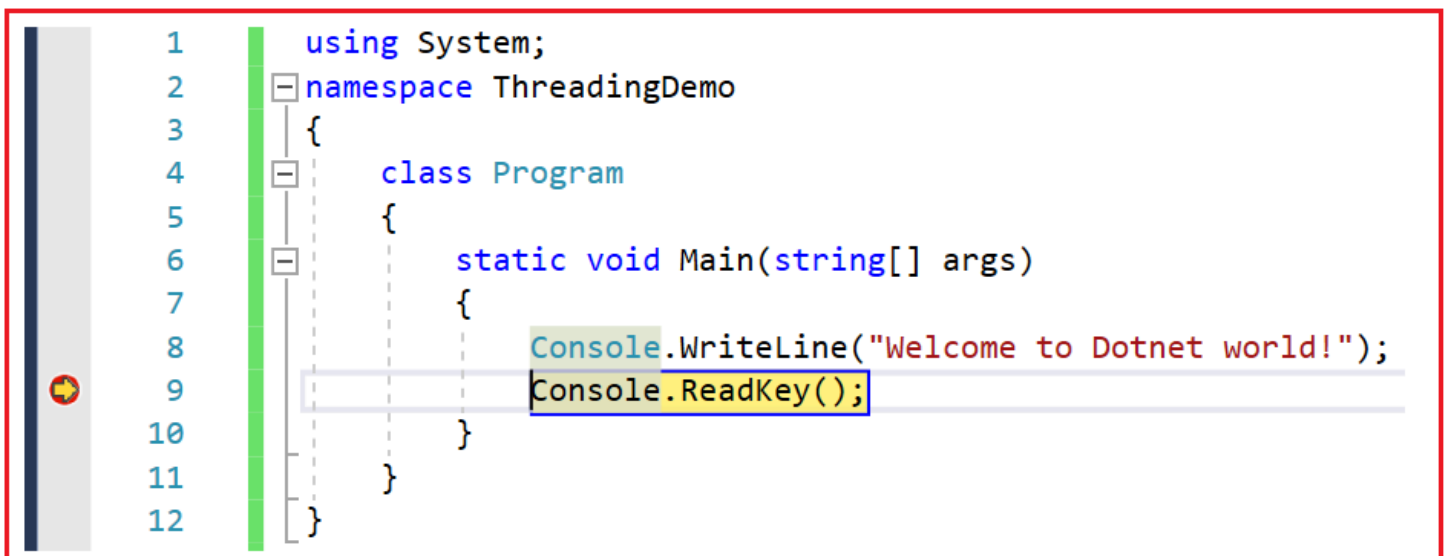
Example to Understand Threading in C#:

Let us see an example to understand Threading in C#. The following is a simple program where we have a class called Program, and in that class, we have a method called Main, which simply prints a message on the Console window.

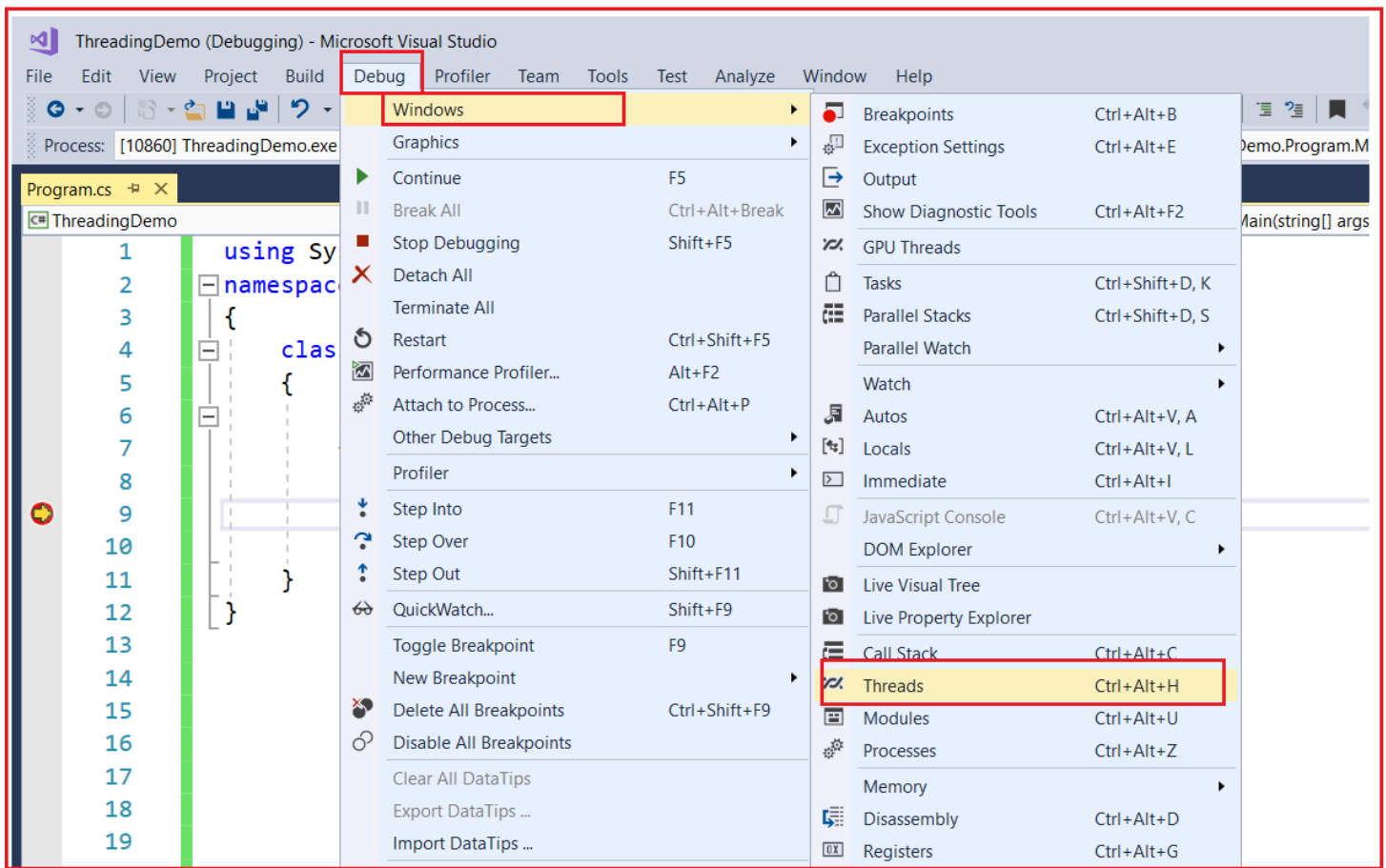
```
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Welcome to Dotnet world!");
            Console.ReadKey();
        }
    }
}
```

Does the above Program Make use of Thread When Run?

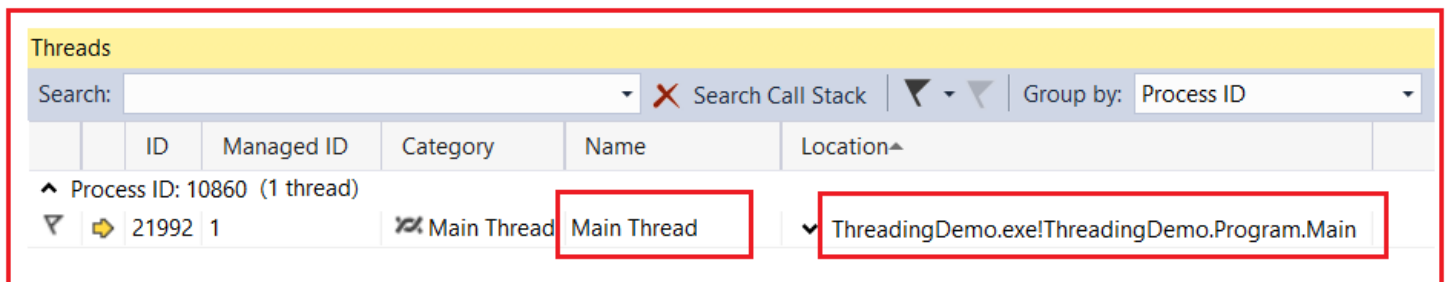
Yes, there is a Thread, and that thread will execute our application code, which is called the Main Thread. Now, let us prove this. Put a debugger point on your application code and then run the application. Let us assume you put the debugger point on line number 9, and when you run the application, you will see that the debugger will hit that point, as shown in the below image.



Once the debugger hits the debugging point, select the **Debug => Windows => Threads** options from the Visual Studio menu, as shown in the below image.



Once you select the **Debug => Windows => Threads** options, then it will open the following window. You can see here that the Main Thread is currently executing the Main method of the Program class.



So, this proves that by default, every application is a single-threaded application, and by default, the Main thread will execute our application code.

Thread Class in C#:

Go to the definition of Thread Class in C#. You will see that it contains one static property called **CurrentThread**, which will return the instance of the currently executing thread, i.e., the thread running your application code. If you go to the definition of Thread class, you will find the following signature.

```

public sealed class Thread : CriticalFinalizerObject, _Thread
{
    -- Other Methods and Properties
    //
    // Summary:
    // Gets the currently running thread.
    //
    // Returns:
    // A System.Threading.Thread that is the representation of the currently running thread.
    public static Thread CurrentThread { get; }
    public string Name { get; set; }
    -- Other Methods and Properties

    public static void Sleep(TimeSpan timeout);
    public static void Sleep(int millisecondsTimeout);
}

```

As you can see, the **CurrentThread** static property return type is Thread, i.e., it will return the instance of the currently executing thread. Along the same line, there is a non-static property called Name, using which we can set and get the Name of the currently executing thread.

Note: By default, the thread does not have any name. You can provide any name to the thread using the Name property of the Thread class. So, modify the program as shown below, where we set the Name property of the thread class and printing the Name property.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t = Thread.CurrentThread;
            //By Default, the Thread does not have any name
            //if you want then you can provide the name explicitly
            t.Name = "Main Thread";
            Console.WriteLine("Current Executing Thread Name :" + t.Name);
            Console.WriteLine("Current Executing Thread Name :" +
Thread.CurrentThread.Name);
            Console.Read();
        }
    }
}

```

Output:

```
Current Executing Thread Name :Main Thread  
Current Executing Thread Name :Main Thread
```

As you can see, to run the application code, one thread is created, i.e., the Main Thread. So, this proves that, by default, every application is a single-threaded application.

What are the drawbacks of Single-Threaded Applications in the .NET Framework?

Single-threaded applications in the .NET Framework have several drawbacks, particularly regarding performance, responsiveness, and scalability. Some of the main drawbacks include:

- **Limited CPU Utilization:** Single-threaded applications can only execute one task or operation simultaneously. This means they cannot fully utilize modern multi-core processors, leading to the underutilization of available CPU resources. In contrast, multi-threaded or parallel applications can distribute work across multiple threads and utilize more CPU cores efficiently.
- **Poor Responsiveness:** In a single-threaded application, time-consuming operations can block the entire execution, making the application appear unresponsive to user input. For example, if a long-running computation occurs on the main thread, the user interface won't update until the computation is complete. This can result in a poor user experience.
- **Difficulty Handling Concurrent Tasks:** Single-threaded applications struggle to handle concurrent tasks, such as responding to user input, while performing background tasks like data loading or computation. The application may become unresponsive during such operations without additional threads or asynchronous programming techniques.
- **Scalability Challenges:** Single-threaded applications often face challenges when scaling to simultaneously handle multiple users or workloads. They may not be well-suited for scenarios requiring high concurrency or parallel processing, such as web servers or real-time data processing applications.
- **Limited Parallelism:** Single-threaded applications cannot perform true parallel processing. This can be a significant drawback when dealing with computationally intensive tasks that can benefit from parallelism, such as data analysis, rendering, or simulations.
- **Inefficient Resource Utilization:** Single-threaded applications may not efficiently utilize other system resources, such as memory and I/O. For example, they might block the thread while waiting for I/O operations to complete, leading to wasted CPU cycles that could be used for other tasks.
- **Difficulty in Achieving Fault Tolerance:** In single-threaded applications, handling errors and exceptions can be more challenging because a failure in one part of the application can disrupt the entire process. Multi-threaded or multi-process architectures can isolate failures to specific threads or processes, making fault tolerance easier.

To address these limitations, developers often use multi-threading, asynchronous programming, or parallel processing techniques provided by the .NET Framework to make applications more responsive, scalable, and efficient. These approaches enable applications to utilize modern hardware better and improve user experiences, especially in concurrent tasks and heavy workload scenarios.

In a single-threaded application, all the logic or code present in the program will be executed by a single thread only, i.e., the Main thread. For example, if we have three methods in our application. All these three methods will be called from the Main method. Then, the Main thread is only responsible for executing all these three methods sequentially, i.e., one by one. It will execute the first method, and once it completes the execution of the first method, it only executes the second method, and so on. Let us understand this with an example. Modify the program as shown below.

```
using System;
```

```
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Method1();
            Method2();
            Method3();
            Console.Read();
        }
        static void Method1()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }

        static void Method2()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method2 :" + i);
            }
        }
        static void Method3()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method3 :" + i);
            }
        }
    }
}
```

Output:

```
Method1 :1
Method1 :2
Method1 :3
Method1 :4
Method1 :5
Method2 :1
Method2 :2
Method2 :3
Method2 :4
Method2 :5
Method3 :1
Method3 :2
Method3 :3
Method3 :4
Method3 :5
```

As you can see in the above output, the methods are called and executed one after the other. The Main thread first executes Method1, and once it completes the execution of Method1, it calls Method2 and Method3.

What is the problem with the above Program Execution?

In our example, we are just writing some simple code to print the values from 1 to 5 within the method body. What will you do if one method takes more than expected time? Suppose Method2 is going to interact with a database, or it is going to invoke any web services or any web APIs, which will take more than 10 seconds to provide the response. In that case, the Method2 execution will be delayed for 10 seconds as it is waiting to get a response from the database, the Web Service, or the Web API. Until Method2 has not completed its execution, Method3 will not be executed because of the sequential execution of the program, i.e., one by one.

Example to Understand the Problem of Single-Threaded Application in C#.

Here, we will not perform any database or Web Service call; instead, we will use the Thread class Sleep method to delay the execution of Method2 for 10 seconds. Following is the signature of the Sleep Method. Another overloaded version of the Sleep method is available, which takes TimeSpan as input, and we will discuss it in our coming articles.

```
public static void Sleep(int millisecondsTimeout);
```

The sleep method takes the time in milliseconds as input and then suspends the current thread execution for that specified number of milliseconds. So, please modify the Program class code as shown below.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    Method1();
    Method2();
    Method3();
    Console.Read();
}
static void Method1()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("Method1 :" + i);
    }
}
static void Method2()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("Method2 :" + i);
        if (i == 3)
        {
            Console.WriteLine("Performing the Database Operation Start");
            //Sleep for 10 seconds
            Thread.Sleep(10000);
            Console.WriteLine("Performing the Database Operation Complete");
        }
    }
}
static void Method3()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("Method3 :" + i);
    }
}
}
}

```

Now, run the application and notice that the Method2 execution is delayed for 10 seconds. Once Method2 completes its execution, only Method3 starts its execution. This is because all three methods are executed by a single thread, which is the drawback of the single-threaded application.

How do we solve the above problem?

To solve the above problem, we have a concept called Multithreading in C#. As we already discussed Operating System has Processes used to run our applications. The Process contains Thread, which will actually run our application code.

A process, by default, uses a single thread to run our application code, but a process can have multiple threads, and multiple threads can run our application code simultaneously. In this case, each thread will perform a different task, or you can say each thread will execute different parts of our application code.

In our application, we have three methods. So, the three methods we defined in our application can be executed by three threads. The advantage is that the execution takes place simultaneously. So, when multiple threads try to execute the application code, the operating system allocates some time for each thread.

In our example, we want to execute the three methods using three different threads. let us say t1, t2, and t3. The thread t1 is going to execute Method1, and thread t2 is going to execute the Method2. At the same time, Method3 will be executed by thread t3. Let us modify the Program class as shown below to execute the three methods with three different Threads. Here, the main thread will execute the Main method, and the Main thread will create the child threads.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Creating Threads
            Thread t1 = new Thread(Method1)
            {
                Name = "Thread1"
            };
            Thread t2 = new Thread(Method2)
            {
                Name = "Thread2"
            };
            Thread t3 = new Thread(Method3)
            {
                Name = "Thread3"
            };
            //Executing the methods
            t1.Start();
            t2.Start();
            t3.Start();
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 Started using " + Thread.CurrentThread.Name);
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
            Console.WriteLine("Method1 Ended using " + Thread.CurrentThread.Name);
        }
        static void Method2()
        {
            Console.WriteLine("Method2 Started using " + Thread.CurrentThread.Name);
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method2 :" + i);
            }
            Console.WriteLine("Method2 Ended using " + Thread.CurrentThread.Name);
        }
        static void Method3()
        {
            Console.WriteLine("Method3 Started using " + Thread.CurrentThread.Name);
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method3 :" + i);
            }
            Console.WriteLine("Method3 Ended using " + Thread.CurrentThread.Name);
        }
    }
}
```

```

        Console.WriteLine("Method2 Started using " + Thread.CurrentThread.
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("Method2 :" + i);
            if (i == 3)
            {
                Console.WriteLine("Performing the Database Operation Start
                //Sleep for 10 seconds
                Thread.Sleep(10000);
                Console.WriteLine("Performing the Database Operation Compl
            }
        }
        Console.WriteLine("Method2 Ended using " + Thread.CurrentThread.Na
    }
    static void Method3()
    {
        Console.WriteLine("Method3 Started using " + Thread.CurrentThread.
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("Method3 :" + i);
        }
        Console.WriteLine("Method3 Ended using " + Thread.CurrentThread.Na
    }
}
}
}
}
}

```

As you can see in the above code, we have created three different instances of the Thread class. To the constructor of the Thread class, we need to pass the method name that needs to be executed by that Thread. Then, we call the Start() method on the Thread class, which will start executing the method. You will not get the output sequentially. Run the application and see the output as shown below. The output may vary in your machine.

```

Main Thread Started
Method1 Started using Thread1
Method1 :1
Method1 :2
Method1 :3
Method1 :4
Method2 Started using Thread2
Method2 :1
Method2 :2
Method2 :3
Performing the Database Operation Started
Method1 :5
Method1 Ended using Thread1
Main Thread Ended
Method3 Started using Thread3
Method3 :1
Method3 :2
Method3 :3
Method3 :4
Method3 :5
Method3 Ended using Thread3
Performing the Database Operation Completed
Method2 :4
Method2 :5
Method2 Ended using Thread2

```

Here, the Main thread is going to create all other Threads. Here, the main thread will execute the Main method, and the Main thread will create the child threads called Worker Threads. Now, to see all the threads, run the application in debug mode by putting debugging point and then open **Debug => Windows => Threads** options, then it will open the following window. You can see here that the Main Thread and the Worker threads execute our application code.

Threads					
Search:		X Search Call Stack		Group by: Process ID	
	ID	Managed ID	Category	Name	Location
^ Process ID: 20352 (4 threads)					
▼	18892	1	Main Thread	Main Thread	▼ ThreadingDemo.exe!ThreadingDemo.Program.Main
▼	21204	3	Worker Thread	Thread1	▼ ThreadingDemo.exe!ThreadingDemo.Program.Method1
▼	14468	4	Worker Thread	Thread2	▼ ThreadingDemo.exe!ThreadingDemo.Program.Method2
▼	19304	5	Worker Thread	Thread3	▼ ThreadingDemo.exe!ThreadingDemo.Program.Method3

What is Multithreading in C#?

If multiple threads are used to execute your application code, it is called Multithreading. Multithreading is a mechanism to implement Concurrent Programming where multiple threads operate simultaneously. Threads are lightweight processes that signify the execution path in a program. Thread usage increases the efficiency of an application and reduces CPU cycle time wastage. The main advantage of using Multithreading is the maximum utilization of CPU resources.

Multithreading in C# refers to the ability of the C# programming language and the .NET Framework to create and manage multiple threads of execution within a single process. Threads are lightweight, independent

sequences of instructions that can run concurrently, allowing you to perform multiple tasks simultaneously. Multithreading is a powerful concept in C# and is used to achieve various goals, such as improving application responsiveness, parallelizing tasks, and efficiently utilizing multi-core processors.

Key Features of Multithreading in C#:

- **Thread Class:** C# provides the `System.Threading.Thread` class, which allows you to create and manage threads. Using this class, you can create new threads, start and stop them, set their priorities, and perform various synchronization and coordination tasks.
- **Parallel Programming:** Multithreading is often used for parallel programming, where multiple threads work on separate parts of a problem to improve performance. The .NET Framework provides libraries and constructs like the Task Parallel Library (TPL) and Parallel LINQ (PLINQ) to simplify parallel programming.
- **Improved Responsiveness:** Multithreading can keep the user interface (UI) responsive while performing time-consuming tasks in the background. For example, in a GUI application, you can run a time-consuming computation on a separate thread so that the UI remains interactive and doesn't freeze.
- **Concurrency:** Multithreading enables concurrent execution of code sections, which is useful for scenarios where multiple threads need to access shared resources, perform I/O operations, or handle multiple client requests simultaneously.
- **Thread Safety:** When multiple threads access shared data or resources, it's crucial to ensure thread safety to prevent data corruption and race conditions. C# provides synchronization primitives like locks, mutexes, semaphores, and the Monitor class to help you achieve thread safety.
- **Async/Await:** C# includes the `async` and `await` keywords, simplifying asynchronous programming. These keywords allow you to write asynchronous code that can run concurrently with other tasks without blocking the calling thread. Asynchronous programming is essential for scalable and responsive applications.

Multithreading can greatly enhance the performance and responsiveness of C# applications, but it also introduces complexities related to synchronization, coordination, and potential issues like deadlocks. Therefore, it's essential to use multithreading judiciously and follow best practices to ensure robust and reliable concurrent code.

What is Thread Class in C#?

1. In C#, the Thread class is used to create threads.
2. With the help of the Thread class, we can create **foreground and background threads**. What are the foreground and background threads that we are going to discuss in our coming article?
3. Thread class also allows us to set the priority of a thread. In our coming articles, we will discuss **how to set the priority of a thread**.
4. The Thread class in C# also provides the current state of a thread. We will discuss this when discussing **Thread Life Cycle** in C#.
5. The Thread class in C# is sealed and cannot be inherited.

In the next article, I will discuss **Constructors of Thread Class in C#** with Examples. In this article, I explain the concept of **Multithreading in C#** with Examples. I hope you understood the basics of C# Multithreading with Examples and enjoy this article.

Thread Class in C# with Examples

In this article, I am going to discuss **Thread Class in C#** with Examples. Please read our previous article before proceeding to this article where we discussed the basic concepts of **Multithreading in C#** with Examples. As part of this article, we are going to discuss the following pointers in detail with examples.

1. **What is Thread Class in C#?**
2. **Properties, Methods, and Constructors of Thread class.**
3. **Understanding the different Constructors of Thread Class in detail.**
4. **Why is the Constructor of Thread Class taking a Parameter of Delegate Type?**
5. **Understanding ThreadStart delegate in C#.**
6. **Thread Function with Parameter.**
7. **Understanding ParameterizedThreadStart Delegate in C#.**
8. **When to use ParameterizedThreadStart over ThreadStart delegate?**
9. **What are the Problems with the ParameterizedThreadStart delegate in C#?**
10. **How to Overcome the Problems of ParameterizedThreadStart delegate in C#?**

What is Thread Class in C#?

The Thread class in C# is responsible for creating custom threads. With the help of the Thread class, we can create both foreground and background threads. What are the **foreground and background threads** that we are going to discuss in our coming articles? The Thread class also allowed us to set the priority of a thread. In our coming session, we will discuss **how to set the priority of a thread**.

If you go to the definition of the Thread class, then you will see that is a sealed class and as it is a sealed class, this class cannot be inherited i.e. further inheritance is not possible. The Thread class in C# provides lots of useful properties, methods, and constructors, which we are going to discuss in this article.

Thread Class Properties in C#:

The Thread class in C# provides lots of properties. Some of the important properties are as follows:

1. **CurrentThread**: This property is used to get the currently running thread. It returns a Thread that is the representation of the currently running thread.
2. **IsAlive**: This property is used to get a value indicating the execution status of the current thread. It returns true if this thread has been started and has not terminated normally or aborted; otherwise, false.
3. **IsBackground**: This property is used to get or set a value indicating whether or not a thread is a background thread. It returns true if this thread is or is to become a background thread; otherwise, false.
4. **ManagedThreadId**: This property is used to get a unique identifier for the current managed thread. It returns an integer that represents a unique identifier for this managed thread.
5. **Name**: This property is used to get or set the name of the thread. It returns a string containing the name of the thread, or null if no name was set.
6. **Priority**: This property is used to get or set a value indicating the scheduling priority of a thread. It returns one of the System.Threading.ThreadPriority values. The default value is System.Threading.ThreadPriority.Normal.
7. **ThreadState**: This property is used to get a value containing the states of the current thread. It returns one of the System.Threading.ThreadState values indicate the state of the current thread. The initial

value is Unstarted.

8. **IsThreadPoolThread**: This property is used to get a value indicating whether or not a thread belongs to the managed thread pool. It returns true if this thread belongs to the managed thread pool; otherwise, false.

Thread Class Methods in C#:

The Thread class in C# provides lots of methods. Some of the important methods are as follows:

1. **Abort()**: This method is used to terminate the thread. Raises a ThreadAbortException in the thread on which it is invoked, to begin the process of terminating the thread. Calling this method usually terminates the thread.
2. **Interrupt()**: This method is used to interrupt a thread that is in the WaitSleepJoin thread state.
3. **Join()**: This method is used to block the calling thread until the thread represented by this instance terminates while continuing to perform standard COM and SendMessage pumping.
4. **ResetAbort()**: This method is used to an System.Threading.Thread.Abort(System.Object) requested for the current thread.
5. **Resume()**: This method is used to resume a thread that has been suspended.
6. **Sleep(Int32)**: This method is used to suspend the current thread for the specified number of milliseconds.
7. **Start()**: This method causes the operating system to change the state of the current instance to System.Threading.ThreadState.Running.
8. **Suspend()**: This method is used to either suspend the thread or if the thread is already suspended, have no effect
9. **Yield()**: This method causes the calling thread to yield execution to another thread that is ready to run on the current processor. The operating system selects the thread to yield to. It returns true if the operating system switched execution to another thread; otherwise, false.

Thread Class Constructors in C#:

The Thread class in C# provides the following 4 constructors.

1. **Thread(ThreadStart start)**: It initializes a new instance of the Thread class. Here, the parameter start specifies a ThreadStart delegate that represents the methods to be invoked when this thread begins executing. It will throw ArgumentNullException, the start parameter is null.
2. **Thread(ParameterizedThreadStart start)**: It initializes a new instance of the Thread class, specifying a delegate that allows an object to be passed to the thread when the thread is started. Here, the parameter start specifies a delegate that represents the methods to be invoked when this thread begins executing. It will throw ArgumentNullException, the start parameter is null.
3. **Thread(ThreadStart start, int maxStackSize)**: It initializes a new instance of the Thread class, specifying the maximum stack size for the thread. Here, the parameter start specifies a ThreadStart delegate that represents the methods to be invoked when this thread begins executing. And the parameter maxStackSize specifies the maximum stack size, in bytes, to be used by the thread, or 0 to use the default maximum stack size specified in the header for the executable. Important For partially trusted code, maxStackSize is ignored if it is greater than the default stack size. No exception is thrown.
4. **Thread(ParameterizedThreadStart start, int maxStackSize)**: It initializes a new instance of the Thread class, specifying a delegate that allows an object to be passed to the thread when the thread is started and specifies the maximum stack size for the thread.

Note: In this article, I am going to discuss the Thread class constructors in detail with examples and the methods and properties of thread class are going to be discussed in our subsequent articles.

Example to Understand Constructors of Thread Class in C#.

Let us understand the constructors of the thread class in C# with an example. Please have a look at the following code. It's a very simple program and here we are executing the DisplayNumbers function using a different thread.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t1 = new Thread(DisplayNumbers);
            t1.Start();
            Console.Read();
        }
        static void DisplayNumbers()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }
    }
}
```

As you can see in the above code, here, we created an instance of the Thread class and to the constructor of the Thread class, we passed the method name that we want the thread to execute. The following line of code exactly does the same.

Thread t1 = new Thread(DisplayNumbers);

Constructors of Thread Class in C#:

As discussed earlier, in C#, the Thread class contains four constructors. If you go to the definition of Thread class then you will see the Constructors as shown below.

```
public sealed class Thread : CriticalFinalizerObject, _Thread
{
    ...public Thread(ThreadStart start);
    ...public Thread(ParameterizedThreadStart start);
    ...public Thread(ThreadStart start, int maxStackSize);
    ...public Thread(ParameterizedThreadStart start, int maxStackSize);
}
```

Now you may have one question, the Thread class constructor which takes one parameter is either of the type ThreadStart or ParameterizedThreadStart, but in our example, we are passing the method name as a parameter to the Thread class Constructor and it works, how? To understand this, let us go to the definition of ThreadStart and you will see that ThreadStart is actually a delegate as shown in the below image.

```
namespace System.Threading
{
    ...public delegate void ThreadStart();
}
```

Why is the Constructor of the Thread class Taking a Parameter of Delegate Type?

As we already discussed, the main objective of creating a Thread in C# is to execute a function. A delegate is a type-safe function pointer. It means the delegate points to a function and when we invoke the delegate, the function which is pointed by the delegate is going to be executed.

In simple words, we can say that all the threads that we create require an entry point (i.e. a pointer to the function) from where it should execute. This is the reason why threads always require a delegate. If you want to learn [Delegates in C#](#) with Examples, then I strongly recommended you read our [Delegates in C#](#) article.

Note: We already discussed in our Delegates article that the signature of the delegate should and must be the same as the signature of the method it points to.

As you can see, the ThreadStart delegate does not take any parameter and the return type is void. In our example, the DisplayNumbers() function signature is the same as the ThreadStart delegate signature as the DisplayNumbers() function return type is void as well as it does not take any parameters.

Thread t1 = new Thread(DisplayNumbers);

So, the above thread instance creation statement is implicitly converted to the ThreadStart delegate instance. So, you can write the above statement as shown in the below image and it will work.

```
Thread t1 = new Thread(DisplayNumbers);
```



```
ThreadStart obj = new ThreadStart(DisplayNumbers);  
Thread t1 = new Thread(obj);
```

As you can see in the above image, it is a two steps process. First, we need to create the ThreadStart Delegate Instance and while creating the instance, to its constructor we need to pass the method name that we want to execute. In the second step, to the Constructor of Thread class, we need to pass the ThreadStart instance as a parameter. And when we start the Thread execution by calling the Start method on the thread instance, internally it will invoke the ThreadStart delegate instance which internally start the execution of the DisplayNumbers method.

Example to Understand ThreadStart Delegate in C#:

In the below example, first, we are creating an instance of ThreadStart delegate and to the constructor of ThreadStart delegate, we pass the DisplayNumbers function as a parameter. Then we create an instance of the Thread class and to the constructor of the Thread class, we pass the ThreadStart delegate instance as a parameter that points to the DisplayNumbers function. Finally, when we call the Start method on the Thread instance, which will invoke the ThreadStart delegate instance and the delegate instance will invoke the method it points to i.e. the DisplayNumbers function is going to start its execution.

```
using System.Threading;  
using System;  
namespace ThreadingDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating the ThreadStart Delegate instance by passing the  
            //method name as a parameter to its constructor  
            ThreadStart obj = new ThreadStart(DisplayNumbers);  
            //Passing the ThreadStart Delegate instance as a parameter to its  
            Thread t1 = new Thread(obj);  
            t1.Start();  
            Console.Read();  
        }  
        static void DisplayNumbers()  
        {  
            for (int i = 1; i <= 5; i++)  
            {  
                Console.WriteLine("Method1 :" + i);  
            }  
        }  
    }  
}
```

```
}
```

You can also combine the above two statements into a single statement as shown below.

```
ThreadStart obj = new ThreadStart(DisplayNumbers);  
Thread t1 = new Thread(obj);  
↓  
Thread t1 = new Thread(new ThreadStart(DisplayNumbers));
```

Creating Thread Class Instance using Anonymous Method in C#:

It is also possible to create a Thread class instance by using the anonymous method as shown in the below example. We know that Anonymous methods are created by using the delegate keyword and they are assigned to a type of delegate.

```
using System.Threading;  
using System;  
namespace ThreadingDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating Thread Class Instance using Lambda Expression  
            Thread t1 = new Thread(delegate ()  
            {  
                for (int i = 1; i <= 5; i++)  
                {  
                    Console.WriteLine("Method1 :" + i);  
                }  
            });  
            t1.Start();  
            Console.Read();  
        }  
    }  
}
```

Creating Thread Class Instance using Lambda Expression in C#:

It is also possible to create the thread class instance by using the lambda expression as shown in the below example. We know that Lambda Expressions are created by using the => Lambda Operator.

```
using System.Threading;  
using System;  
namespace ThreadingDemo
```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Thread Class Instance using Lambda Expression
            Thread t1 = new Thread(() =>
            {
                for (int i = 1; i <= 5; i++)
                {
                    Console.WriteLine("Method1 :" + i);
                }
            });
            t1.Start();
            Console.Read();
        }
    }
}

```

Thread Function with Parameter in C#:

The examples that we have worked on so far do not take any parameters. That is the method that we want to execute by the custom thread that does not take any parameter. Now, let us proceed and try to understand how to work with the thread function which takes the input parameter.

Let's change the DisplayNumbers() method implementation to take one parameter. Now, this method takes one input parameter of the object type. And then convert that object type to an integer value and then print the numbers up to that value starting from 1. For a better understanding, please have a look at the below example.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //DisplayNumbers is now a non-static method, so we need to
            //refer it by using the instance
            Program obj = new Program();
            Thread t1 = new Thread(obj.DisplayNumbers);
            t1.Start(5);
            Console.Read();
        }
        public void DisplayNumbers(object Max)
        {
            int Number = Convert.ToInt32(Max);
            for (int i = 1; i <= Number; i++)
            {

```

```

        Console.WriteLine("Method1 :" + i);
    }
}
}

```

When the thread method or thread function (the function which is going to be executed by the custom thread) takes one parameter, then the Thread class uses the ParameterizedThreadStart delegate internally. The definition of the ParameterizedThreadStart delegate in C# is shown in the below image.

```

namespace System.Threading
{
    ...public delegate void ParameterizedThreadStart(object obj);
}

```

As you can see the ParameterizedThreadStart delegate is taking one parameter of object type and like the ThreadStart delegate it also does not return any value. Now the DisplayNumbers() method signature is the same as the signature of this ParameterizedThreadStart delegate. So, the .NET Framework internally converts the statements as shown in the below image.

```

Thread t1 = new Thread(obj.DisplayNumbers);
↓
Thread t1 = new Thread(new ParameterizedThreadStart(obj.DisplayNumbers));

```

Creating the ParameterizedThreadStart Instance Manually in C#:

Let us see how to create the ParameterizedThreadStart delegate instance manually and pass that instance to the Thread class Constructor. For a better understanding, please have a look at the below example. In the below example, first, we are creating an instance of ParameterizedThreadStart delegate and to the constructor of ParameterizedThreadStart delegate, we pass the DisplayNumbers function as a parameter. Then we create an instance of the Thread class and to the constructor of the Thread class, we pass the ParameterizedThreadStart delegate instance as a parameter that points to the DisplayNumbers function. Finally, when we call the Start method by passing the value 5 on the Thread instance, it will invoke the delegate instance by passing the same value 5 and that delegate instance when invoked, will execute the DisplayNumbers function by passing the value 5.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();

```

```

        ParameterizedThreadStart PTSD = new
ParameterizedThreadStart(obj.DisplayNumbers);
        Thread t1 = new Thread(PTSD);
        t1.Start(5);
        Console.Read();
    }
    public void DisplayNumbers(object Max)
    {
        int Number = Convert.ToInt32(Max);
        for (int i = 1; i <= Number; i++)
        {
            Console.WriteLine("Method1 :" + i);
        }
    }
}

```

Now run the above code and you will get the output as expected as shown in the below image.

```

Method1 :1
Method1 :2
Method1 :3
Method1 :4
Method1 :5

```

When to use ParameterizedThreadStart Delegate over ThreadStart Delegate in C#?

You need to use the ParameterizedThreadStart delegate if your method takes any values else you just need to use the ThreadStart delegate which does not take any parameter.

What are the Problems with the ParameterizedThreadStart delegate in C#?

As you can see, the parameter type of the ParameterizedThreadStart delegate is an object type. So, the parameter of the thread function is also going to be the object data type. And you cannot change the data type from object to any other type and if you try then it will give you a compile-time error. As the thread function operates on object data type, so we can pass any type of value and it accepts. As a result, the function is not going to be type-safe as we can pass any type of value. Again, as it operates on object data type, so boxing and unboxing will come into the picture and we already discussed in our [Boxing and Unboxing](#) article how it degrades our application performance. So, we should avoid object types.

Example to Pass String Value to a Thread Function in C#:

Let us try to pass a string value to the thread function and see what happens as shown below in the below example.

```

using System.Threading;
using System;
namespace ThreadingDemo

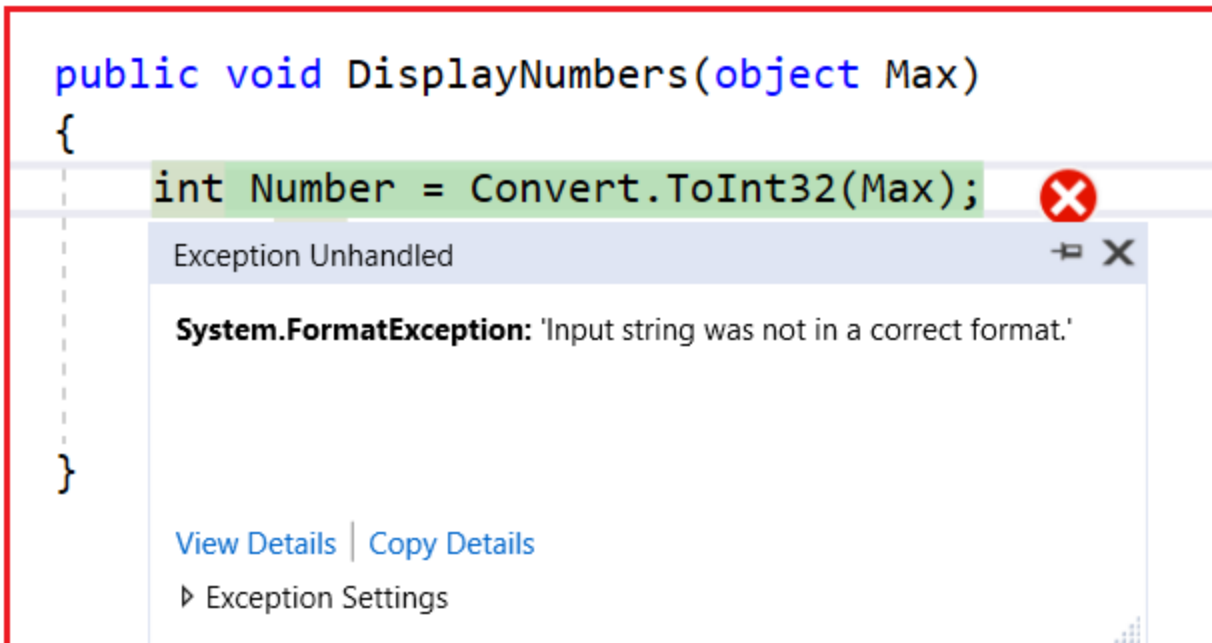
```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();
            ParameterizedThreadStart PTSD = new
ParameterizedThreadStart(obj.DisplayNumbers);
            Thread t1 = new Thread(PTSD);
            t1.Start("Hi");
            Console.Read();
        }
        public void DisplayNumbers(object Max)
        {
            int Number = Convert.ToInt32(Max);
            for (int i = 1; i <= Number; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }
    }
}

```

Now you will not get any compile-time error, but once you run the application, then you will get a runtime error as shown below in the below image.



In the next article, I am going to discuss [How to Pass Data to a Thread Function in a Type-Safe Manner in C#](#) with Examples. Here, in this article, I try to explain the **Thread Class in C#** with Examples. I hope you understood how to use the `ThreadStart` and `ParameterizedThreadStart` Delegates in C# Multithreading with Examples.

How to Pass Data to Thread Function in Type-Safe Manner in C#

In this article, I am going to discuss **How to Pass Data to Thread Function in Type-Safe Manner in C#** with Examples. Please read our previous article where we discussed the [Thread Class in C#](#) with Examples. As part of this article, we are going to discuss the following pointers.

1. **How to Pass data to the Thread function in C#?**
2. **How to make the thread function type-safe in C#?**

How to Pass Data to the Thread Function in C#

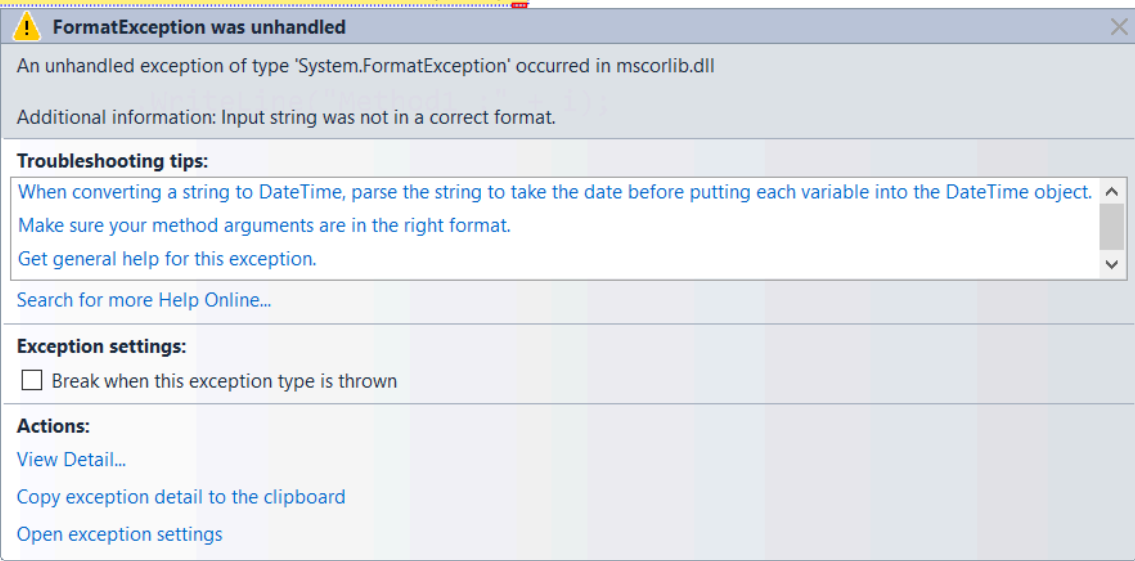
Let us understand this with an example. In the below example, the DisplayNumbers function takes an argument of the object type. Then in the main method, we created an instance of the ParameterizedThreadStart delegate, and to the constructor of the ParameterizedThreadStart delegate, we pass the DisplayNumbers function. Next, we created an instance of the Thread class and to the constructor of the Thread class, we pass the ParameterizedThreadStart delegate instance as a parameter that points to the DisplayNumbers function. Finally, we call the Start method and pass a string value "Hi", and here we will not get any compile-time error.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();
            ParameterizedThreadStart PTSD = new
ParameterizedThreadStart(obj.DisplayNumbers);
            Thread t1 = new Thread(PTSD);

            t1.Start("Hi");
            Console.Read();
        }
        public void DisplayNumbers(object Max)
        {
            int Number = Convert.ToInt32(Max);
            for (int i = 1; i <= Number; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }
    }
}
```

At the time of compilation, we will not get any compile-time error. But when we run the application, we will get the following runtime error.

```
public void DisplayNumbers(object Max)
{
    int Number = Convert.ToInt32(Max);
    for
    {
    }
}
```



This is because the thread function is not type-safe as it operates on the object data type. Let's see how to make the thread function type-safe so that we can pass the data in a type-safe manner. So, whenever we are saying type safe means there should be any kind of boxing and unboxing.

How to Make the Thread Function Type-Safe in C#?

When we are saying type-safe, it means we should not use the object data type. Here in our example, we need to use the data type as an integer. So at the time of compilation, if we pass any data other than an integer, then it should give us a compile-time error. Let us see how to achieve this step by step.

Step1:

In order to pass the data in a type-safe manner to a Thread function in C#, first, you need to encapsulate the thread function and the data it requires in a helper class. So, create a class file with the NumberHelper.cs and then copy and paste the following code into it.

```
using System;
namespace ThreadingDemo
{
    public class NumberHelper
    {
        int _Number;

        public NumberHelper(int Number)
        {
            _Number = Number;
        }
    }
}
```

```

    public void DisplayNumbers()
    {
        for (int i = 1; i <= _Number; i++)
        {
            Console.WriteLine("value : " + i);
        }
    }
}

```

As you can see, we have created the above NumberHelper class with one private variable i.e. _Number, one parameterized constructor, and one method i.e. DisplayNumbers. The private variable _Number is going to hold the target number. The constructor takes one input parameter of integer type and then assigns that value to the private variable. So, while we are creating the instance of NumberHelper class we need to supply the Number value. Finally, the DisplayNumbers function is used to display the values starting from 1 to the value that is present in the _Number variable.

Step2:

In the main method, we need to create an instance of NumberHelper class and to its constructor, we need to pass the integer value. Then create the ThreadStart delegate instance and pass the DisplayNumbers function as a parameter to its constructor. So, please modify the Program.cs class file as shown below.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Max = 10;
            NumberHelper obj = new NumberHelper(Max);

            Thread T1 = new Thread(new ThreadStart(obj.DisplayNumbers));

            T1.Start();
            Console.Read();
        }
    }
}

```

Now run the application and it should display the output as expected as shown below.

```
value : 1  
value : 2  
value : 3  
value : 4  
value : 5  
value : 6  
value : 7  
value : 8  
value : 9  
value : 10
```

In the next article, I am going to discuss [How to Retrieve Data From a Thread Function in C#](#) with Examples. Here, in this article, I try to explain **How to Pass Data to a Thread Function in a Type-Safe Manner in C#** with Examples. I hope you enjoy this How to Pass Data to Thread Function in Type-Safe Manner in C# with Examples article.

How to Retrieve Data from a Thread Function in C#

In this article, I am going to discuss **How to Retrieve Data from a Thread Function in C# using the Callback Method** with Examples. Please read our previous article before proceeding to this article where we discussed [How to Pass Data to Thread Function in a Type-Safe Manner in C#](#) with Examples. Here, we will see how to use a call-back method in C# to return data from a thread function. As part of this article, we will discuss the following pointers.

1. [What is a Callback Method in C#?](#)
2. [How does a callback method work in C#?](#)
3. [How to retrieve data from a thread function using a callback method in C#?](#)

How to Retrieve Data from a Thread Function in C#?

As of now, we have discussed the uses of ThreadStart and ParameterizedThreadStart delegates. If you notice, the return type of these two delegates is void as shown in the below image.

```
public delegate void ThreadStart();  
public delegate void ParameterizedThreadStart(object obj);
```

So, using the above two delegates we cannot return any data from a method as the return type is void. Then the question that comes to our mind is how to retrieve the data from a thread function in C#? The answer is by using a callback method.

How to Retrieve Data from a Thread Function using Callback Method in C#:

Let us see an example with step by step procedure to understand how we can use a callback method to retrieve the data from a thread function in C#. Please follow the below steps.

Step1:

In order to retrieve the data from a thread function, first, you need to encapsulate the thread function and the data it requires in a helper class. To the constructor of the Helper class, you need to pass the required data if any as well as a delegate representing the callback method.

From the thread function body, you need to invoke the callback delegate just before the thread function ends. And one more thing you need to take care that the callback delegate and the actual callback method signature should be the same.

So, create a class file with the **NumberHelper.cs** and then copy and paste the following code into it. The code is explained through comments, so please go through the comment lines.

```
using System;  
namespace ThreadingDemo  
{
```

```

// First Create the callback delegate with the same signature of the call
public delegate void ResultCallbackDelegate(int Results);
//Creating the Helper class
public class NumberHelper
{
    //Creating two private variables to hold the Number and ResultCallback
    private int _Number;
    private ResultCallbackDelegate _resultCallbackDelegate;
    //Initializing the private variables through constructor
    //So while creating the instance you need to pass the value for Number
callback delegate
    public NumberHelper(int Number, ResultCallbackDelegate resultCallbackI
    {
        _Number = Number;
        _resultCallbackDelegate = resultCallbackDelagate;
    }
    //This is the Thread function which will calculate the sum of the num
    public void CalculateSum()
    {
        int Result = 0;
        for (int i = 1; i <= _Number; i++)
        {
            Result = Result + i;
        }
        //Before the end of the thread function call the callback method
        if (_resultCallbackDelegate != null)
        {
            _resultCallbackDelegate(Result);
        }
    }
}
}
}

```

Step2:

In the second step, we need to create the callback method whose signature should be the same as the signature of the CallBack Delegate. In our example, ResultCallBackMethod is the callback method and its signature is the same as the signature of the ResultCallbackDelegate delegate. Within the Main method, we need to create an instance of the ResultCallbackDelegate delegate and while creating the instance we need to pass the ResultCallBackMethod as the parameter to its constructor. So when we invoke the delegate it will call the ResultCallBackMethod method.

Please modify the Program class code as shown below. The example code is self-explained. So, please go through the comment lines for a better understanding.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Create the ResultCallbackDelegate instance and to its constructor
        //pass the callback method name
        ResultCallbackDelegate resultCallbackDelegate = new
ResultCallbackDelegate(ResultCallBackMethod);
        int Number = 10;
        //Creating the instance of NumberHelper class by passing the Number
        //the callback delegate instance
        NumberHelper obj = new NumberHelper(Number, resultCallbackDelegate);
        //Creating the Thread using ThreadStart delegate
        Thread T1 = new Thread(new ThreadStart(obj.CalculateSum));

        T1.Start();
        Console.Read();
    }
    //Callback method and the signature should be the same as the callback
signature
    public static void ResultCallBackMethod(int Result)
    {
        Console.WriteLine("The Result is " + Result);
    }
}
}

```

Now run the application and you should see the output as expected.

Output: **The Result is 55**

What is a Callback Function in C#?

We can define a callback function as a function pointer that is being passed as an argument to another function. And then it is expected to call back that function at some point in time.

In our example, we call the thread function of NumberHelper class from the Main method of the Program class. While creating the instance of NumberHelper class we pass the callback function as an argument to that class constructor. And then we expected that callback method to be called at some point in time by the NumberHelper method.

In the next article, I am going to discuss the [Join and IsAlive Methods of Thread Class in C#](#) with Examples. Here, in this article, I try to explain **How to Retrieve Data from a Thread Function in C#** using the Callback Method with Examples. I hope you understood this concept and enjoy How to Retrieve Data from a Thread Function in C# using the Callback Method with Examples article.

Join Method and IsAlive Property of Thread Class in C# with Examples

In this article, I am going to discuss the use of the **Join Method and IsAlive Property of Thread Class in C#** with Examples. Please read our previous article where we discussed [How to Return Data from a Thread Function in C# using the Callback method](#) with an Example. As part of this article, we are going to discuss the following pointers.

1. [Understanding the need for Join Method of Thread Class in C#.](#)
2. [Examples using different overloaded versions of the Join Method.](#)
3. [Examples to Understand the use of IsAlive Property of Thread Class in C#.](#)

Join Method of Thread Class in C#.

Let us understand the use of the Join Method of Thread Class in C# with Examples. For a better understanding, please have a look at the following example. In the below example we have created three methods and then execute these three methods using three different threads. The point that you need to remember is the threads thread1, thread2, and thread3 are called the child threads of the main thread. This is because these three threads are created by the main thread only. Here, thread1 executes Method1, thread2 executes Method2, and thread3 executes Method3. Further, if you notice, we have delayed the execution of Method1 by 3 seconds, Method2 execution by 2 seconds, and Method3 execution by 5 seconds using the Thread class static Sleep method which takes the time in milliseconds and will make the current thread sleep for that millisecond. Here, we are using the overloaded version of the Thread class constructor which takes the ThreadStart delegate as a parameter.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Main Thread creating three child threads
            Thread thread1 = new Thread(Method1);
            Thread thread2 = new Thread(Method2);
            Thread thread3 = new Thread(Method3);
            thread1.Start();
            thread2.Start();
            thread3.Start();
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 - Thread1 Started");
            Thread.Sleep(3000);
            Console.WriteLine("Method1 - Thread 1 Ended");
        }
    }
}
```

```

    }
    static void Method2()
    {
        Console.WriteLine("Method2 - Thread2 Started");
        Thread.Sleep(2000);
        Console.WriteLine("Method2 - Thread2 Ended");
    }
    static void Method3()
    {
        Console.WriteLine("Method3 - Thread3 Started");
        Thread.Sleep(5000);
        Console.WriteLine("Method3 - Thread3 Ended");
    }
}
}

```

Now, run the application and see the output. The output may vary on your machine when you run the application.

```

Main Thread Started
Main Thread Ended
Method1 - Thread1 Started
Method3 - Thread3 Started
Method2 - Thread2 Started
Method2 - Thread2 Ended
Method1 - Thread 1 Ended
Method3 - Thread3 Ended

```

As you can see from the above output, the Main thread is not waiting for all the child threads to complete their execution or task. If you want the Main thread should not be exited until and unless all the child thread or any of the child threads completes their task then you need to use the Join method of the Thread class in C#.

Join Method of Thread Class in C#:

The Join Method of the Thread Class in C# blocks the current thread and makes it wait until the child thread on which the Join method invoked completes its execution. There are three overloaded versions available for the Join Method in Thread class as shown below.

```

public void Join();

public bool Join(int millisecondsTimeout);

public bool Join(TimeSpan timeout);

```

The following are the definitions of the above three overloaded versions of the Join Method of Thread class in C# provided by Microsoft:

1. **Join():** This method blocks the calling thread until the thread represented by this instance terminates while continuing to perform standard COM and SendMessage pumping. It will throw ThreadStateException if the caller attempted to join a thread that is in the System.Threading.ThreadState.Unstarted state.
2. **Join(int millisecondsTimeout):** This method blocks the calling thread until the thread represented by this instance terminates or the specified time elapses while continuing to perform standard COM and SendMessage pumping. The parameter millisecondsTimeout specifies the number of milliseconds to wait for the thread to terminate. It returns true if the thread has terminated; false if the thread has not terminated after the amount of time specified by the millisecondsTimeout parameter has elapsed. It will throw ArgumentOutOfRangeException if the value of millisecondsTimeout is negative and is not equal to System.Threading.Timeout.Infinite in milliseconds. It will throw ThreadStateException if the thread has not been started.
3. **Join(TimeSpan timeout):** This method blocks the calling thread until the thread represented by this instance terminates or the specified time elapses while continuing to perform standard COM and SendMessage pumping. Here, the parameter timeout specifies a System.TimeSpan is set to the amount of time to wait for the thread to terminate. It returns true if the thread is terminated; false if the thread has not terminated after the amount of time specified by the timeout parameter has elapsed. It throws ArgumentOutOfRangeException if the value of timeout is negative and is not equal to System.Threading.Timeout.Infinite in milliseconds, or greater than System.Int32.MaxValue milliseconds. It throws ThreadStateException if the caller attempted to join a thread that is in the System.Threading.ThreadState.Unstarted state.

We can simplify the above definitions as follows:

Join(): The first version of the Join method which does not take any parameter will block the calling thread (i.e. the Parent thread) until the thread (child thread) completes its execution. In this case, the calling thread is going to wait for an indefinite time until the thread on which the Join Method is invoked is completed.

Join(int millisecondsTimeout): The second version of the Join Method allows us to specify the time out. It means it will block the calling thread until the child thread terminates or the specified time elapses. This overloaded takes time in milliseconds. This method returns true if the thread has terminated and returns false if the thread has not terminated after the amount of time specified by the millisecondsTimeout parameter has elapsed.

Join(TimeSpan timeout): The third overloaded version of this method is the same as the second overloaded version. The only difference is that here we need to use TimeSpan to set the amount of time to wait for the thread to terminate.

Example to Understand the Join Method of Thread Class in C#

For a better understanding of how to use the Thread Class Join Method in C#, please have a look at the below example. In the below example, we have called the Join method on all three threads which means it will block the main thread until all the child threads complete their tasks.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

        Console.WriteLine("Main Thread Started");
        //Main Thread creating three child threads
        Thread thread1 = new Thread(Method1);
        Thread thread2 = new Thread(Method2);
        Thread thread3 = new Thread(Method3);
        thread1.Start();
        thread2.Start();
        thread3.Start();
        thread1.Join(); //Block Main Thread until thread1 completes its execution
        thread2.Join(); //Block Main Thread until thread2 completes its execution
        thread3.Join(); //Block Main Thread until thread3 completes its execution
        Console.WriteLine("Main Thread Ended");
        Console.Read();
    }
    static void Method1()
    {
        Console.WriteLine("Method1 - Thread1 Started");
        Thread.Sleep(1000);
        Console.WriteLine("Method1 - Thread 1 Ended");
    }
    static void Method2()
    {
        Console.WriteLine("Method2 - Thread2 Started");
        Thread.Sleep(2000);
        Console.WriteLine("Method2 - Thread2 Ended");
    }
    static void Method3()
    {
        Console.WriteLine("Method3 - Thread3 Started");
        Thread.Sleep(5000);
        Console.WriteLine("Method3 - Thread3 Ended");
    }
}
}

```

Now, run the above code and you will see that the Main thread is started first and will wait until all the child threads complete their execution as shown in the below output.

```

Main Thread Started
Method1 - Thread1 Started
Method3 - Thread3 Started
Method2 - Thread2 Started
Method1 - Thread 1 Ended
Method2 - Thread2 Ended
Method3 - Thread3 Ended
Main Thread Ended

```

Now, for example, if you don't want the main thread to wait until thread3 completes its execution. Then you just need to call the Join method on thread1 and thread2 as shown in the below example.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Main Thread creating three child threads
            Thread thread1 = new Thread(Method1);
            Thread thread2 = new Thread(Method2);
            Thread thread3 = new Thread(Method3);
            thread1.Start();
            thread2.Start();
            thread3.Start();
            thread1.Join(); //Block Main Thread until thread1 completes its execution
            thread2.Join(); //Block Main Thread until thread2 completes its execution
            //Now, Main Thread will not wait for thread3 to complete its execution
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 - Thread1 Started");
            Thread.Sleep(1000);
            Console.WriteLine("Method1 - Thread 1 Ended");
        }
        static void Method2()
        {
            Console.WriteLine("Method2 - Thread2 Started");
            Thread.Sleep(2000);
            Console.WriteLine("Method2 - Thread2 Ended");
        }
        static void Method3()
        {
            Console.WriteLine("Method3 - Thread3 Started");
            Thread.Sleep(5000);
            Console.WriteLine("Method3 - Thread3 Ended");
        }
    }
}

```

Output:

```
Main Thread Started
Method3 - Thread3 Started
Method1 - Thread1 Started
Method2 - Thread2 Started
Method1 - Thread 1 Ended
Method2 - Thread2 Ended
Main Thread Ended
Method3 - Thread3 Ended
```

Other Overloaded Versions of Thread Class Join Method in C#:

You need to use the second and third overloaded version of the Thread Class Join Method in C# when you want the main thread to wait for a specified amount of time. For example, you want the main thread to wait for 3 seconds for thread2 and thread3 to complete their task. Then you need to use the Join method as shown below in the below example. Remember the overloaded versions which take time in milliseconds and TimeSpan returns a boolean value indicating whether the thread completes its execution or not. Boolean true means method execution completed and false means method execution not completed.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Main Thread creating three child threads
            Thread thread1 = new Thread(Method1);
            Thread thread2 = new Thread(Method2);
            Thread thread3 = new Thread(Method3);
            thread1.Start();
            thread2.Start();
            thread3.Start();

            //Now, Main Thread will block for 3 seconds and wait thread2 to c
execution
            if (thread2.Join(TimeSpan.FromSeconds(3)))
            {
                Console.WriteLine("Thread 2 Execution Completed in 3 second");
            }
            else
            {
                Console.WriteLine("Thread 2 Execution Not Completed in 3 secon
            }
            //Now, Main Thread will block for 3 seconds and wait thread3 to c
execution
            if (thread3.Join(3000))
            {
```

```

        Console.WriteLine("Thread 3 Execution Completed in 3 second");
    }
    else
    {
        Console.WriteLine("Thread 3 Execution Not Completed in 3 second");
    }
    Console.WriteLine("Main Thread Ended");
    Console.Read();
}
static void Method1()
{
    Console.WriteLine("Method1 - Thread1 Started");
    Thread.Sleep(1000);
    Console.WriteLine("Method1 - Thread 1 Ended");
}
static void Method2()
{
    Console.WriteLine("Method2 - Thread2 Started");
    Thread.Sleep(2000);
    Console.WriteLine("Method2 - Thread2 Ended");
}
static void Method3()
{
    Console.WriteLine("Method3 - Thread3 Started");
    Thread.Sleep(5000);
    Console.WriteLine("Method3 - Thread3 Ended");
}
}
}

```

Now, run the application and observe the output. You will see that Method2 execution is completed in 3 seconds while Method3 execution is not completed. Further notice after the Main method execution, Method3 is going to complete its execution. So, the Main thread is blocked for 3 seconds and after 3 seconds the Main thread continues its execution.

```

Main Thread Started
Method2 - Thread2 Started
Method1 - Thread1 Started
Method3 - Thread3 Started
Method1 - Thread 1 Ended
Method2 - Thread2 Ended
Thread 2 Execution Completed in 3 second
Thread 3 Execution Not Completed in 3 second
Main Thread Ended
Method3 - Thread3 Ended

```

IsAlive Property of Thread Class in C#:

The IsAlive property gets a value indicating the execution status of the current thread. It returns true if the thread has been started and has not terminated normally or aborted; otherwise, false. That means the IsAlive

property of the Thread class returns true if the thread is still executing else returns false. Let us understand this with an example.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            Thread thread1 = new Thread(Method1);
            thread1.Start();
            if (thread1.IsAlive)
            {
                Console.WriteLine("Thread1 Method1 is still Executing");
            }
            else
            {
                Console.WriteLine("Thread1 Method1 Completed its work");
            }
            //Wait Till thread1 to complete its execution
            thread1.Join();
            if (thread1.IsAlive)
            {
                Console.WriteLine("Thread1 Method1 is still Executing");
            }
            else
            {
                Console.WriteLine("Thread1 Method1 Completed its work");
            }
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 - Thread1 Started");
            //Making thread to sleep for 2 seconds
            Thread.Sleep(TimeSpan.FromSeconds(2));
            Console.WriteLine("Method1 - Thread 1 Ended");
        }
    }
}
```

Output:

```
Main Thread Started  
Thread1 Method1 is still Executing  
Method1 - Thread1 Started  
Method1 - Thread 1 Ended  
Thread1 Method1 Completed its work  
Main Thread Ended
```

In the next article, I am going to discuss [Thread Synchronization in C#](#) with Examples. Here, in this article, I try to explain **how to use the IsAlive Property and Join Method of Thread class in C#** using Examples. I hope you enjoy this IsAlive Property and Join Method of Thread class in C# using the Examples article.

Thread Synchronization in C# with Examples

In this article, I am going to discuss **Thread Synchronization in C#** with Examples. Please read our previous article where we discussed the significance of the [IsAlive Property and Join Method of Thread Class in C#](#) with Examples. At the end of this article, you will understand what is thread synchronization and how you can implement thread synchronization to protect shared resources in multithread applications with real-time examples.

What is Thread Synchronization in C#?

Data inconsistency occurs when more than one threads access a shared resource such as in-memory data (instance or class variables) and external objects such as files at the same time. Let us understand this with an example. Consider that we have two threads Thread1 and Thread2, and both the threads access a shared resource let's say Resource1 simultaneously. If Thread1 is trying to read data from the shared resource Resource1 when Thread2 is attempting to write data onto the shared resource Resource1, then there would be data inconsistency. Hence, in situations like this thread synchronization comes into the picture.

Synchronization in C# language is a process that allows access to shared resources smoothly. Synchronization in C# ensures that only one thread is accessing the shared resource at any given point in time, preventing other threads from doing the same at the same time.

Thread Synchronization in C# is a mechanism that is used to restrict multiple threads from accessing a shared resource at the same time. In simple words, we can also say that thread synchronization can help us to prevent multiple threads from gaining access to a shared resource simultaneously. As a result, we can have one and only one thread entering a critical section to access the shared resource at any given point in time.

How Thread Synchronization is Achieved in C#?

Synchronization in C# can be achieved in multiple ways. One of the ways to achieve Synchronization in C# is by using the feature of lock, which locks the access to a block of code within the locked object. When a thread locks an object, no other thread can access the block of code within the locked object. Only when a thread releases the lock, then it is available for other threads to access it.

In C# Language, every object has a built-in lock. By using the feature of Synchronization, we can lock an object. Locking an object can be done by using the lock keyword, and the following is the syntax to use the lock.

```
lock(object)
{
    //Statement1
    //Statement2
    //And more statements to be synchronized
}
```

So, when a thread acquires a lock over an object, then that particular thread can only access the block of statements within the locked object. Now, all the other threads wishing to access the same block of statements within the same locked object will have to wait until, the thread that has got the lock on the object, releases the lock, by exiting the block of statements.

Example without Thread Synchronization in C#:

Before we show you the example of how to use the synchronization between threads by locking an object and its practical use, let us first see what actually happens without using synchronization on executing multiple threads, which are trying to access the same resource.

In the below example, we are creating three different threads that are going to access the same resource i.e. in this case the shared resource is `SomeMethod`. As you can see, we have delayed the `SomeMethod` execution by 1 second, and all three threads trying to execute the same method and they will execute it. The first thread which executes the method does not get its sole access, this thread executes the method for a while and after some time, other threads will come and execute the same method. So, here, we will not get the output as expected.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            Thread thread2 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            Thread thread3 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            thread1.Start();
            thread2.Start();
            thread3.Start();
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            Console.Write("[Welcome To The ");
            Thread.Sleep(1000);
            Console.WriteLine("World of Dotnet!"]);
        }
    }
}
```

When I run the above code, I am getting the following output. The output might be varied in your machine.

```
[Welcome To The [Welcome To The [Welcome To The World of Dotnet!]
World of Dotnet!]
World of Dotnet!]
```

As you can see in the above output, here we are not getting the output as expected. So, the point that you need to keep in mind is that if the shared resource is not protected in a multithreaded environment from concurrent access, then the output or the behavior of the application becomes inconsistent.

Example using Thread Synchronization in C#

In the below example, we are creating three threads that are going to access the SomeMethod, but this time access to SomeMethod will be synchronized because we are going to use the lock, to lock the object within which the method is going to be accessed by multiple threads. The first thread to enter the method gets its sole access until it exits the method, thereby avoiding the collision between multiple threads trying to access a method.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            Thread thread2 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            Thread thread3 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            thread1.Start();
            thread2.Start();
            thread3.Start();
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            // Locking the Shared Resource for Thread Synchronization
            lock (lockObject)
            {
                Console.Write("[Welcome To The ");
                Thread.Sleep(1000);
                Console.WriteLine("World of Dotnet!]");
            }
        }
    }
}
```

```

    }
}
}

```

Output:

```

[Welcome To The World of Dotnet!]
[Welcome To The World of Dotnet!]
[Welcome To The World of Dotnet!]

```

The first thread enters the SomeMethod locked the method (code written within the lock (object)), and gets its sole access and once this thread has finished its execution of the method, then only another thread will come and acquire a lock on the critical section i.e. the code written within the lock object. In this way, the lock object will make sure that at any given point in time, only one thread can access the shared resource which will also ensure data consistency.

Realtime Example to Understand Thread Synchronization in C#:

Let us see one real-time example to understand thread synchronization and why it is important to protect shared resources in our application. In the below example, I am going to show you how to protect a shared variable from concurrent access in a multithread environment.

In the below example, we are implementing the ticket booking functionality for a movie. Let us assume the number of available tickets is 3 and three different threads tried to book tickets. Thread1 tries to book 1 ticket, thread2 tries to book 2 tickets and thread3 tries to book 3 tickets. Let us first see the problem without thread synchronization. In the below code, we are not implementing thread synchronization and hence sometime you will see that all three threads are able to book tickets. See, the available tickets are 3 and we are able to book 6 tickets and this is the issue without thread synchronization.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            BookMyShow bookMyShow = new BookMyShow();
            Thread t1 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread1"
            };
            Thread t2 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread2"
            };
            Thread t3 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread3"
            }

```

```

        };

        t1.Start();
        t2.Start();
        t3.Start();
        Console.ReadKey();
    }
}

public class BookMyShow
{
    int AvailableTickets = 3;
    static int i = 1, j = 2, k = 3;
    public void BookTicket(string name, int wantedtickets)
    {
        if (wantedtickets <= AvailableTickets)
        {
            Console.WriteLine(wantedtickets + " booked to " + name);
            AvailableTickets = AvailableTickets - wantedtickets;
        }
        else
        {
            Console.WriteLine("No tickets Available to book");
        }
    }
    public void TicketBookig()
    {
        string name = Thread.CurrentThread.Name;
        if (name.Equals("Thread1"))
        {
            BookTicket(name, i);
        }
        else if (name.Equals("Thread2"))
        {
            BookTicket(name, j);
        }
        else
        {
            BookTicket(name, k);
        }
    }
}
}

```

Output:

```

2 booked to Thread2
1 booked to Thread1
3 booked to Thread3

```

As you can see in the above output, all the threads are able to book the tickets. This is possible because all the threads are able to access the critical section code of the program simultaneously. Now, let us proceed and see how we can restrict this i.e. how we can allow only one thread to execute the critical section code.

Real-Time Example using Thread Synchronization in C#

In the below example, we have used the thread synchronization mechanism to lock the critical section code by using the lock object. Now, the lock object will make sure that only one thread can execute the critical section code, and once the thread completes the execution of the critical section code, then another thread can enter and execute the critical section code. With this, now you will get the output as expected and you will see that you cannot book more than 3 tickets.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            BookMyShow bookMyShow = new BookMyShow();
            Thread t1 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread1"
            };
            Thread t2 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread2"
            };
            Thread t3 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread3"
            };

            t1.Start();
            t2.Start();
            t3.Start();
            Console.ReadKey();
        }
    }
    public class BookMyShow
    {
        private object lockObject = new object();
        int AvailableTickets = 3;
        static int i = 1, j = 2, k = 3;
        public void BookTicket(string name, int wantedtickets)
        {
            lock(lockObject)
            {
                if (wantedtickets <= AvailableTickets)
```

```

        {
            Console.WriteLine(wantedtickets + " booked to " + name);
            AvailableTickets = AvailableTickets - wantedtickets;
        }
        else
        {
            Console.WriteLine("No tickets Available to book");
        }
    }
}
public void TicketBookig()
{
    string name = Thread.CurrentThread.Name;
    if (name.Equals("Thread1"))
    {
        BookTicket(name, i);
    }
    else if (name.Equals("Thread2"))
    {
        BookTicket(name, j);
    }
    else
    {
        BookTicket(name, k);
    }
}
}
}
}

```

Output:

```

1 booked to Thread1
2 booked to Thread2
No tickets Available to book

```

So, by using the feature of synchronization (lock), we can avoid a conflict between threads trying to access the same resource simultaneously. We can achieve thread synchronization in C# by using the followings. From our next article onwards, we are going to discuss the following in detail.

1. **Lock**
2. **Monitor**
3. **Mutex**
4. **Semaphore**
5. **SemaphoreSlim**

Note: Thread Synchronization in C# is a mechanism that ensures that two or more concurrent processes or threads do not execute some particular section of the program, especially the critical section. In this technique, one thread executes the critical section of a program and the other thread waits until the thread finished its execution. If a proper synchronization mechanism will be not applied then race conditions will happen.

Why do we need Thread Synchronization in Multithreading?

We need Thread Synchronization in Multithreading because of the following:

1. **Atomicity:** Thread Synchronization supports atomicity, which ensures that multiple threads in the application are not allowed to access a shared resource concurrently to prevent data inconsistency. The code section of our program which causes data inconsistency is known as the critical section. The critical section of our program is executed atomically by one and only one thread which ensures Atomicity. The Ticket booking example is an example of Atomicity.
2. **Ordering:** We generally want two or more threads to perform a task in a particular order or we want to restrict access to shared resources to a particular number of threads only. Usually, we don't have much control over all this, which is one reason for race conditions. Thread synchronization provides support for ordering so that you can have control over your threads to perform the tasks as per your requirement. The first example that we discussed in this article is the example of ordering.

What is Exclusive Lock and Non-Exclusive Lock in C#?

When a process or a thread wants to access an object, it requests a lock on that object. There are two types of locks that determine access to shared resources – Exclusive Lock and Non-Exclusive lock.

1. **Exclusive Lock:** An exclusive lock makes sure that only one thread can gain access or enter a critical section at any given point in time. In C#, we can implement Exclusive Lock using the lock keyword, Monitor class, Mutex Class, and SpinLock class.
2. **Non-Exclusive Lock:** Non-Exclusive locks provide read-only access to a shared resource and limit concurrency, i.e., limit the number of concurrent accesses to a shared resource. In C#, we can implement Non-Exclusive Lock using the Semaphore, SemaphoreSlim, and ReaderWriterLockSlim classes.

In the next article, I am going to discuss [Thread Synchronization using Lock in C#](#) with Examples. Here, in this article, I try to explain **Thread Synchronization in C#** with Examples. I hope you enjoy this Thread Synchronization in C# with Examples article.

Thread Synchronization Using Lock in C#

In this article, I am going to discuss **Thread Synchronization using Lock in C#** with Examples. Please read our previous article where we discussed [Thread Synchronization in C#](#) with Examples.

1. [Why do we need to Protect the Shared Resources in Multithreading in C#?](#)
2. [Accessing a Shared Resource in a Single-Threaded Environment in C#](#)
3. [Accessing a Shared Resource in a Multithreaded Environment in C#](#)
4. [How to Protect a Shared Resource in a Multithread Environment from Concurrent Access in C#?](#)
5. [What is the Lock Statement in C#?](#)
6. [How is the Lock Statement internally work in C#?](#)
7. [Protecting Shared Variable using Lock Statement in C# with Example](#)

Why do we need to Protect the Shared Resources in Multithreading in C#?

In a multithreading application, it is very important for us to handle multiple threads for executing critical section code or you can say shared resources. For example, if we have a shared resource, and multiple threads want to access the shared resource, then we need to protect the shared resource from concurrent access otherwise we will get some inconsistent behavior or output.

In C#, we can use Lock and Monitor to provide thread safety in a multithreaded application. Both lock and monitor provide a mechanism that ensures that only one thread is executing the critical section code at any given point in time to avoid any functional breaking of code or to avoid inconsistent behavior or output.

In this article, I am going to discuss how to protect the shared resource in a multithread environment using the lock and in the next article, I am going to discuss how to do the same thing using the [Monitor in C#](#).

Accessing a Shared Resource in a Single-Threaded Environment in C#:

Before understanding how to use the lock to protect the shared resource in a multithread environment in C#, let us first understand the problem if we are not protecting the shared resource in a multithread environment. In the following example, we have a shared resource i.e. DisplayMessage() method and we call that method three times from the Main method as shown below.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            DisplayMessage();
            DisplayMessage();
            DisplayMessage();
            Console.Read();
        }
        static void DisplayMessage()
```

```

    {
        Console.Write("[Welcome to the ");
        Thread.Sleep(1000);
        Console.WriteLine("world of dotnet!"]);
    }
}

```

Output:

```

[Welcome to the world of dotnet]
[Welcome to the world of dotnet]
[Welcome to the world of dotnet]

```

As the above program is a single-threaded program, so we got the output as expected. Here, the shared resource is the DisplayMessage method which is going to be executed three times sequentially, and hence we got the correct output. Now, let us proceed and see what happens if we access the shared resources in a multithreaded environment.

Accessing a Shared Resource in a Multithreaded Environment in C#:

In the following example, we have created three different threads and then invoke the same DisplayMessage() method using all these three different threads. Here, DisplayMessage() method is the shared resource and this shared resource is simultaneously or concurrently invoked by three different threads. Here, we are not protecting the shared resource, and all three threads accessing the shared resource which results in inconsistent output.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t1 = new Thread(DisplayMessage);
            Thread t2 = new Thread(DisplayMessage);
            Thread t3 = new Thread(DisplayMessage);
            t1.Start();
            t2.Start();
            t3.Start();
            Console.Read();
        }
        static void DisplayMessage()
        {
            Console.Write("[Welcome to the ");
            Thread.Sleep(1000);
            Console.WriteLine("world of dotnet!"]);
        }
    }
}

```

```
}  
}
```

Output:

```
[Welcome to the [Welcome to the [Welcome to the world of dotnet!]
world of dotnet!]
world of dotnet!]
```

As you can see, here we are not getting the output as expected. So, the point that you need to keep in mind is that if the shared resource is not protected in a multithreaded environment from concurrent access, then the output or the behavior of the application becomes inconsistent.

How to Protect a Shared Resource in a Multithread Environment from Concurrent Access in C#?

We can protect the shared resources in a multithread environment from concurrent access by using the concept Monitor and Locking in C#. Let us see how to protect the shared resource using the lock statement in C# and see the output. In the following example, we have created one readonly object i.e LockDisplayMethod, and then we created a block using the lock keyword. To the lock keyword, we pass the LockDisplayMethod object, and the section or block or particular resource that we want to protect should be placed inside the lock block which is shown in the below example.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t1 = new Thread(DisplayMessage);
            Thread t2 = new Thread(DisplayMessage);
            Thread t3 = new Thread(DisplayMessage);
            t1.Start();
            t2.Start();
            t3.Start();
            Console.Read();
        }
        private static readonly object LockDisplayMethod = new object();
        static void DisplayMessage()
        {
            lock(LockDisplayMethod)
            {
                Console.Write("[Welcome to the ");
                Thread.Sleep(1000);
                Console.WriteLine("world of dotnet!"]");
            }
        }
    }
}
```

```
}  
}  
}
```

Now run the application and see the output as expected as shown below. As you can see in the below output, we are getting the result as expected and this is because we are now protecting the shared resource using the lock statement which will ensure that only one thread will be able to access the critical section at any given point in time.

```
[Welcome to the world of dotnet!]  
[Welcome to the world of dotnet!]  
[Welcome to the world of dotnet!]
```

What is the lock statement in C#?

According to [Microsoft](#), the lock statement acquires the mutual-exclusion lock for a given object, executes a statement block, and then releases the lock. While a lock is held, the thread that holds the lock can again acquire and release the lock. Any other thread is blocked from acquiring the lock and waits until the lock is released.

Note: When you want to synchronize thread access to a shared resource, you should lock the shared resource on a dedicated object instance (for example, `private readonly object _lockObject = new object();` or `private static readonly object _lockObject = new object();`). Avoid using the same lock object instance for different shared resources, as it might result in a deadlock.

How is the lock Statement internally work in C#?

The lock statement in C# was internally converted to a try-finally block when we compiled the code. The compiled code of the lock statement will look like the below. You can see, it is internally using the Monitor class Enter and Exit method. In our next article, we will discuss the [Monitor Class Enter and Exit Methods](#) in detail, for now, to understand, what we can say is, it acquires an exclusive lock within the try block by invoking the Monitor class Enter method and it releases the exclusive lock within the finally block by calling the Monitor class Exit method.

```
bool lockWasTaken = false;
var temp = new object;
try
{
    Monitor.Enter(temp, ref lockWasTaken);
    // body
}
finally
{
    if (lockWasTaken)
    {
        Monitor.Exit(temp);
    }
}
```

You can also verify the IL Code by using the ILDASM tool. Now, if you open the application using the ILDASAM tool and if you verify the DisplayMessage method IL Code, then you can see the try and finally blocks along with the Monitor class Enter and Exit methods as shown in the below image.

```

.method private hidebysig static void DisplayMessage() cil managed
{
    // Code size          67 (0x43)
    .maxstack 2
    .locals init ([0] object U_0,
                  [1] bool U_1)
    IL_0000:  nop
    IL_0001:  ldsfld      object ThreadingDemo.Program::LockDisplayMethod
    IL_0006:  stloc.0
    IL_0007:  ldc.i4.0
    IL_0008:  stloc.1
    .try
    {
        IL_0009:  ldloc.0
        IL_000a:  ldloca.s     U_1
        IL_000c:  call         void [mscorlib]System.Threading.Monitor::Enter(object,
                                                                bool&)

        IL_0011:  nop
        IL_0012:  nop
        IL_0013:  ldstr        "[Welcome to the "
        IL_0018:  call         void [mscorlib]System.Console::Write(string)
        IL_001d:  nop
        IL_001e:  ldc.i4       0x3e8
        IL_0023:  call         void [mscorlib]System.Threading.Thread::Sleep(int32)
        IL_0028:  nop
        IL_0029:  ldstr        "world of dotnet!]"
        IL_002e:  call         void [mscorlib]System.Console::WriteLine(string)
        IL_0033:  nop
        IL_0034:  nop
        IL_0035:  leave.s      IL_0042
    } // end .try
    finally
    {
        IL_0037:  ldloc.1
        IL_0038:  brfalse.s   IL_0041
        IL_003a:  ldloc.0
        IL_003b:  call         void [mscorlib]System.Threading.Monitor::Exit(object)
        IL_0040:  nop
        IL_0041:  endfinally
    } // end handler
    IL_0042:  ret
} // end of method Program::DisplayMessage

```

Protecting Shared Variable using Lock Statement in C# with Examples:

The section or block or particular resource that you want to protect should be placed inside the lock block. Let us understand this with an example. In the below example, we are only protecting the shared Count variable from concurrent access.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static int Count = 0;
    }
}

```

```

static void Main(string[] args)
{
    Thread t1 = new Thread(IncrementCount);
    Thread t2 = new Thread(IncrementCount);
    Thread t3 = new Thread(IncrementCount);
    t1.Start();
    t2.Start();
    t3.Start();
    //Wait for all three threads to complete their execution
    t1.Join();
    t2.Join();
    t3.Join();
    Console.WriteLine(Count);
    Console.Read();
}
private static readonly object LockCount = new object();
static void IncrementCount()
{
    for (int i = 1; i <= 1000000; i++)
    {
        //Only protecting the shared Count variable
        lock (LockCount)
        {
            Count++;
        }
    }
}
}
}
}

```

When you run the above program, it will give you the output as expected as 3000000. Now, let us see what happens if we are not protecting our shared variable Count. In the below example, we are not protecting the Count shared variable and hence all three threads simultaneously access the variable and try to increment the value, and hence we will get some unexpected output.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static int Count = 0;
        static void Main(string[] args)
        {
            Thread t1 = new Thread(IncrementCount);
            Thread t2 = new Thread(IncrementCount);
            Thread t3 = new Thread(IncrementCount);
            t1.Start();
            t2.Start();
            t3.Start();
            //Wait for all three threads to complete their execution

```

```

        t1.Join();
        t2.Join();
        t3.Join();
        Console.WriteLine(Count);
        Console.Read();
    }

    static void IncrementCount()
    {
        for (int i = 1; i <= 1000000; i++)
        {
            Count++;
        }
    }
}

```

Every time, you run the application, you will get a different output. So, it is important for us to protect our shared resources in a multithreaded application, or else we will not get the expected output.

Here, in this article, I try to explain **Thread Synchronization using Lock in C#** with Examples. In the next article, I am going to discuss [How to Protect Shared Resources in a Multithread Environment using Monitor Class in C#](#) with Examples. I hope you enjoy this Thread Synchronization using Lock in C# with Examples article.

Monitor Class in C# with Examples

In this article, I will discuss **How to Protect Shared Resources in Multithreading using Monitor Class in C#** with Examples. Please read our previous article discussing [How to Protect shared Resources using Lock in C#](#) from Concurrent Access with Examples. As part of this article, we are going to discuss the following pointers.

1. [Understanding the Monitor Class in C#](#)
2. [Example to understand Monitor Class in C# to Protect Shared Resource from Concurrent Access](#)
3. [How does the Monitor Class work in C#?](#)
4. [Example to Understand Monitor.Enter\(lockObject, ref IslockTaken\) Method in C#](#)
5. [Example to Understand TryEnter\(Object, TimeSpan, Boolean\) Method of Monitor Class in C#](#)
6. [Example to Understand Wait\(\) and Pulse\(\) Methods of Monitor Class in C#](#)
7. [Difference Between Monitor and Lock in C#](#)
8. [Limitations of Locks and Monitors in C#](#)

We have already discussed that both Monitor and Lock are used to provide thread safety to a shared resource in a multithreaded application in C#. In our previous article, we have seen how to use the lock mechanism to achieve thread safety in a multi-thread environment. Now, let us proceed and try to understand the Monitor Class and its methods in detail to understand how to protect the shared resource using the monitor class in C# with Examples.

Understanding the Monitor Class in C#:

According to [Microsoft](#), the Monitor Class in C# Provides a mechanism that synchronizes access to objects. Let us simplify the above definition. In simple words, we can say that, like the lock, we can also use this Monitor Class to protect the shared resources in a multi-threaded environment from concurrent access. This can be done by acquiring an exclusive lock on the object so that only one thread can enter the critical section at any given point in time.

The Monitor class is a static class belonging to the System.Threading namespace. As a static class, it provides a collection of static methods, as shown in the image below. Using these static methods, we can provide synchronized access to the critical section associated with a particular object.

```

namespace System.Threading
{
    ...public static class Monitor
    {
        ...public static void Enter(object obj);
        ...public static void Enter(object obj, ref bool lockTaken);
        ...public static void Exit(object obj);
        ...public static bool IsEntered(object obj);
        ...public static void Pulse(object obj);
        ...public static void PulseAll(object obj);
        ...public static bool TryEnter(object obj, TimeSpan timeout);
        ...public static void TryEnter(object obj, int millisecondsTimeout, ref bool lockTaken);
        ...public static void TryEnter(object obj, ref bool lockTaken);
        ...public static bool TryEnter(object obj);
        ...public static bool TryEnter(object obj, int millisecondsTimeout);
        ...public static void TryEnter(object obj, TimeSpan timeout, ref bool lockTaken);
        ...public static bool Wait(object obj, int millisecondsTimeout, bool exitContext);
        ...public static bool Wait(object obj, TimeSpan timeout, bool exitContext);
        ...public static bool Wait(object obj, int millisecondsTimeout);
        ...public static bool Wait(object obj, TimeSpan timeout);
        ...public static bool Wait(object obj);
    }
}

```

Let's understand the roles and responsibilities of each method in the Monitor class. The following is the list of important methods in the Monitor class.

1. **Enter():** When we invoke the Enter method of the Monitor class, it acquires an exclusive lock on the specified object. This also marks the beginning of a critical section or the beginning of a shared resource.
2. **Exit():** When the Monitor class's Exit method is invoked, it releases the lock on the specified object. This marks the end of a critical section or the end of the shared resource protected by the locked object.
3. **Pulse():** When the Pulse method is invoked of the Monitor class, it sends a signal to a thread in the waiting queue of a change in the locked object's state.
4. **Wait():** When the Monitor class's Wait method is invoked, it releases the lock on an object and blocks the current thread until it reacquires the lock.
5. **PulseAll():** When the Monitor class's PulseAll method is invoked, it sends signals to all waiting threads about a change in the locked object's state.
6. **TryEnter():** When we invoke the Monitor class's TryEnter method, it attempts to acquire an exclusive lock on the specified object.

Note: If this is not clear at the moment, don't worry. We will try to understand all the above methods with examples, and you will also see that many overloaded versions of the above methods are available.

Example to understand Monitor Class in C# to Protect Shared Resource from Concurrent Access:

The following is the syntax for using the Enter and Exit methods of the Monitor class to protect a shared resource in a multithreaded environment from concurrent access in C#. All the methods of the Monitor class are static. So, you can see here that we are accessing the Enter and Exit methods using the class name, i.e., Monitor.

```

object lockObject = new object();

try
{
    //Trying to Enter Into the Critical Section

    Monitor.Enter(lockObject);
    //Access the Shared Resource Here
}
finally
{
    //Release the lock or Exit from the Critical Section
    Monitor.Exit(lockObject);
}

```

Let us see an example to understand how to use the Monitor class Enter and Exit method to protect a shared resource in a multithreaded environment in C# from concurrent access. In the example below, we have one shared resource, and we are accessing that resource concurrently using three different threads. Then, we used the Monitor class Enter and Exit Methods to protect the critical section code. In this case, all three threads will try to acquire an exclusive lock, but at any given point in time, only one thread gets an exclusive lock and will enter into the critical section, and all other threads will wait until the thread releases the lock.

```

using System;
using System.Threading;
namespace MonitorDemo
{
    class Program
    {
        private static readonly object lockPrintNumbers = new object();
        public static void PrintNumbers()
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Trying to enter in
critical section");

            try
            {
                Monitor.Enter(lockPrintNumbers);
                Console.WriteLine(Thread.CurrentThread.Name + " Entered into 1
section");

                for (int i = 0; i < 5; i++)
                {
                    Thread.Sleep(100);
                    Console.Write(i + ",");
                }
                Console.WriteLine();
            }
        }
    }
}

```

```

    }
    finally
    {
        Monitor.Exit(lockPrintNumbers);
        Console.WriteLine(Thread.CurrentThread.Name + " Exit from critical
section");
    }
}
static void Main(string[] args)
{
    Thread[] Threads = new Thread[3];
    for (int i = 0; i < 3; i++)
    {
        Threads[i] = new Thread(PrintNumbers)
        {
            Name = "Child Thread " + i
        };
    }
    foreach (Thread t in Threads)
    {
        t.Start();
    }
    Console.ReadLine();
}
}
}

```

Output:

```

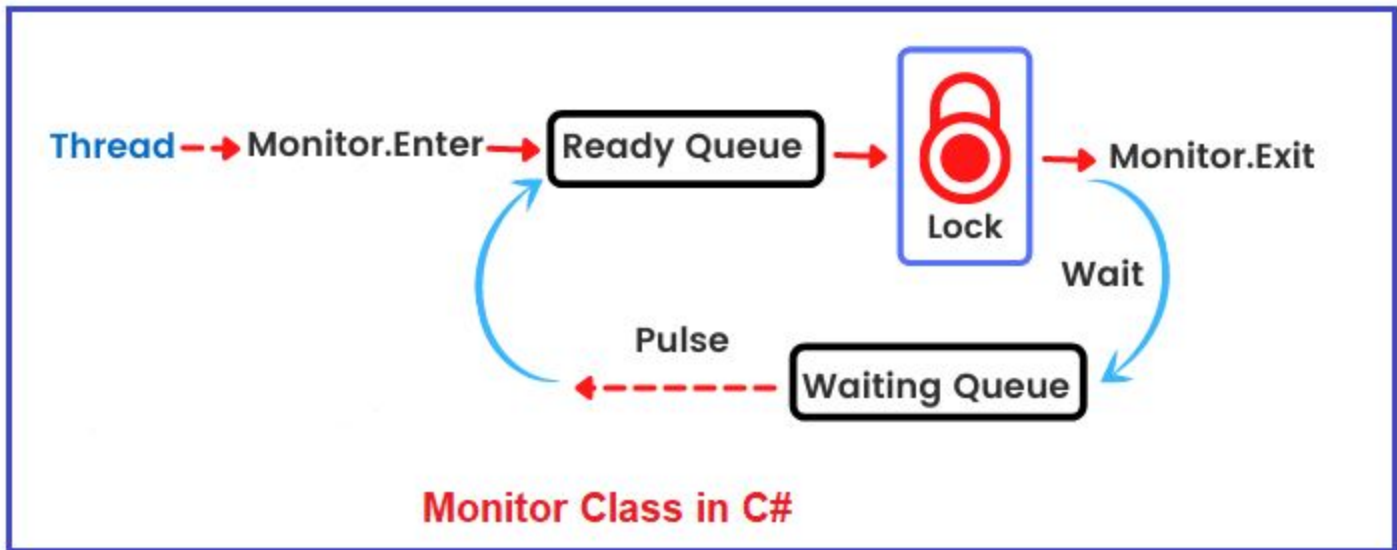
Child Thread 0 Trying to enter into the critical section
Child Thread 0 Entered into the critical section
Child Thread 2 Trying to enter into the critical section
Child Thread 1 Trying to enter into the critical section
0,1,2,3,4,
Child Thread 0 Exit from critical section
Child Thread 2 Entered into the critical section
0,1,2,3,4,
Child Thread 1 Entered into the critical section
Child Thread 2 Exit from critical section
0,1,2,3,4,
Child Thread 1 Exit from critical section

```

How does the Monitor Class work in C#?

The Monitor class in C# provides a wait-based synchronization mechanism that allows only one thread to access the critical section code at a time to avoid the race condition. All the other threads must wait and halt the execution until the locked object is released.

To understand how the Monitor class works in C#, please have a look at the following diagram. As shown in the image below, as soon as a thread executes the Enter method of the Thread class, it will be in the Ready Queue, and in the same way, many threads can be in the Ready Queue. Then, one of the threads from the Ready Queue will acquire an Exclusive Lock on the Object and will enter inside the Critical Section and execute the code, and at that time, no other threads can get a chance to enter into the Critical Section. When we execute the Exit method of the thread class, the currently executing thread will move into the waiting queue and send one signal to the threads in the ready queue. One of the Threads from the Ready queue will acquire the lock, and enter the Critical section, and start executing the code of the Critical Section. This is how the Monitor class works in C#.



Monitor.Enter(lockObject, ref IslockTaken) Method in C#:

Let us understand the other overloaded version of the Enter method. The **Monitor.Enter(lockObject, ref IslockTaken)** acquires an exclusive lock on the specified object. It then automatically sets a value that indicates whether the lock was taken or not. The second parameter, which is a Boolean parameter, returns true if the lock is acquired; otherwise, it returns false. The syntax for using this overloaded version is given below.

```

private static readonly object lockObject = new object();
bool IsLockTaken = false;

try
{
    //Trying to Enter into the Critical Section
    Monitor.Enter(lockObject, ref IsLockTaken);
    if(IsLockTaken)
    {
        //Entered Into the Critical Section
        //Access the Shared Resource
    }
}
finally
{
    //Exit From Critical Section
    if (IsLockTaken)
    {
        Monitor.Exit(lockObject);
    }
}

```

The following example shows how to use **Enter(lockObject, ref IslockTaken)** method of the Monitor class in C#. The following example is the same as the previous example, except here, we are using the overloaded version of the Enter method, which takes two parameters. The second boolean parameter specifies whether the thread acquires a lock or not, true indicates that it acquires a lock on the object and false indicates that it does not acquire a lock on the object and again in the finally block we are checking the boolean value and accordingly we are releasing the lock.

```

using System;
using System.Threading;
namespace MonitorDemo
{
    class Program
    {
        private static readonly object lockPrintNumberst = new object();
        public static void PrintNumbers()
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Trying to enter in
critical section");
            bool IsLockTaken = false;

            try
            {
                Monitor.Enter(lockPrintNumberst, ref IsLockTaken);

```

```

        if(IsLockTaken)
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Entered ir
critical section");
            for (int i = 0; i < 5; i++)
            {
                Thread.Sleep(100);
                Console.Write(i + ",");
            }
            Console.WriteLine();
        }
    }
    finally
    {
        if (IsLockTaken)
        {
            Monitor.Exit(lockPrintNumberst);
            Console.WriteLine(Thread.CurrentThread.Name + " Exit from
section");
        }
    }
}
static void Main(string[] args)
{
    Thread[] Threads = new Thread[3];
    for (int i = 0; i < 3; i++)
    {
        Threads[i] = new Thread(PrintNumbers)
        {
            Name = "Child Thread " + i
        };
    }
    foreach (Thread t in Threads)
    {
        t.Start();
    }
    Console.ReadLine();
}
}
}

```

Output:

```
Child Thread 0 Trying to enter into the critical section
Child Thread 0 Entered into the critical section
Child Thread 1 Trying to enter into the critical section
Child Thread 2 Trying to enter into the critical section
0,1,2,3,4,
Child Thread 0 Exit from critical section
Child Thread 1 Entered into the critical section
0,1,2,3,4,
Child Thread 1 Exit from critical section
Child Thread 2 Entered into the critical section
0,1,2,3,4,
Child Thread 2 Exit from critical section
```

Example to Understand TryEnter(Object, TimeSpan, Boolean) Method of Monitor Class in C#:

This method attempts to acquire an exclusive lock on the specified object for a specified amount of time. It automatically sets a value that indicates whether the lock was taken or not. The syntax for using the **TryEnter(Object, TimeSpan, Boolean)** Method of Monitor Class in C# is given below.

```
var lockObj = new object();
var timeout = TimeSpan.FromMilliseconds(500);
bool lockTaken = false;

try {
    Monitor.TryEnter(lockObj, timeout, ref lockTaken);
    if (lockTaken) {
        // The critical section.
    }
    else {
        // The lock was not acquired.
    }
}
finally {
    // Ensure that the lock is released.
    if (lockTaken) {
        Monitor.Exit(lockObj);
    }
}
```

For a better understanding, please have a look at the example below, which shows how to use the TryEnter(Object, TimeSpan, Boolean) Method of the Monitor Class in C#. In the example below, we specified the timeout as 1000 milliseconds, or you can say 1 second. If the thread does not acquire the lock within 1 second, it will not enter the critical section.

```

using System;
using System.Threading;
namespace MonitorDemo
{
    class Program
    {
        private static readonly object lockPrintNumbers = new object();
        public static void PrintNumbers()
        {
            TimeSpan timeout = TimeSpan.FromMilliseconds(1000);
            bool lockTaken = false;
            try
            {
                Console.WriteLine(Thread.CurrentThread.Name + " Trying to enter critical section");
                Monitor.TryEnter(lockPrintNumbers, timeout, ref lockTaken);
                if (lockTaken)
                {
                    Console.WriteLine(Thread.CurrentThread.Name + " Entered in critical section");
                    for (int i = 0; i < 5; i++)
                    {
                        Thread.Sleep(100);
                        Console.Write(i + ",");
                    }
                    Console.WriteLine();
                }
                else
                {
                    // The lock was not acquired.
                    Console.WriteLine(Thread.CurrentThread.Name + " Lock was not acquired");
                }
            }
            finally
            {
                // To Ensure that the lock is released.
                if (lockTaken)
                {
                    Monitor.Exit(lockPrintNumbers);
                    Console.WriteLine(Thread.CurrentThread.Name + " Exit from section");
                }
            }
        }

        static void Main(string[] args)
        {
            Thread[] Threads = new Thread[3];
            for (int i = 0; i < 3; i++)
            {
                Threads[i] = new Thread(PrintNumbers)

```

```

        {
            Name = "Child Thread " + i
        };
    }
    foreach (Thread t in Threads)
    {
        t.Start();
    }
    Console.ReadLine();
}
}
}

```

When you run the above code, you will get the following output. The output may vary on your machine. As you can see, all three threads try to acquire a lock on the object within 1 second. Two of the three threads acquire an exclusive lock on the object, while one is unable to acquire an exclusive lock, and hence, that thread will not enter the critical section.

```

Child Thread 1 Trying to enter into the critical section
Child Thread 1 Entered into the critical section
Child Thread 0 Trying to enter into the critical section
Child Thread 2 Trying to enter into the critical section
0,1,2,3,4,
Child Thread 1 Exit from critical section
Child Thread 2 Entered into the critical section
0,1,2,3,Child Thread 0 Lock was not acquired
4,
Child Thread 2 Exit from critical section

```

Example to Understand Wait() and Pulse() Methods of Monitor Class in C#:

The Wait() Method of Monitor Class is used to Release the lock on an object in order to permit other threads to lock and access the object. The calling thread waits while another thread accesses the object. The Pulse signals are used to notify waiting threads about changes to a locked object's state.

Let us understand this with one real-time example. Our business requirement is to print the even and Odd number sequence using two different threads. One thread will print the even numbers, and another thread will print the odd numbers.

Thread T1: 0,2,4,6,8...

Thread T2: 1,3,5,7,9...

Output: 0,1,2,3,4,5,6,7,8,9...

To solve the above problem, let us use the signaling mechanism using the Monitor Class Wait() and Pulse() Methods in C#. In the following example, we use the Monitor.Wait() method to make the thread waiting and Monitor.Pulse() method is used to signal other threads. The process is as follows:

1. First, the Even thread will start to print the number on the console.
2. Then, the Even thread will signal the Odd thread to be ready to print the number using the Monitor.Pulse() method.

3. Then, the Event thread will call the Monitor.Wait() method, which will allow the current thread to block and the Odd thread to start execution.
4. The Odd Thread will also do the same thing.
5. The Odd thread will start to print the number on the console.
6. Then, the Odd thread will signal the Even thread to be ready to print the number using the Monitor.Pulse() method.
7. Then the Odd thread will call the Monitor.Wait() method, which will allow the current thread to block and allow the Even thread to start execution.
8. The same process is going on.

Since both the Odd and Even threads share the same console window to print the number, we need to put a lock on the console IO. We want the sequence to start with the even number, so the Even thread must run first. Once we start the Even thread, we need to pause for a moment before starting the Odd thread using the Sleep() method of the Thread class in C# to avoid any chance of starting the Odd thread first.

```
using System;
using System.Threading;
namespace odd_even_sequence
{
    class Program
    {
        //Upto the limit numbers will be printed on the Console
        const int numberLimit = 20;
        static readonly object _lockMonitor = new object();
        static void Main(string[] args)
        {
            Thread EvenThread = new Thread(PrintEvenNumbers);
            Thread OddThread = new Thread(PrintOddNumbers);
            //First Start the Even thread.
            EvenThread.Start();
            //Puase for 10 ms, to make sure Even thread has started
            //or else Odd thread may start first resulting different sequence.
            Thread.Sleep(100);
            //Next, Start the Odd thread.
            OddThread.Start();
            //Wait for all the child's threads to complete
            OddThread.Join();
            EvenThread.Join();
            Console.WriteLine("\nMain method completed");
            Console.ReadKey();
        }
        //Printing of Even Numbers Function
        static void PrintEvenNumbers()
        {
            try
            {
                //Implement lock as the Console is shared between two threads
                Monitor.Enter(_lockMonitor);
                for (int i = 0; i <= numberLimit; i = i + 2)
                {
                    //Printing Even Number on Console)
                }
            }
        }
    }
}
```

```

        Console.Write($"{i} ");
        //Notify Odd thread that I'm done, you do your job
        //It notifies a thread in the waiting queue of a change in
        //locked object's state.
        Monitor.Pulse(_lockMonitor);
        //I will wait here till Odd thread notify me
        //Monitor.Wait(monitor);
        //Without this logic application will wait forever
        bool isLast = false;
        if (i == numberLimit)
        {
            isLast = true;
        }
        if (!isLast)
        {
            //I will wait here till Odd thread notify me
            //Releases the lock on an object and blocks the current
            //until it reacquires the lock.
            Monitor.Wait(_lockMonitor);
        }
    }
}
finally
{
    //Release the lock
    Monitor.Exit(_lockMonitor);
}
}
//Printing of Odd Numbers Function
static void PrintOddNumbers()
{
    try
    {
        //Hold lock as the Console is shared between two threads
        Monitor.Enter(_lockMonitor);
        for (int i = 1; i <= numberLimit; i = i + 2)
        {
            //Printing the odd numbers on the console
            Console.Write($"{i} ");
            //Notify Even thread that I'm done, you do your job
            Monitor.Pulse(_lockMonitor);
            // I will wait here till even thread notify me
            // Monitor.Wait(monitor);
            // without this logic application will wait forever
            bool isLast = false;
            if (i == numberLimit - 1)
            {
                isLast = true;
            }
            if (!isLast)
            {

```

```

        //I will wait here till Even thread notify me
        Monitor.Wait(_lockMonitor);
    }
}
finally
{
    //Release lock
    Monitor.Exit(_lockMonitor);
}
}
}
}

```

Output:

```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Main method completed

```

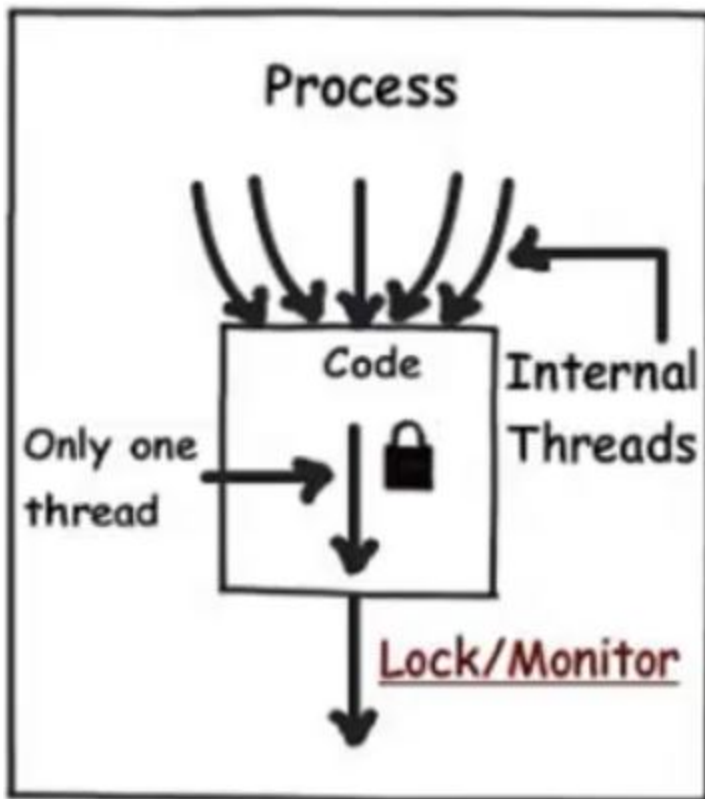
Difference Between Monitor and Lock in C#

The Difference between monitor and lock in C# is that lock internally wraps the Enter and Exit methods in a try...finally block with exception handling. For the Monitor class in C#, we need to use the try and finally block explicitly to release the lock properly. So, Lock = Monitor + try-finally.

The lock provides the basic functionality to acquire an exclusive lock on a synchronized object. But, If you want more control to implement advanced multithreading solutions using TryEnter(), Wait(), Pulse(), and PulseAll() methods, then the Monitor class is your option.

Limitations of Locks and Monitors in C#:

Locks and Monitors help us ensure that our code is thread-safe. That means when we run our code in a multi-threaded environment, we don't end up with inconsistent results. For a better understanding, please have a look at the image below.



However, there are some limitations to locks and monitors. The locks and monitors ensure thread safety for threads that are in process, i.e., the threads that are generated by the application itself, i.e., Internal Threads. But if the threads come from external applications (Out-Process) or External Threads, then Locks and Monitors have no control over them. So, in a situation like this, we need to use Mutex. In our next article, we will discuss Mutex.

Monitor class Methods in Detail:

Let us understand the Roles and Responsibilities of each method of the Monitor class according to Microsoft.

1. **Enter(object obj):** This method acquires an exclusive lock on the specified object. It takes one object parameter to acquire the monitor lock. If the parameter obj is null, it will throw an `ArgumentNullException`.
2. **Enter(object obj, ref bool lockTaken):** This method also acquires an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken. Here, the parameter obj specifies the object on which to wait. The parameter lockTaken specifies the result of the attempt to acquire the lock passed by reference. The input must be false. The output is true if the lock is acquired; otherwise, the output is false. The output is set even if an exception occurs during the attempt to acquire the lock. If no exception occurs, the output of this method will always be true. It will throw `ArgumentException` if the input to lockTaken is true. It will throw `ArgumentNullException` if the obj parameter is null.

TryEnter Methods:

There are six overloaded versions of the TryEnter method available in the Monitor class. They are as follows:

1. **public static bool TryEnter(object obj, TimeSpan timeout):** Attempts, for the specified amount of time, to acquire an exclusive lock on the specified object.

2. **public static void TryEnter(object obj, int millisecondsTimeout, ref bool lockTaken):** Attempts, for the specified number of milliseconds, to acquire an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken.
3. **public static void TryEnter(object obj, ref bool lockTaken):** Attempts to acquire an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken.
4. **public static bool TryEnter(object obj):** Attempts to acquire an exclusive lock on the specified object.
5. **public static bool TryEnter(object obj, int millisecondsTimeout):** Attempts, for the specified number of milliseconds, to acquire an exclusive lock on the specified object.
6. **public static void TryEnter(object obj, TimeSpan timeout, ref bool lockTaken):** Attempts, for the specified amount of time, to acquire an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken.

All these methods are also used to acquire an exclusive lock on the specified object. Further, if you notice, all these methods return a type of bool. So, the TryEnter() method returns true if the current thread acquires the lock; otherwise, false. The following are the parameters used in the TryEnter method.

1. **object obj:** All the six overloaded versions take one object type parameter which specifies the object on which to acquire the lock. If the object parameter this method takes is null, then it will throw ArgumentNullException.
2. **TimeSpan timeout:** Some TryEnter() methods take TimeSpan timeout as a parameter, and this parameter specifies a System.TimeSpan represents the amount of time to wait for the lock. A value of -1 millisecond specifies an infinite wait. It will throw ArgumentOutOfRangeException if the value of timeout in milliseconds is negative and is not equal to System.Threading.Timeout.Infinite (-1 millisecond), or is greater than System.Int32.MaxValue.
3. **int millisecondsTimeout:** Again, two overloaded versions take int millisecondsTimeout as a parameter and this parameter specifies the number of milliseconds to wait for the lock. It will throw ArgumentOutOfRangeException if millisecondsTimeout is negative, and not equal to System.Threading.Timeout.Infinite.
4. **ref bool lockTaken:** Also three overloaded versions take ref bool lockTaken as a parameter and this parameter specifies the result of the attempt to acquire the lock, passed by reference. The input must be false. The output is true if the lock is acquired; otherwise, the output is false. The output is set even if an exception occurs during the attempt to acquire the lock. It will ArgumentException if the input to lockTaken is true.

Note: Both Enter and TryEnter methods are used to acquire an exclusive lock for an object. This action marks the beginning of a critical section. No other thread can enter the critical section unless it executes the instructions in the critical section using a different locked object.

Wait Methods of Monitor Class in C#:

There are five overloaded versions of the Wait method available in the Monitor class. They are as follows:

1. **public static bool Wait(object obj):** It Releases the lock on an object and blocks the current thread until it reacquires the lock.
2. **public static bool Wait(object obj, TimeSpan timeout):** Releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue.
3. **public static bool Wait(object obj, int millisecondsTimeout):** It releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue.

4. **public static bool Wait(object obj, TimeSpan timeout, bool exitContext):** It releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue. Optionally exits the synchronization domain for the synchronized context before the wait and reacquires the domain afterward.
5. **public static bool Wait(object obj, int millisecondsTimeout, bool exitContext):** It releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue. This method also specifies whether the synchronization domain for the context (if in a synchronized context) is exited before the wait and reacquired afterward.

All these Wait Methods are used to release the lock on an object and block the current thread until it reacquires the lock. All these methods return type is boolean. So, these methods return true if the call is returned because the caller reacquired the lock for the specified object. This method does not return if the lock is not reacquired. Following are the parameters used in the Wait method.

1. **object obj:** The object on which to wait. If the obj parameter is null, it will throw an ArgumentNullException.
2. **TimeSpan timeout:** A System.TimeSpan represents the amount of time to wait before the thread enters the ready queue. It will throw ArgumentOutOfRangeException if the value of the timeout parameter in milliseconds is negative and does not represent System.Threading.Timeout.Infinite (-1 millisecond), or is greater than System.Int32.MaxValue.
3. **int millisecondsTimeout:** The number of milliseconds to wait before the thread enters the ready queue. If the value of the millisecondsTimeout parameter is negative and not equal to System.Threading.Timeout.Infinite, it will throw ArgumentOutOfRangeException.
4. **bool exitContext:** true to exit and reacquire the synchronization domain for the context (if in a synchronized context) before the wait; otherwise, false.
5. **ref bool lockTaken:** The result of the attempt to acquire the lock, passed by reference. The input must be false. The output is true if the lock is acquired; otherwise, the output is false. The output is set even if an exception occurs during the attempt to acquire the lock.

Note: The Wait methods are used to release the lock on an object and permit other threads to lock and access the object by blocking the current thread until it reacquires the lock. The calling thread waits while another thread accesses the object. Pulse signals are used to notify waiting threads about changes to an object's state.

Pulse and PulseAll Method of Monitor Class in C#:

The following two methods are used to send a signal to one or more waiting threads. The signal notifies a waiting thread that the state of the locked object has changed, and the owner of the lock is ready to release the lock.

1. **Pulse(object obj):** This method notifies a thread in the waiting queue of a change in the locked object's state. The obj parameter specifies the object a thread is waiting for. If the obj parameter is null, then it will throw ArgumentNullException.
2. **PulseAll(object obj):** This method notifies all waiting threads of a change in the object's state. The obj parameter specifies the object that sends the pulse. If the obj parameter is null, then it will throw ArgumentNullException.

Exit():

The Exit method is used to release the exclusive lock from the specified object. This action marks the end of a critical section protected by the locked object.

1. **Exit(object obj):** This method releases an exclusive lock on the specified object. The parameter obj specifies the object on which to release the lock. If the obj parameter is null, it will throw an ArgumentNullException.

IsEntered() Method:

1. **IsEntered(object obj):** This function determines whether the current thread holds the lock on the specified object. The parameter obj specifies the object to test. It returns true if the current thread holds the lock on obj; otherwise, it returns false. If the obj is null, it will throw an ArgumentNullException.

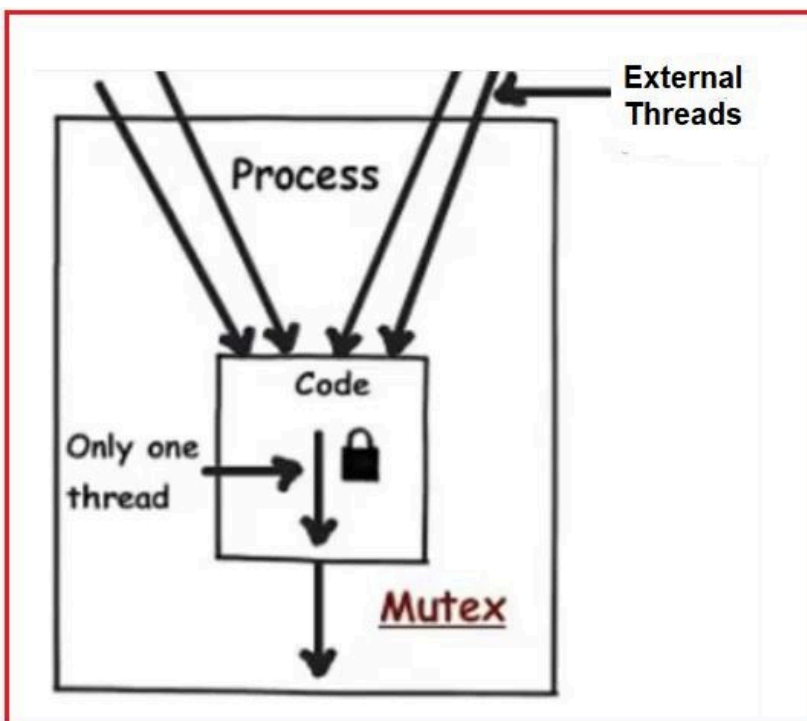
In the next article, I will discuss the [Mutex in C#](#) with Examples. In this article, I try to explain how to protect shared resources in multithreading using the Monitor class from Concurrent Access in C# with examples. I hope you enjoy this article.

Mutex Class in C# with Example

In this article, I am going to discuss **Mutex Class in C#** i.e. How to use Mutex in C# to Protect Shared Resources in Multithreading with Examples. Please read our previous article where we discussed [How to Protect Shared Resources in Multithreading using Monitor Class in C#](#) with Examples.

Why Mutex as we already have Lock and Monitor for Thread Safety?

Mutex also helps us to ensure that our code is thread-safe. That means when we run our code in a multi-threaded environment then we don't end up with inconsistent results. The Locks and Monitors ensure thread safety for threads that are InProcess i.e. the threads that are generated by the application itself i.e. Internal Threads. But if the threads are coming from OutProcess i.e. from external applications then Locks and Monitors have no control over them. Whereas Mutex ensures thread safety for threads that are Out-Process i.e. the threads that are generated by the external applications i.e. External Threads.

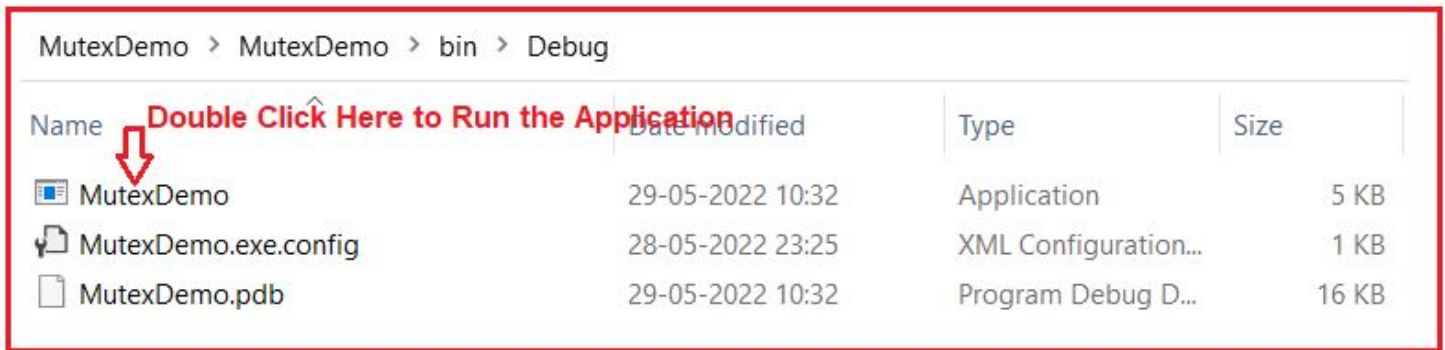


Let us first understand what is External Process Or External Threads and then we will understand Mutex in C#. Let us first create a console application and then copy and paste the below code into it.

```
using System;
namespace MutexDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Application Is Running.....");
            Console.ReadKey();
        }
    }
}
```

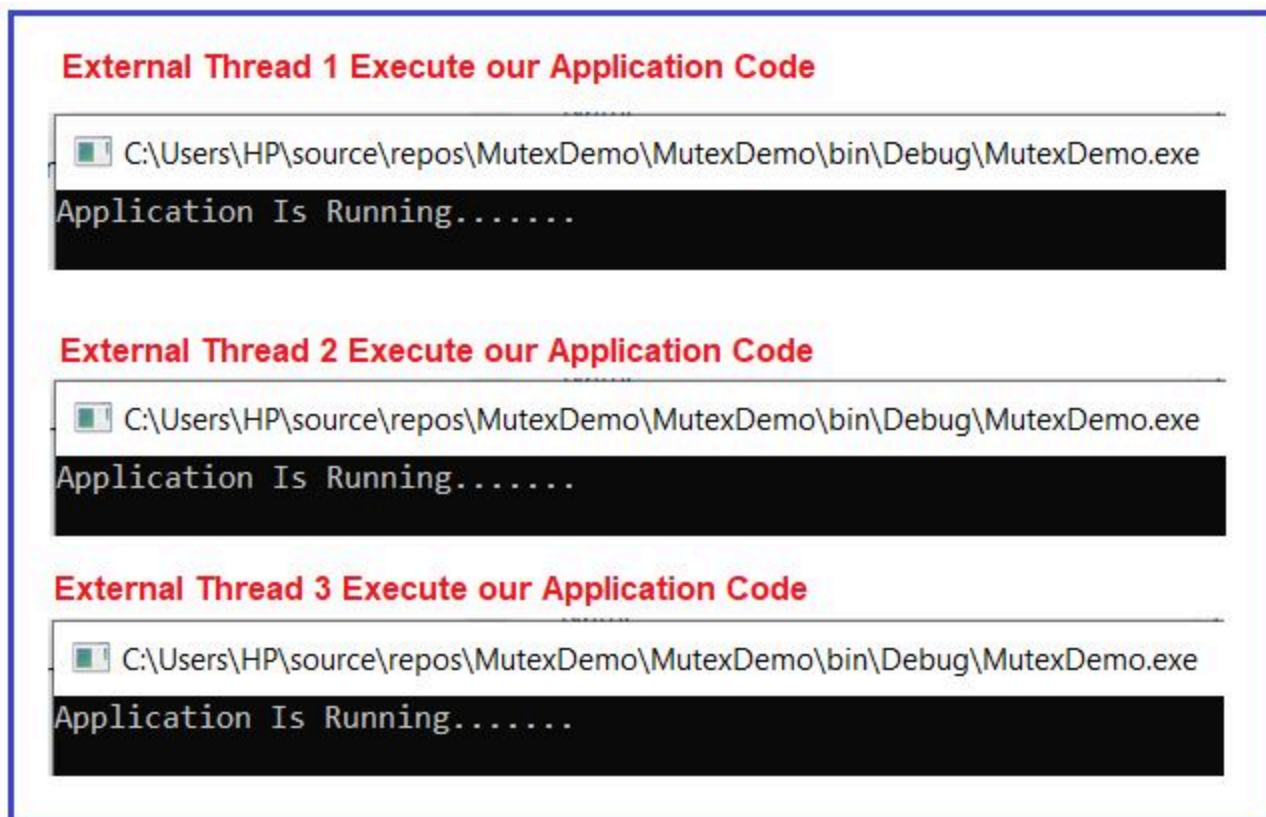
}

Now, build the project and go to the project's bin\Debug directory and there you will find the application EXE file as shown in the below image.



MutexDemo > MutexDemo > bin > Debug			
Name	Date Modified	Type	Size
 MutexDemo	29-05-2022 10:32	Application	5 KB
 MutexDemo.exe.config	28-05-2022 23:25	XML Configuration...	1 KB
 MutexDemo.pdb	29-05-2022 10:32	Program Debug D...	16 KB

When you double-click on the Application EXE file, an external thread will execute the application code. And now if you double-click multiple times, then each time a new external thread will be created and execute our application code as shown in the below image. I double-click on the EXE three times, so three external threads are accessing our application at the same time.



External Thread 1 Execute our Application Code

```
C:\Users\HP\source\repos\MutexDemo\MutexDemo\bin\Debug\MutexDemo.exe
Application Is Running.....
```

External Thread 2 Execute our Application Code

```
C:\Users\HP\source\repos\MutexDemo\MutexDemo\bin\Debug\MutexDemo.exe
Application Is Running.....
```

External Thread 3 Execute our Application Code

```
C:\Users\HP\source\repos\MutexDemo\MutexDemo\bin\Debug\MutexDemo.exe
Application Is Running.....
```

But, if you want to restrict that only external thread access to our application code at any given point in time, then we need to use the Mutex in C#. Now, let us rewrite the previous example using the C# Mutex Class and see what happens when we try to access the application code multiple times from outside using External Threads, and then we will discuss the C# Mutex Class in detail.

Modify the code Program.cs class file as follows. So, when one external thread accesses our code, no more external threads can access the code. The following example exactly does the same using C# Mutex Class.

This is one of the use cases of Mutex in C#. Don't go with the code, I will explain the code later part of this article.

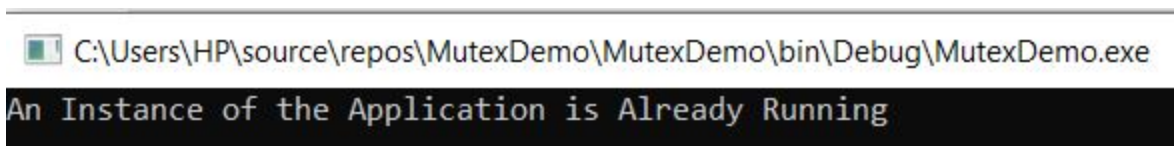
```
using System;
using System.Threading;
namespace MutexDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using(Mutex mutex = new Mutex(false, "MutexDemo"))
            {
                //Checking if Other External Thread is Running
                if(!mutex.WaitOne(5000, false))
                {
                    Console.WriteLine("An Instance of the Application is Already Running");

                    Console.ReadKey();
                    return;
                }
                Console.WriteLine("Application Is Running.....");
                Console.ReadKey();
            }
        }
    }
}
```

Now close all the instances which are running. Then build the project and again go to the projects bin\Debug directory and again click on the Exe file for the first time. You will get the following result.



Now, again click on the EXE file. This time it will wait for 5 seconds and then it will give you the following message. This ensures that only one external thread can access our application code at any given point in time.



Now, I hope you understand the need for Mutex Class in C#. Let us proceed further and try to understand the Mutex Class in C# with multiple examples.

What is Mutex Class in C#?

Mutex works like a lock i.e. acquired an exclusive lock on a shared resource from concurrent access, but it works across multiple processes. As we already discussed exclusive locking is basically used to ensure that at any given point in time, only one thread can enter into the critical section.

When two or more threads need to access a shared resource at the same time, the system needs a synchronization mechanism to ensure that only one thread at a time uses the resource. Mutex is a synchronization mechanism that grants exclusive access to the shared resource to only one external thread. If a thread acquires a mutex, the second thread that wants to acquire that mutex is suspended until the first thread releases the mutex. If this is not clear at the moment then don't worry, we try to understand this with some examples.

Constructors, and Methods of Mutex Class in C#:

Let us understand the different Constructors, and Methods of the Mutex Class in C#. If you right-click on the Mutex class and select go to definition, then you will see the following. As you can see, Mutex is a sealed class and it is inherited from WaitHandle class. As it is a sealed class, so no further inheritance is possible i.e. no class can be derived from this sealed Mutex class in C#.

```
namespace System.Threading
{
    ...public sealed class Mutex : WaitHandle
    {
        ...public Mutex();
        ...public Mutex(bool initiallyOwned);
        ...public Mutex(bool initiallyOwned, string name);
        ...public Mutex(bool initiallyOwned, string name, out bool createdNew);
        ...public Mutex(bool initiallyOwned, string name, out bool createdNew, MutexSecurity mutexSecurity);

        ...public static Mutex OpenExisting(string name);
        ...public static Mutex OpenExisting(string name, MutexRights rights);
        ...public static bool TryOpenExisting(string name, out Mutex result);
        ...public static bool TryOpenExisting(string name, MutexRights rights, out Mutex result);
        ...public MutexSecurity GetAccessControl();
        ...public void ReleaseMutex();
        ...public void SetAccessControl(MutexSecurity mutexSecurity);
    }
}
```

Constructors of Mutex Class in C#:

The Mutex Class in C# provides the following five constructors that we can use to create an instance of the Mutex class.

1. **Mutex()**: It initializes a new instance of the Mutex class with default properties.
2. **Mutex(bool initiallyOwned)**: It initializes a new instance of the Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex.
3. **Mutex(bool initiallyOwned, string name)**: It initializes a new instance of the System.Threading.Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex, and a string that is the name of the mutex.

4. **Mutex(bool initiallyOwned, string name, out bool createdNew):** It initializes a new instance of the System.Threading.Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex, a string that is the name of the mutex, and a Boolean value that, when the method returns, indicates whether the calling thread was granted initial ownership of the mutex.
5. **Mutex(bool initiallyOwned, string name, out bool createdNew, MutexSecurity mutexSecurity):** It initializes a new instance of the System.Threading.Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex, a string that is the name of the mutex, and a Boolean variable that, when the method returns, indicates whether the calling thread was granted initial ownership of the mutex, and the access control security to be applied to the named mutex.

Following are the parameters used in the Constructors of Mutex Class in C#.

1. **initiallyOwned:** true to give the calling thread initial ownership of the named system mutex if the named system mutex is created as a result of this call; otherwise, false.
2. **name:** The name of the Mutex. If the value is null, the Mutex is unnamed.
3. **createdNew:** When this method returns, contains a Boolean that is true if a local mutex was created (that is, if the name is null or an empty string) or if the specified named system mutex was created; false if the specified named system mutex already existed. This parameter is passed uninitialized.
4. **mutexSecurity:** A System.Security.AccessControl.MutexSecurity object that represents the access control security to be applied to the named system mutex.

Methods of Mutex Class in C#:

The Mutex Class in C# provides the following methods.

1. **OpenExisting(string name):** This method is used to open the specified named mutex if it already exists. It returns an object that represents the named system mutex. Here, the parameter name specifies the name of the system mutex to open. It will throw ArgumentException if the name is an empty string. -or- name is longer than 260 characters. It will throw ArgumentNullException if the name is null.
2. **OpenExisting(string name, MutexRights rights):** This method is used to open the specified named mutex, if it already exists, with the desired security access. It returns an object that represents the named system mutex. Here, the parameter name specifies the name of the system mutex to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access.
3. **TryOpenExisting(string name, out Mutex result):** This method is used to open the specified named mutex, if it already exists, and returns a value that indicates whether the operation succeeded. Here, the parameter name specifies the name of the system mutex to open. When this method returns, the result contains a Mutex object that represents the named mutex if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named mutex was opened successfully; otherwise, false.
4. **TryOpenExisting(string name, MutexRights rights, out Mutex result):** This method is used to open the specified named mutex, if it already exists, with the desired security access, and returns a value that indicates whether the operation succeeded. Here, the parameter name specifies the name of the system mutex to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access. When this method returns, the result contains a Mutex object that represents the named mutex if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named mutex was opened successfully; otherwise, false.
5. **ReleaseMutex():** This method is used to release Mutex once.

6. **GetAccessControl():** This method gets a System.Security.AccessControl.MutexSecurity object that represents the access control security for the named mutex. It returns a System.Security.AccessControl.MutexSecurity object that represents the access control security for the named mutex.
7. **SetAccessControl(mutexSecurity):** This method is used to set the access control security for a named system mutex. The parameter mutexSecurity specifies a System.Security.AccessControl.MutexSecurity object that represents the access control security to be applied to the named system mutex.

Note: The **Mutex** Class in C# is inherited from **WaitHandle** class and the **WaitHandle** class provides the **WaitOne()** method which we need to call to lock the resource. Note that a Mutex can only be released from the same thread which obtained it.

Example to Understand Mutex in C# to Protect Shared Resource in Multithreading:

The following example shows how a local Mutex object is used to synchronize access to a protected resource. Because each calling thread is blocked until it acquires ownership of the mutex, it must call the ReleaseMutex method to release ownership of the mutex. The code is self-explained. So, please go through the comment lines.

```
using System;
using System.Threading;
namespace MutexDemo
{
    class Program
    {
        private static Mutex mutex = new Mutex();
        static void Main(string[] args)
        {
            //Create multiple threads to understand Mutex
            for (int i = 1; i <= 5; i++)
            {
                Thread threadObject = new Thread(MutexDemo)
                {
                    Name = "Thread " + i
                };
                threadObject.Start();
            }
            Console.ReadKey();
        }
        //Method to implement synchronization using Mutex
        static void MutexDemo()
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Wants to Enter Critical
Section for processing");
            try
            {
                //Blocks the current thread until the current WaitOne method returns
                signal.
                //Wait until it is safe to enter.
            }
        }
    }
}
```

```

        mutex.WaitOne();
        Console.WriteLine("Success: " + Thread.CurrentThread.Name + "
Processing now");
        Thread.Sleep(2000);
        Console.WriteLine("Exit: " + Thread.CurrentThread.Name + " is
its task");
    }
    finally
    {
        //Call the ReleaseMutex method to unblock so that other thread
//that are trying to gain ownership of the mutex can enter
        mutex.ReleaseMutex();
    }
}
}
}

```

Output:

```

Thread 1 Wants to Enter Critical Section for processing
Success: Thread 1 is Processing now
Thread 3 Wants to Enter Critical Section for processing
Thread 2 Wants to Enter Critical Section for processing
Thread 4 Wants to Enter Critical Section for processing
Thread 5 Wants to Enter Critical Section for processing
Exit: Thread 1 is Completed its task
Success: Thread 3 is Processing now
Exit: Thread 3 is Completed its task
Success: Thread 5 is Processing now
Exit: Thread 5 is Completed its task
Success: Thread 4 is Processing now
Exit: Thread 4 is Completed its task
Success: Thread 2 is Processing now
Exit: Thread 2 is Completed its task

```

Note: The Mutex Class is inherited from WaitHandle abstract class and the WaitHandle abstract class implements the IDisposable interface. So, when you have finished using the type (in this case Mutex class), you should dispose of it either directly or indirectly. To dispose of the type directly, call its Dispose method in a try/catch/finally block, even though you can also call it from the destructor. To dispose of it indirectly, use a language construct such as using block in C#.

In the following example, each thread calls the WaitOne(Int32) method to acquire the mutex. If the time-out interval elapses, the method returns false, and the thread neither acquires the mutex nor gains access to the resource. The ReleaseMutex method is called only by the thread that acquires the mutex. In the below example, we are calling the Dispose method from the destructor.

```

using System;
using System.Threading;
namespace MutexDemo

```

```

{
    class Program
    {
        private static Mutex mutex = new Mutex();
        static void Main(string[] args)
        {
            //Create multiple threads to understand Mutex
            for (int i = 1; i <= 3; i++)
            {
                Thread threadObject = new Thread(MutexDemo)
                {
                    Name = "Thread " + i
                };
                threadObject.Start();
            }
            Console.ReadKey();
        }
        //Method to implement synchronization using Mutex
        static void MutexDemo()
        {
            // Wait until it is safe to enter, and do not enter if the request
            Console.WriteLine(Thread.CurrentThread.Name + " Wants to Enter Cri
Section for processing");
            if (mutex.WaitOne(1000))
            {
                try
                {
                    Console.WriteLine("Success: " + Thread.CurrentThread.Name
Processing now");
                    Thread.Sleep(2000);
                    Console.WriteLine("Exit: " + Thread.CurrentThread.Name + '
Completed its task");
                }
                finally
                {
                    //Call the ReleaseMutex method to unblock so that other th
                    //that are trying to gain ownership of the mutex can enter
                    mutex.ReleaseMutex();
                    Console.WriteLine(Thread.CurrentThread.Name + " Has Releas
mutex");
                }
            }
            else
            {
                Console.WriteLine(Thread.CurrentThread.Name + " will not acqui
mutex");
            }
        }
    }
    ~Program()
    {
        mutex.Dispose();
    }
}

```

```

    }
}
}

```

Output:

```

Thread 3 Wants to Enter Critical Section for processing
Success: Thread 3 is Processing now
Thread 1 Wants to Enter Critical Section for processing
Thread 2 Wants to Enter Critical Section for processing
Thread 1 will not acquire the mutex
Thread 2 will not acquire the mutex
Exit: Thread 3 is Completed its task
Thread 3 Has Released the mutex

```

OpenExisting Method Example of Mutex Class in C#:

In the below example, we use the OpenExisting method of the Mutex Class in C#. If this method throws an exception, the specified named Mutex does not exist or is inaccessible. The IsSingleInstance method uses this behavior. First, create a console application and copy and paste the following code.

```

using System;
using System.Threading;
namespace MutexDemo
{
    class Program
    {
        static Mutex _mutex;

        static void Main()
        {
            //If IsSingleInstance returns true continue with the Program else
            if (!IsSingleInstance())
            {
                Console.WriteLine("More than one instance"); // Exit program.
            }
            else
            {
                Console.WriteLine("One instance"); // Continue with program.
            }
            // Stay Open.
            Console.ReadLine();
        }
        static bool IsSingleInstance()
        {
            try
            {

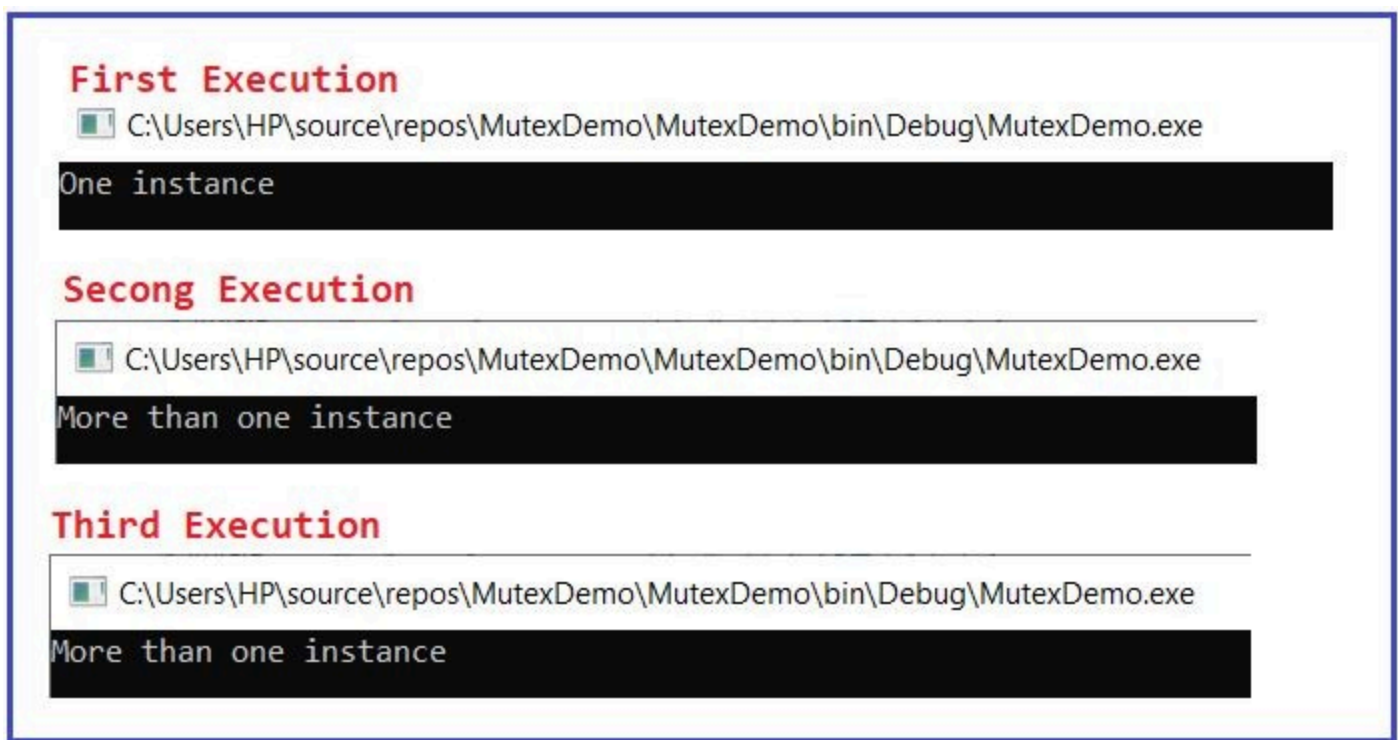
```

```

        // Try to open Existing Mutex.
        //If MyMutex is not opened, then it will throw an exception
        Mutex.OpenExisting("MyMutex");
    }
    catch
    {
        // If exception occurred, there is no such mutex.
        _mutex = new Mutex(true, "MyMutex");
        // Only one instance.
        return true;
    }
    // More than one instance.
    return false;
}
}
}

```

Now, build the project and then go to the projects bin\Debug directory and click on the Application EXE file three times and you will get the following results.



Note: Mutex only allows one external thread to access our application code. But if we want more control over the number of external threads to access our application code, then we need to use Semaphore Class in C#.

In the next article, I am going to discuss the use of [Semaphore Class in C#](#) with Examples. Here, in this article, I try to explain **How to use Mutex in C# to Protect Shared Resources in Multithreading** with Examples. I hope you enjoy this Mutex Class in C# with Examples article.

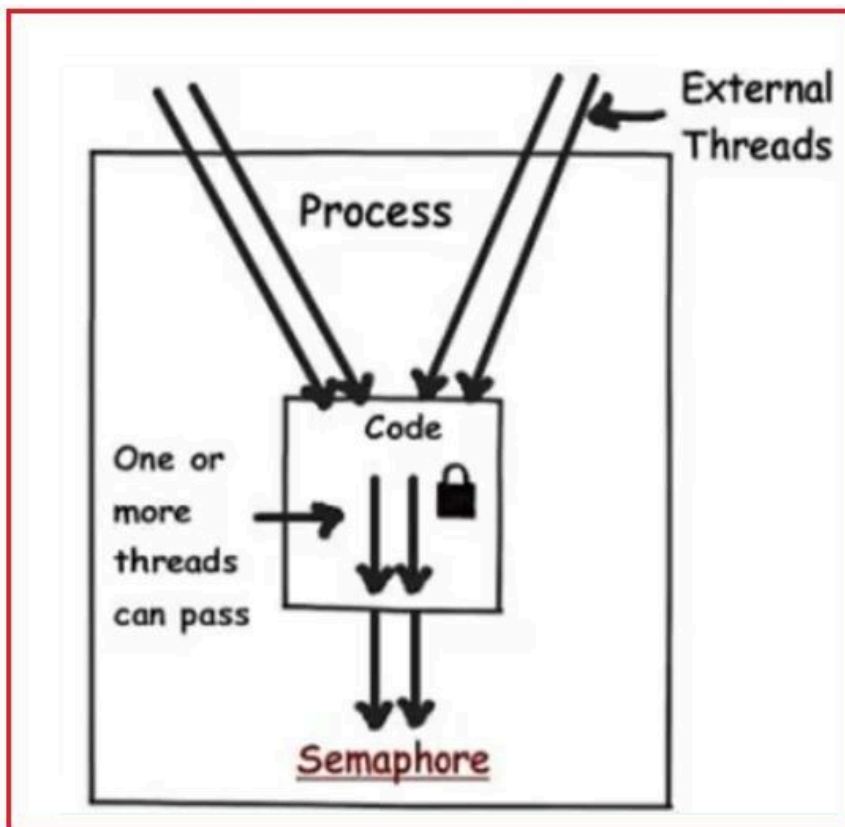
Semaphore Class in C# with Example

In this article, I am going to discuss **How to Implement Thread Synchronization using Semaphore Class in C#** with Examples. Please read our previous article where we discussed [How to use Mutex in C# to Protect Shared Resources in Multithreading from Concurrent Access](#) with Examples. As part of this article, we are going to discuss the following pointers.

1. **Why do we need Semaphore as we already have Lock, Monitor, and Mutex in C#?**
2. **What is Semaphore in C#?**
3. **How does Semaphore work in C#?**
4. **How to use the Semaphore class?**
5. **Understanding the different methods of Semaphore class in C# with examples.**

Why do we need Semaphore as we already have Lock, Monitor, and Mutex in C#?

Like Lock, Monitor, and Mutex the Semaphore is also used to provide thread safety in C# Multi-threaded Applications. The Lock and Monitors are basically used to provide thread safety for threads that are generated by the application itself i.e. Internal Threads. On the other hand, Mutex ensures thread safety for threads that are generated by external applications i.e. External Threads. Using Mutex, only one external thread can access our application code at any given point in time and this we have already seen in our previous article. But, if we want more control over the number of external threads that can access our application code, then we need to use the Semaphore class in C#. For a better understanding, please have a look at the below image.



Let us first see an example of how to restrict the number of external threads to access our application code using Semaphore and then we will understand the C# Semaphore Class in detail. In the below example, we

are creating the semaphore instance to allow a maximum of two threads to access our application code i.e. the code in between the WaitOne method and Release method.

```
using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        public static Semaphore semaphore = null;
        static void Main(string[] args)
        {
            try
            {
                //Try to Open the Semaphore if Exists, if not throw an exception
                semaphore = Semaphore.OpenExisting("SemaphoreDemo");
            }
            catch(Exception Ex)
            {
                //If Semaphore not Exists, create a semaphore instance
                //Here Maximum 2 external threads can access the code at the same time
                semaphore = new Semaphore(2, 2, "SemaphoreDemo");
            }
            Console.WriteLine("External Thread Trying to Acquiring");
            semaphore.WaitOne();
            //This section can be access by maximum two external threads: Start
            Console.WriteLine("External Thread Acquired");
            Console.ReadKey();
            //This section can be access by maximum two external threads: End
            semaphore.Release();
        }
    }
}
```

Now, build the project and then run the application EXE file three times. The first two times, you will see the message that External Thread Acquired, but when you run the EXE file for the 3rd time, you will see the message External Thread Trying to Acquiring only as shown in the below image.

First Run

 C:\Users\HP\source\repos\MutexDemo\SemaphoreDemo\bin\Debug\SemaphoreDemo.exe

```
External Thread Trying to Acquiring  
External Thread Acquired
```

Second Run

 C:\Users\HP\source\repos\MutexDemo\SemaphoreDemo\bin\Debug\SemaphoreDemo.exe

```
External Thread Trying to Acquiring  
External Thread Acquired
```

Third Run

 C:\Users\HP\source\repos\MutexDemo\SemaphoreDemo\bin\Debug\SemaphoreDemo.exe

```
External Thread Trying to Acquiring
```

Now, I hope you understand the basic need for the Semaphore Class in C#. Let us proceed further and understand the C# Semaphore Class in detail.

What is Semaphore Class in C#?

The Semaphore Class in C# is used to limit the number of external threads that can have access to a shared resource concurrently. In other words, we can say that Semaphore allows one or more external threads to enter into the critical section and execute the task concurrently with thread safety. So, in real-time, we need to use Semaphore when we have a limited number of resources and we want to limit the number of threads that can use it.

Constructors, and Methods of Semaphore Class in C#:

Let us understand the different Constructors, and Methods of Semaphore Class in C#. If you right-click on the Semaphore class and select go to definition, then you will see the following. Here, you can see that Semaphore is a sealed class and it is inherited from WaitHandle class. As a sealed class, no further inheritance is possible.

```
...public sealed class Semaphore : WaitHandle  
{  
    ...public Semaphore(int initialCount, int maximumCount);  
    ...public Semaphore(int initialCount, int maximumCount, string name);  
    ...public Semaphore(int initialCount, int maximumCount, string name, out bool createdNew);  
    ...public Semaphore(int initialCount, int maximumCount, string name, out bool createdNew, SemaphoreSecurity semaphoreSecurity);  
  
    ...public static Semaphore OpenExisting(string name);  
    ...public static Semaphore OpenExisting(string name, SemaphoreRights rights);  
    ...public static bool TryOpenExisting(string name, out Semaphore result);  
    ...public static bool TryOpenExisting(string name, SemaphoreRights rights, out Semaphore result);  
    ...public SemaphoreSecurity GetAccessControl();  
    ...public int Release();  
    ...public int Release(int releaseCount);  
    ...public void SetAccessControl(SemaphoreSecurity semaphoreSecurity);  
}
```

Constructors of Semaphore Class in C#:

The Semaphore Class in C# provides the following four constructors that we can use to create an instance of the Semaphore class.

1. **Semaphore(int initialCount, int maximumCount)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries.
2. **Semaphore(int initialCount, int maximumCount, string name)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries, and optionally specifying the name of a system semaphore object.
3. **Semaphore(int initialCount, int maximumCount, string name, out bool createdNew)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries, optionally specifying the name of a system semaphore object, and specifying a variable that receives a value indicating whether a new system semaphore was created.
4. **Semaphore(int initialCount, int maximumCount, string name, out bool createdNew, SemaphoreSecurity semaphoreSecurity)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries, optionally specifying the name of a system semaphore object, specifying a variable that receives a value indicating whether a new system semaphore was created, and specifying security access control for the system semaphore.

Parameters used in Semaphore Class Constructors:

1. **initialCount**: The initial number of requests for the semaphore that can be granted concurrently. It throws `ArgumentException` if `initialCount` is greater than `maximumCount`.
2. **maximumCount**: The maximum number of requests for the semaphore that can be granted concurrently. It will throw `ArgumentOutOfRangeException` if `maximumCount` is less than 1 or `initialCount` is less than 0.
3. **name**: The name of a named system semaphore object.
4. **createdNew**: When this method returns, contains true if a local semaphore was created (that is, if the name is null or an empty string) or if the specified named system semaphore was created; false if the specified named system semaphore already existed. This parameter is passed uninitialized.
5. **semaphoreSecurity**: A `System.Security.AccessControl.SemaphoreSecurity` object that represents the access control security to be applied to the named system semaphore.

Methods of Semaphore Class in C#:

The Semaphore Class in C# provides the following methods.

1. **OpenExisting(string name)**: This method is used to open a specified named semaphore if it already exists. It returns an object that represents the named system semaphore. Here, the parameter name specifies the name of the system semaphore to open. It will throw `ArgumentException` if the name is an empty string. -or- name is longer than 260 characters. It will throw `ArgumentNullException` if the name is null.
2. **OpenExisting(string name, SemaphoreRights rights)**: This method is used to open the specified named semaphore, if it already exists, with the desired security access. It returns an object that represents the named system semaphore. Here, the parameter name specifies the name of the system semaphore to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access.
3. **TryOpenExisting(string name, out Semaphore result)**: This method is used to open the specified named Semaphore, if it already exists, and returns a value that indicates whether the operation

succeeded. Here, the parameter name specifies the name of the system Semaphore to open. When this method returns, the result contains a Semaphore object that represents the named Semaphore if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named mutex was opened successfully; otherwise, false.

4. **TryOpenExisting(string name, SemaphoreRights rights, out Semaphore result)**: This method is used to open the specified named Semaphore, if it already exists, with the desired security access, and returns a value that indicates whether the operation succeeded. Here, the parameter name specifies the name of the system Semaphore to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access. When this method returns, the result contains a Semaphore object that represents the named Semaphore if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named Semaphore was opened successfully; otherwise, false.
5. **Release()**: This method exit the semaphore and returns the previous count. It returns the count on the semaphore before the Release method was called.
6. **Release(int releaseCount)**: This method exit the semaphore a specified number of times and returns the previous count. Here, the parameter releaseCount specifies the number of times to exit the semaphore. It returns the count on the semaphore before the Release method was called.
7. **GetAccessControl()**: This method Gets the access control security for a named system semaphore.
8. **SetAccessControl(SemaphoreSecurity semaphoreSecurity)**: This method Sets the access control security for a named system semaphore.

Note: The **Semaphore** Class in C# is inherited from **WaitHandle** abstract class and the WaitHandle class provides the WaitOne() method which we need to call to lock the resource. Note that a Semaphore object can only be released from the same thread which obtained it.

1. **WaitOne() Method**: Threads can enter into the critical section by using the WaitOne method. We need to call the WaitOne method on the semaphore object. If the Int32 variable which is maintained by semaphore is greater than 0 then it allows the thread to enter into the critical section.

How does Semaphore work in C#?

The Semaphores are Int32 variables that are stored in operating system resources. When we initialize the semaphore object we initialize it with a number. This number basically used to limit the threads that can enter the critical section.

So, when a thread enters the critical section, it decreases the value of the Int32 variable by 1 and when a thread exits from the critical section, it then increases the value of the Int32 variable by 1. The most important point that you need to remember is when the value of the Int32 variable is 0, then no thread can enter into the critical section.

How to Create a Semaphore in C#?

You can use the following statement to create the Semaphore instance in C#. Here, we use the overloaded version of the constructor which takes two parameters to create an instance of the semaphore class.

```
Semaphore semaphoreObject = new Semaphore(initialCount: 2, maximumCount: 3);
```

As you can see in the above statement, we are passing two values to the Constructor of the Semaphore class while initializing. These two values represent InitialCount and MaximumCount. The maximumCount defines how many maximum threads can enter the critical section and the initialCount set the value of the Int32 variable.

The InitialCount parameter sets the value for the Int32 variable. That is it defines the initial number of requests for the semaphore that can be granted concurrently. The MaximumCount parameter defines the maximum number of requests for the semaphore that can be granted concurrently.

For example, if we set the maximum count value as 3 and the initial count value is 0, it means 3 threads are already in the critical section, so no more new threads can enter the critical section. If we set the maximum count value as 3 and the initial count value as 2. It means a maximum of 3 threads can enter the critical section and there is one thread that is currently in the critical section, so two new threads can enter the critical section.

Note1: When a thread enters the critical section, it decreases the initialCount variable value by 1 and when a thread exits from the critical section, it then increases the initialCount variable value by 1. And when the value of the initialCount variable is 0, then no thread can enter into the critical section. The second parameter maximumCount always must be equal to or greater than the first parameter initialCount otherwise we will get an exception.

Note2: We need to call the Release() method when the thread wants to exit from the critical section. When this method is called, it increments the Int32 variable which is maintained by the semaphore object.

Example to understand Semaphore in C#:

Let us see an example for a better understanding of how to use the Semaphore to implement Thread Synchronization to Protect shared resources in multithreading from concurrent access in C#. Please have a look at the below example. In the below example, we initialize a semaphore object with 2 initialcount and maximum of 3 threads that can enter the critical section. We start the for loop with runs from 0 to 10. We started threads using the Thread class and the call shared resource DoSomeTask method.

Each thread calls the WaitOne method of semaphore object before doing the required task. The WaitOne method will decrease the initialcount variable value by 1. So, the WaitOne method will limit the number of threads to access the shared resource. After completing the task each thread calls the Release method which will increment initialcount variable value by 1 of the semaphore object. This allows further threads to enter into a critical section.

```
using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        public static Semaphore semaphore = new Semaphore(2, 3);
        static void Main(string[] args)
        {
            for (int i = 1; i <= 10; i++)
            {
                Thread threadObject = new Thread(DoSomeTask)
                {
                    Name = "Thread " + i
                };
                threadObject.Start();
            }
            Console.ReadKey();
        }
    }
}
```

```

    }
    static void DoSomeTask()
    {
        Console.WriteLine(Thread.CurrentThread.Name + " Wants to Enter into
Section for processing");
        try
        {
            //Blocks the current thread until the current WaitHandle receives
signal.
            semaphore.WaitOne();
            //Decrease the Initial Count Variable by 1
            Console.WriteLine("Success: " + Thread.CurrentThread.Name + "
its work");
            Thread.Sleep(5000);
            Console.WriteLine(Thread.CurrentThread.Name + "Exit.");
        }
        finally
        {
            //Release() method to release semaphore
            //Increase the Initial Count Variable by 1
            semaphore.Release();
        }
    }
}

```

Output:

```
Success: Thread 3 is Doing its work
Thread 1 Wants to Enter into Critical Section for processing
Thread 5 Wants to Enter into Critical Section for processing
Thread 2 Wants to Enter into Critical Section for processing
Success: Thread 1 is Doing its work
Thread 4 Wants to Enter into Critical Section for processing
Thread 6 Wants to Enter into Critical Section for processing
Thread 7 Wants to Enter into Critical Section for processing
Thread 9 Wants to Enter into Critical Section for processing
Thread 8 Wants to Enter into Critical Section for processing
Thread 10 Wants to Enter into Critical Section for processing
Thread 3Exit.
Thread 1Exit.
Success: Thread 4 is Doing its work
Success: Thread 2 is Doing its work
Thread 4Exit.
Success: Thread 5 is Doing its work
Thread 2Exit.
Success: Thread 6 is Doing its work
Thread 5Exit.
Success: Thread 7 is Doing its work
Thread 6Exit.
Success: Thread 8 is Doing its work
Thread 7Exit.
Success: Thread 9 is Doing its work
Thread 8Exit.
Success: Thread 10 is Doing its work
Thread 9Exit.
Thread 10Exit.
```

As you can see in the above output, here maximum of two threads are entering the critical section and doing their tasks at any given point in time.

In the next article, I am going to discuss [How to Implement Thread Synchronization using SemaphoreSlim Class in C#](#) with Examples. Here, in this article, I try to explain **How to Implement Thread Synchronization using Semaphore Class in C#** with Examples. I hope you enjoy this article and understand the concept of Semaphore Class in C# with Examples.

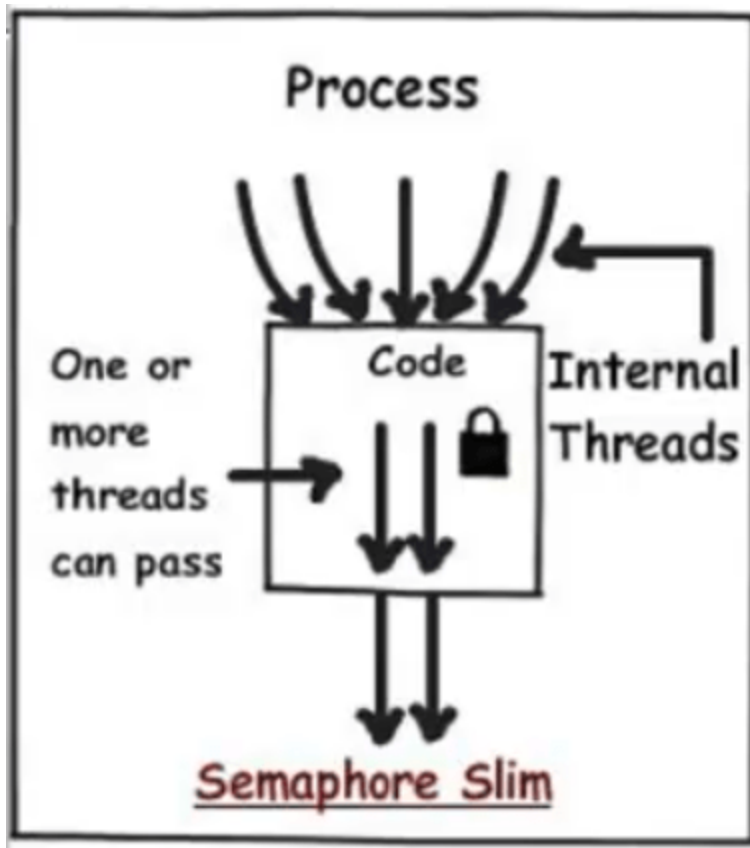
SemaphoreSlim Class in C# with Examples

In this article, I am going to discuss **How to Implement Thread Synchronization using SemaphoreSlim Class in C#** with Examples. Please read our previous article where we discussed [How to Implement Thread Synchronization using Semaphore Class in C#](#) with Examples. The SemaphoreSlim Class represents a lightweight alternative to Semaphore that limits the number of threads that can access a resource or pool of resources concurrently.

Why do we need SemaphoreSlim as we already have Lock, Monitor, Mutex, and Semaphore in C#?

Like [Lock](#), [Monitor](#), [Mutex](#), and [Semaphore](#), the SemaphoreSlim class in C# is also used to provide thread safety. The Lock and Monitors are basically used to provide thread safety for Internal Threads i.e. the threads generated by the application itself. On the other hand, Mutex and Semaphore ensure thread safety for threads that are generated by the external applications i.e. External Threads. Using Mutex, only one external thread can access our application code at any given point in time. And, if we want more control over the number of external threads that can access our application code, then can use Semaphore in C#.

Using Lock and Monitor, only one internal thread can access our application code at any given point in time. But, if we want more control over the number of internal threads that can access our application code, then we need to use SemaphoreSlim class in C#. For a better understanding, please have a look at the below image.



What is SemaphoreSlim class in C#?

The SemaphoreSlim Class in C# is recommended for synchronization within a single app. A lightweight semaphore controls access to a pool of resources that is local to your application. It represents a lightweight alternative to Semaphore that limits the number of threads that can access a resource or pool of resources concurrently.

Constructors, and Methods of SemaphoreSlim Class in C#:

Let us understand the different Constructors, and Methods of SemaphoreSlim Class in C#. If you right-click on the SemaphoreSlim class and select go to definition, then you will see the following class definition. Its a class and it implements the IDisposable interface.

```
namespace System.Threading
{
    ...public class SemaphoreSlim : IDisposable
    {
        ...public SemaphoreSlim(int initialCount);
        ...public SemaphoreSlim(int initialCount, int maxCount);

        ...public int CurrentCount { get; }
        ...public WaitHandle AvailableWaitHandle { get; }

        ...public void Dispose();
        ...public int Release();
        ...public int Release(int releaseCount);
        ...public bool Wait(TimeSpan timeout);
        ...public bool Wait(TimeSpan timeout, CancellationToken cancellationToken);
        ...public bool Wait(int millisecondsTimeout);
        ...public bool Wait(int millisecondsTimeout, CancellationToken cancellationToken);
        ...public void Wait();
        ...public void Wait(CancellationToken cancellationToken);
        ...public Task WaitAsync();
        ...public Task<bool> WaitAsync(int millisecondsTimeout);
        ...public Task<bool> WaitAsync(TimeSpan timeout);
        ...public Task<bool> WaitAsync(TimeSpan timeout, CancellationToken cancellationToken);
        ...public Task<bool> WaitAsync(int millisecondsTimeout, CancellationToken cancellationToken);
        ...public Task WaitAsync(CancellationToken cancellationToken);
        ...protected virtual void Dispose(bool disposing);
    }
}
```

Constructors of SemaphoreSlim Class in C#:

The SemaphoreSlim Class in C# provides the following two constructors that we can use to create an instance of the SemaphoreSlim class.

1. **SemaphoreSlim(int initialCount):** It initializes a new instance of the SemaphoreSlim class, specifying the initial number of requests that can be granted concurrently. Here, the parameter initialCount specifies the initial number of requests for the semaphore that can be granted concurrently. It will throw ArgumentOutOfRangeException if the initialCount is less than 0.
2. **SemaphoreSlim(int initialCount, int maxCount):** It initializes a new instance of the SemaphoreSlim class, specifying the initial and maximum number of requests that can be granted concurrently. Here, the parameter initialCount specifies the initial number of requests for the semaphore that can be granted concurrently. And the parameter maxCount specifies the maximum number of requests for the semaphore that can be granted concurrently. It will throw ArgumentOutOfRangeException if

initialCount is less than 0, or initialCount is greater than maxCount, or maxCount is equal to or less than 0.

Methods of SemaphoreSlim Class in C#:

The SemaphoreSlim Class in C# provides the following methods.

Wait Method:

There are multiple overloaded versions of the Wait method available in SemaphoreSlim Class. They are as follows:

1. **Wait():** It blocks the current thread until it can enter the System.Threading.SemaphoreSlim.
2. **Wait(TimeSpan timeout):** It blocks the current thread until it can enter the SemaphoreSlim, using a TimeSpan to specify the timeout. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.
3. **Wait(CancellationToken cancellationToken):** It blocks the current thread until it can enter the SemaphoreSlim while observing a CancellationToken.
4. **Wait(TimeSpan timeout, CancellationToken cancellationToken):** It blocks the current thread until it can enter the SemaphoreSlim, using a TimeSpan that specifies the timeout, while observing a CancellationToken. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.
5. **Wait(int millisecondsTimeout):** It blocks the current thread until it can enter the SemaphoreSlim, using a 32-bit signed integer that specifies the timeout. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.
6. **Wait(int millisecondsTimeout, CancellationToken cancellationToken):** It blocks the current thread until it can enter the SemaphoreSlim, using a 32-bit signed integer that specifies the timeout, while observing a CancellationToken. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.

Parameters:

The following are the parameter descriptions used in the Wait methods.

1. **timeout:** A TimeSpan that represents the number of milliseconds to wait, a TimeSpan that represents -1 milliseconds to wait indefinitely, or a TimeSpan that represents 0 milliseconds to test the wait handle and return immediately.
2. **cancellationToken:** The System.Threading.CancellationToken to observe.
3. **millisecondsTimeout:** The number of milliseconds to wait, System.Threading.Timeout.Infinite(-1) to wait indefinitely, or zero to test the state of the wait handle and return immediately.

Note: The Async versions of all the above methods are also available.

Release Method:

There are two overloaded versions of the Release method available in SemaphoreSlim class. They are as follows:

1. **Release():** It releases the SemaphoreSlim object once. It returns the previous count of the SemaphoreSlim.
2. **Release(int releaseCount):** It releases the SemaphoreSlim object a specified number of times. It returns the previous count of the SemaphoreSlim. Here, the parameter releaseCount specifies the number of times to exit the semaphore.

How SemaphoreSlim work in C#?

When we instantiate a SemaphoreSlim, we can specify the maximum number of threads that can enter the critical section concurrently. We also specify the initial number of threads that can enter the critical section concurrently. This defines the semaphore's count. The count is decremented each time a thread enters the SemaphoreSlim and incremented each time a thread releases the SemaphoreSlim.

To enter the SemaphoreSlim, a thread has to call one of the Wait or WaitAsync overload methods. To release the SemaphoreSlim, the thread has to call one of the Release methods. When the count reaches zero, subsequent calls to the Wait method block until other threads release the SemaphoreSlim. If multiple threads are blocked, there is no guaranteed order, such as FIFO or LIFO, that controls when threads enter the semaphore.

Example to Understand SemaphoreSlim Class in C#:

In the below example, we have created a Function called SemaphoreSlimFunction which gives access to a resource, the Wait method blocks the current thread until it can access the resource, and the Release method is required to release a resource once work is done. To understand SemaphoreSlim, we created five threads inside the Main method which will try to access SemaphoreSlimFunction simultaneously but we limited the access to three threads using the SemaphoreSlim object.

```
using System;
using System.Threading;
namespace SemaphoreSlimDemo
{
    class Program
    {
        //only 3 threads can access resource simlteniously
        static SemaphoreSlim semaphore = new SemaphoreSlim(initialCount:3);
        static void Main(string[] args)
        {
            for (int i = 1; i <= 5; i++)
            {
                int count = i;
                Thread t = new Thread(() => SemaphoreSlimFunction("Thread " +
* count));
                t.Start();
            }
            Console.ReadLine();
        }
        static void SemaphoreSlimFunction(string name, int seconds)
        {
            Console.WriteLine($"{name} Waits to access resource");
            semaphore.Wait();
        }
    }
}
```

```

        Console.WriteLine($"{name} was granted access to resource");
        Thread.Sleep(seconds);
        Console.WriteLine($"{name} is completed");
        semaphore.Release();
    }
}
}

```

Output:

```

Thread 1 Waits to access resource
Thread 1 was granted access to resource
Thread 2 Waits to access resource
Thread 2 was granted access to resource
Thread 3 Waits to access resource
Thread 3 was granted access to resource
Thread 4 Waits to access resource
Thread 5 Waits to access resource
Thread 1 is completed
Thread 4 was granted access to resource
Thread 2 is completed
Thread 5 was granted access to resource
Thread 3 is completed
Thread 4 is completed
Thread 5 is completed

```

Note: We use SemaphoreSlim instance to limit the concurrent threads that can access a shared resource in a multi-threaded environment. If threads trying to access a resource are more than the declared limit, only limited threads will be granted access and others will have to wait.

Another Example to Understand SemaphoreSlim Class in C#:

In the below example, we create one SemaphoreSlim instance with a maximum count of three threads and an initial count of zero threads. The example then starts five tasks, all of which block waiting for the semaphore. The main thread calls the Release(Int32) overload to increase the semaphore count to its maximum, which allows three tasks to enter the semaphore. Each time the semaphore is released, the previous semaphore count is displayed.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Example
{
    // Create the semaphore.
    private static SemaphoreSlim semaphore = new SemaphoreSlim(0, 3);
    // A padding interval to make the output more orderly.
    private static int padding;
    public static void Main()

```

```

{
    Console.WriteLine($"{semaphore.CurrentCount} tasks can enter the sema
Task[] tasks = new Task[5];
    // Create and start five numbered tasks.
    for (int i = 0; i <= 4; i++)
    {
        tasks[i] = Task.Run(() =>
        {
            // Each task begins by requesting the semaphore.
            Console.WriteLine($"Task {Task.CurrentId} begins and waits for
semaphore");
            int semaphoreCount;
            semaphore.Wait();
            try
            {
                Interlocked.Add(ref padding, 100);
                Console.WriteLine($"Task {Task.CurrentId} enters the sema
                // The task just sleeps for 1+ seconds.
                Thread.Sleep(1000 + padding);
            }
            finally
            {
                semaphoreCount = semaphore.Release();
            }
            Console.WriteLine($"Task {Task.CurrentId} releases the semaph
previous count: {semaphoreCount}");
        });
    }
    // Wait for one second, to allow all the tasks to start and block.
    Thread.Sleep(1000);
    // Restore the semaphore count to its maximum value.
    Console.WriteLine("Main thread calls Release(3) --> ");
    semaphore.Release(3);
    Console.WriteLine($"{semaphore.CurrentCount} tasks can enter the sema
    // Main thread waits for the tasks to complete.
    Task.WaitAll(tasks);
    Console.WriteLine("Main thread Exits");
    Console.ReadKey();
}
}

```

Output:

```
0 tasks can enter the semaphore
Task 1 begins and waits for the semaphore
Task 2 begins and waits for the semaphore
Task 4 begins and waits for the semaphore
Task 3 begins and waits for the semaphore
Task 5 begins and waits for the semaphore
Main thread calls Release(3) --> 3 tasks can enter the semaphore
Task 4 enters the semaphore
Task 3 enters the semaphore
Task 1 enters the semaphore
Task 4 releases the semaphore; previous count: 0
Task 1 releases the semaphore; previous count: 1
Task 2 enters the semaphore
Task 3 releases the semaphore; previous count: 1
Task 5 enters the semaphore
Task 2 releases the semaphore; previous count: 2
Task 5 releases the semaphore; previous count: 1
Main thread Exits
```

In the next article, I am going to discuss [Why and How a Deadlock Occurs in a Multithreaded Application in C#](#) with Examples. Here, in this article, I try to explain **How to Implement Thread Synchronization using SemaphoreSlim Class in C#** with Examples. I hope you enjoy this article and understand the concept of SemaphoreSlim Class in C# with Examples.

Deadlock in C# with Example

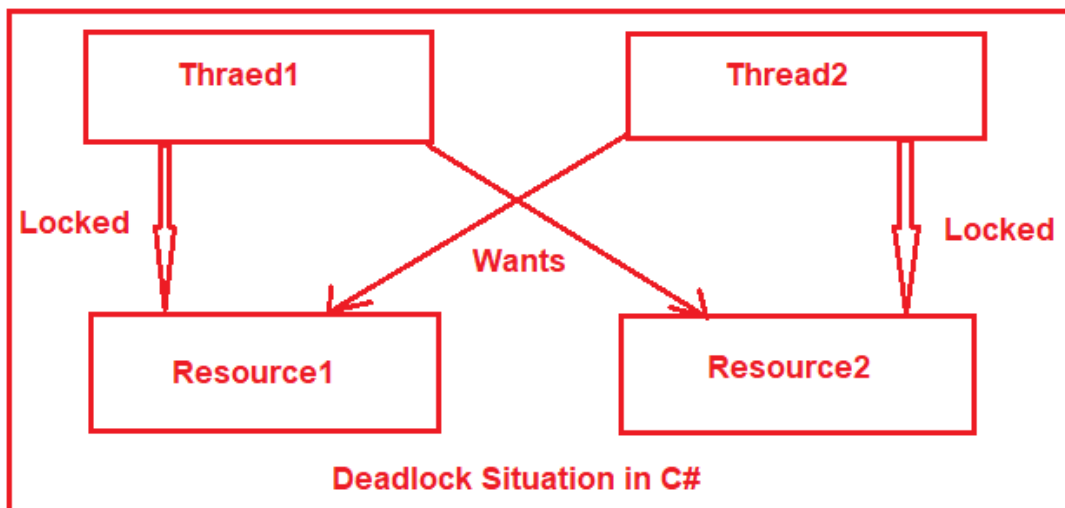
In this article, I am going to discuss **Deadlock in C#** with Examples. Please read our previous article discussing [SemaphoreSlim in C#](#) with Examples. Deadlock is one of the most important aspects to understand as a developer. As part of this article, we are going to discuss the following pointers.

1. **What is deadlock?**
2. **Why did a Deadlock occur?**
3. **How a deadlock can occur in a multithreaded application?**
4. **How to avoid a Deadlock by using Monitor.TryEnter method?**
5. **How Do We Avoid Deadlock by acquiring locks in a specific order?**

What is a Deadlock in C#?

In simple words, we can define a deadlock in C# as a situation where two or more threads are **unmoving or frozen** in their execution because they are waiting for each other to finish.

For example, let's say we have two threads, Thread1 and Thread2, and two resources, Resource1 and Resource2. Thread1 locked **Resource1** and tried to acquire a lock on **Resource2**. At the same time, **Thread2** acquired a lock on **Resource2** and tried to acquire a lock on **Resource1**.



As you can see in the above image, **Thread1** is waiting to acquire a **lock on Resource2**, which is held by **Thread2**. **Thread2** also can't finish its work and release the lock on **Resource2** because it is waiting to acquire a lock on **Resource1**, which Thread1 locks, and hence a Deadlock situation occurred.

Deadlock can occur if the following conditions hold true:

1. **Mutual Exclusion:** This implies that only one thread can access a resource at a particular time.
2. **Hold and Wait:** This is a condition in which a thread holds at least one resource and waits for at least one resource already acquired by another thread.
3. **Circular Wait:** This is a condition in which two or more threads are waiting for a resource acquired by the next member in the chain.

Example to understand Deadlock in C#:

Let's understand Deadlock in C# with an example. Create a class file named **Account.cs** and copy and paste the following code into it.

```
namespace DeadLockDemo
{
    public class Account
    {
        public int ID { get; }
        private double Balance { get; set;}
        public Account(int id, double balance)
        {
            ID = id;
            Balance = balance;
        }

        public void WithdrawMoney(double amount)
        {
            Balance -= amount;
        }
        public void DepositMoney(double amount)
        {
            Balance += amount;
        }
    }
}
```

The above Account class is very straightforward. We created the class with two properties, ID and Balance, which we initialize through the constructor of this class. So, at the time of creating the Account class instance, we need to pass the ID and Balance value. We have also created two methods. The WithdrawMoney method is used to withdraw the amount, while the DepositMoney method is used to add the amount.

AccountManager.cs:

Next, create another class file named **AccountManager.cs** and copy and paste the following code into it.

```
using System;
using System.Threading;
namespace DeadLockDemo
{
    public class AccountManager
    {
        private Account FromAccount;
        private Account ToAccount;
        private double TransferAmount;
        public AccountManager(Account AccountFrom, Account AccountTo, double
AmountTransfer)
        {
            FromAccount = AccountFrom;
```

```

        ToAccount = AccountTo;
        TransferAmount = AmountTransfer;
    }
    public void FundTransfer()
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} trying to acquire
{FromAccount.ID}");
        lock (FromAccount)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} acquired lock
{FromAccount.ID}");
            Console.WriteLine($"{Thread.CurrentThread.Name} Doing Some wor
Thread.Sleep(1000);
            Console.WriteLine($"{Thread.CurrentThread.Name} trying to acqu
{ToAccount.ID}");
            lock (ToAccount)
            {
                FromAccount.WithdrawMoney(TransferAmount);
                ToAccount.DepositMoney(TransferAmount);
            }
        }
    }
}

```

In the above code, we created two Account type variables to hold the FromAccount and ToAccount details, i.e., the Account from where the amount is going to be deducted and the account to whom the amount is created. We also created another double-type variable, i.e., TransferAmount, to hold the amount that is going to be deducted from the FromAccount and credited to the ToAccount. Through the constructor of this class, we are initializing these variables.

We also created the FundTransfer method, which will perform the required task. As you can see, it first acquires a lock on the From Account and then does some work. After 1 second, it backs up and tries to acquire a lock on the To Account.

Modifying the Main Method:

Now modify the Main method of the Program class as shown below. Here, for accountManager1, Account1001 is the FromAccount, and Account1002 is the ToAccount. Similarly, for accountManager2, Account1002 is the FromAccount and Account1001 is the ToAccount

```

using System;
using System.Threading;
namespace DeadLockDemo
{
    class Program
    {
        public static void Main()
        {
            Console.WriteLine("Main Thread Started");
        }
    }
}

```

```

Account Account1001 = new Account(1001, 5000);
Account Account1002 = new Account(1002, 3000);
AccountManager accountManager1 = new AccountManager(Account1001, /
5000);

Thread thread1 = new Thread(accountManager1.FundTransfer)
{
    Name = "Thread1"
};
AccountManager accountManager2 = new AccountManager(Account1002, /
6000);

Thread thread2 = new Thread(accountManager2.FundTransfer)
{
    Name = "Thread2"
};
thread1.Start();
thread2.Start();
thread1.Join();
thread2.Join();
Console.WriteLine("Main Thread Completed");
Console.ReadKey();
    }
}
}

```

Output:

```

Main Thread Started
Thread1 trying to acquire lock on 1001
Thread1 acquired lock on 1001
Thread1 Doing Some work
Thread2 trying to acquire lock on 1002
Thread2 acquired lock on 1002
Thread2 Doing Some work
Thread1 trying to acquire lock on 1002
Thread2 trying to acquire lock on 1001

```

Note: For thread1, Account1001 is resource1 and Account1002 is resource2. On the other hand, for thread2, Account1002 is resource1, and Account1001 is resource2. Now, run the application and see if a deadlock occurred.

The reason for the deadlock is that **thread1** acquired an exclusive lock on **Account1001** and then did some processing. In the meantime, **thread2** started, and it acquired an exclusive lock on **Account1002** and then did some processing. Then **thread1** back and wanted to acquire a lock on **Account1002** which is already locked by thread2. Similarly, **thread2** is back and wants to acquire a lock on Account1001, which is already locked by thread1 and hence deadlock.

Avoiding Deadlock by using Monitor.TryEnter method?

One of the overloaded versions (**TryEnter(object obj, int millisecondsTimeout)**) of the **Monitor.TryEnter** method takes the second parameter as the time out in milliseconds. Using that parameter, we can specify a timeout for the thread to release the lock. If a thread is holding a resource for a long time while the other thread is waiting, then Monitor will provide a time limit and force the lock to release it. So that the other thread can enter the critical section. Modifying the **AccountManager** class as shown below:

```
using System;
using System.Threading;
namespace DeadLockDemo
{
    public class AccountManager
    {
        private Account FromAccount;
        private Account ToAccount;
        private double TransferAmount;
        public AccountManager(Account AccountFrom, Account AccountTo, double
AmountTransfer)
        {
            this.FromAccount = AccountFrom;
            this.ToAccount = AccountTo;
            this.TransferAmount = AmountTransfer;
        }
        public void FundTransfer()
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} trying to acquire
{FromAccount.ID}");

            lock (FromAccount)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} acquired lock
{FromAccount.ID}");
                Console.WriteLine($"{Thread.CurrentThread.Name} Doing Some wor
Thread.Sleep(3000);
                Console.WriteLine($"{Thread.CurrentThread.Name} trying to acqui
{ToAccount.ID}");

                if (Monitor.TryEnter(ToAccount, 3000))
                {
                    Console.WriteLine($"{Thread.CurrentThread.Name} acquired l
{ToAccount.ID}");

                    try
                    {
                        FromAccount.WithdrawMoney(TransferAmount);
                        ToAccount.DepositMoney(TransferAmount);
                    }
                    finally
                    {
                        Monitor.Exit(ToAccount);
                    }
                }
            }
        }
    }
}
```

```

        else
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} Unable to
lock on {ToAccount.ID}, So existing.");
        }
    }
}
}
}
}

```

Output:

```

Main Thread Started
Thread1 trying to acquire lock on 1001
Thread1 acquired lock on 1001
Thread1 Doing Some work
Thread2 trying to acquire lock on 1002
Thread2 acquired lock on 1002
Thread2 Doing Some work
Thread1 trying to acquire lock on 1002
Thread2 trying to acquire lock on 1001
Thread1 Unable to acquire lock on 1002, So existing.
Thread2 acquired lock on 1001
Main Thread Completed

```

As you can see in the output, thread1 releases the lock and exits from the critical section, which allows thread2 to enter the critical section and complete its task.

How Do We Avoid Deadlock in C# by acquiring locks in a specific order?

Please modify the AccountManager class as shown below.

```

using System;
using System.Threading;
namespace DeadLockDemo
{
    public class AccountManager
    {
        private Account FromAccount;
        private Account ToAccount;
        private readonly double TransferAmount;
        private static readonly Mutex mutex = new Mutex();
        public AccountManager(Account AccountFrom, Account AccountTo, double
AmountTransfer)
        {
            this.FromAccount = AccountFrom;
            this.ToAccount = AccountTo;
            this.TransferAmount = AmountTransfer;
        }
    }
}

```

```

public void FundTransfer()
{
    object _lock1, _lock2;
    if (FromAccount.ID < ToAccount.ID)
    {
        _lock1 = FromAccount;
        _lock2 = ToAccount;
    }
    else
    {
        _lock1 = ToAccount;
        _lock2 = FromAccount;
    }
    Console.WriteLine($"{Thread.CurrentThread.Name} trying to acquire
{((Account)_lock1).ID}");

    lock (_lock1)
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} acquired lock
{((Account)_lock1).ID}");
        Console.WriteLine($"{Thread.CurrentThread.Name} Doing Some wor
Thread.Sleep(3000);
        Console.WriteLine($"{Thread.CurrentThread.Name} trying to acqu
{((Account)_lock2).ID}");
        lock(_lock2)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} acquired l
{((Account)_lock2).ID}");
            FromAccount.WithdrawMoney(TransferAmount);
            ToAccount.DepositMoney(TransferAmount);
        }
    }
}
}
}
}
}

```

Output:

```

Main Thread Started
Thread1 trying to acquire lock on 1001
Thread1 acquired lock on 1001
Thread1 Doing Some work
Thread2 trying to acquire lock on 1001
Thread1 trying to acquire lock on 1002
Thread1 acquired lock on 1002
Thread2 acquired lock on 1001
Thread2 Doing Some work
Thread2 trying to acquire lock on 1002
Thread2 acquired lock on 1002
Main Thread Completed

```

In the next article, I am going to show you the [Performance of a multithreaded program when running on a single-core/processor machine versus a multi-core/processor](#) machine. In this article, I try to explain deadlock in C# using different approaches. I hope you enjoy this Deadlock in C# with Examples article.

Performance Testing of a Multithreaded Application in C#

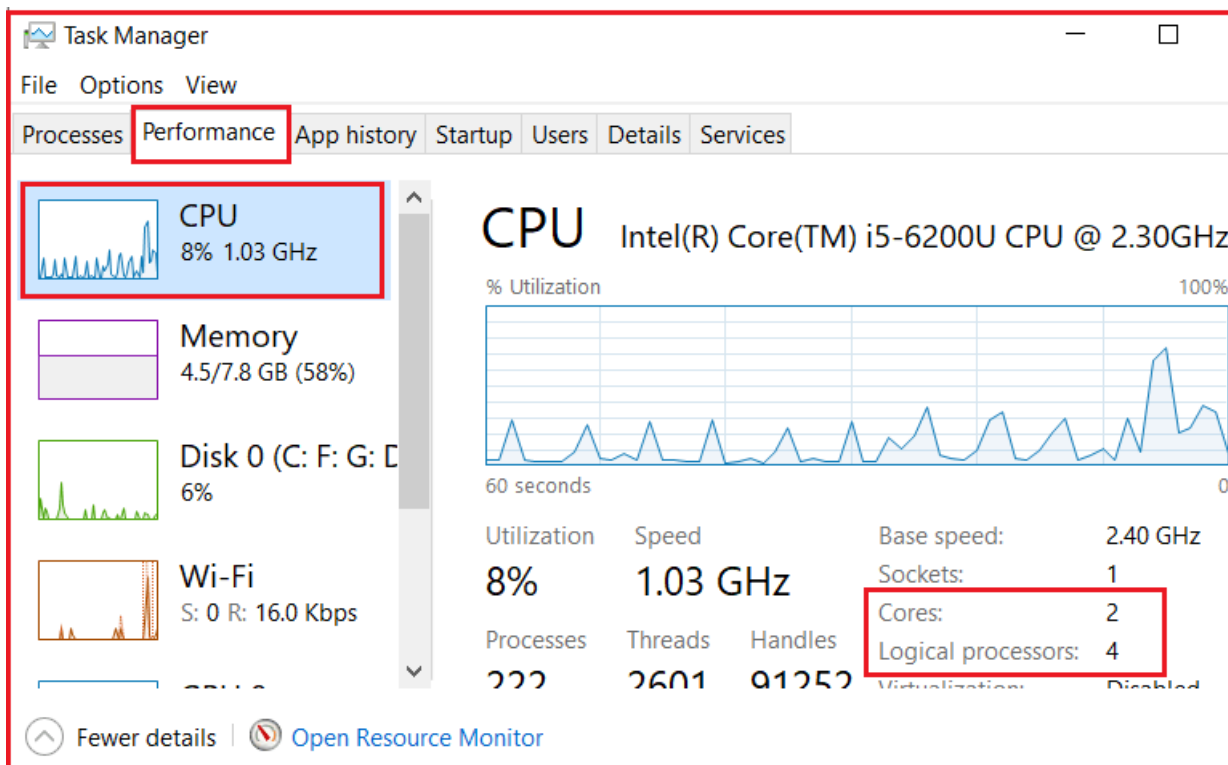
In this article, I am going to discuss the **Performance Testing of a multithreaded application in C#** with an example. Please read our previous article where we discussed [Deadlock in C#](#) with Examples. As part of this article, I will show you the performance implications of a multithreaded application when we run the application on a machine having a single core/processor versus a machine having a multi-core/processor.

How to find out how many processors you have on your machine?

You can find out how many processors you have on your machine in many different ways. Some of them are as follows:

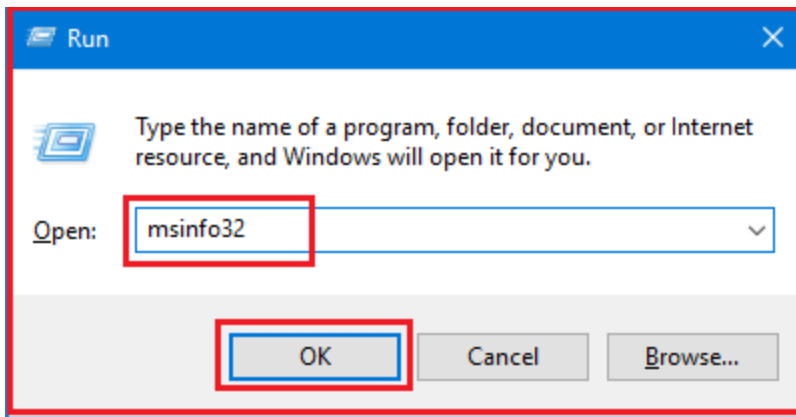
Way1: Using Task Manager

Right-click on the Taskbar and select the “**Task Manager**” option from the context menu. Then click on the “**Performance**” tab and select the “**CPU**” from the left side panel and then you will see the cores and Logical processors on the right side as shown in the below image.

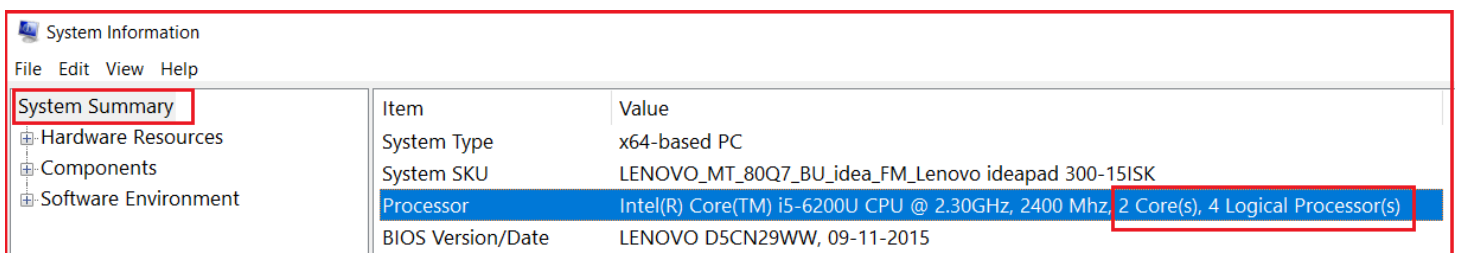


Way2: Using msinfo32 Command

Press the **Windows key + R** to open the **Run** command, then type **msinfo32** and click on the **OK** button as shown in the below image.



Once you click on the OK button, it will open up the System Information app. From that select the **Summary** option and scroll down until you find Processor. The details will tell you both how many cores and logical processors your CPU has as shown in the below image.



Way3: Using dot net code.

You can use the following code in any type of .net application to find out the total processors on the machine.

[Environment.ProcessorCount](#)

Way4: Using the command prompt

On the Windows command prompt write the following code and press enter

`echo %NUMBER_OF_PROCESSORS%`

Performance Testing With Multiple Processors:

If you have a machine with multiple processors, then multiple threads can execute your application code in parallel on different cores. For example, if your machine has two cores and there are two threads to execute your application code, then each thread is going to be run on an individual core. As a result, we will get better performance.

If you have two threads and if each thread takes 10 milliseconds to complete the execution, then on a machine with 2 processors, the total time taken is 10 milliseconds.

Performance Testing With Single Processor:

If you have a machine with a single processor, then multiple threads are going to execute one after the other. It is not possible for a single-core processor machine to execute multiple threads in parallel. The operating system switches between the threads so fast, it just gives us the illusion that the threads are running in parallel. On a single-core or processor machine, multiple threads can negatively affect the performance as there is overhead involved with context-switching.

If you have two threads and each thread takes 10 milliseconds to complete the execution, then on a machine with a single processor, the total time taken is 20 milliseconds plus thread context switching time if any.

Example: Using a Single Thread

```
using System;
using System.Diagnostics;
using System.Threading;
namespace MultithreadingPerformanceTesting
{
    class Program
    {
        public static void Main()
        {
            Stopwatch stopwatch = Stopwatch.StartNew();
            stopwatch = Stopwatch.StartNew();
            EvenNumbersSum();
            OddNumbersSum();
            stopwatch.Stop();
            Console.WriteLine($"Total time in milliseconds :
{stopwatch.ElapsedMilliseconds}");
            Console.ReadKey();
        }
        public static void EvenNumbersSum()
        {
            double Evensum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 0)
                {
                    Evensum = Evensum + count;
                }
            }
            Console.WriteLine($"Sum of even numbers = {Evensum}");
        }
        public static void OddNumbersSum()
        {
            double Oddsum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 1)
                {
                    Oddsum = Oddsum + count;
                }
            }
            Console.WriteLine($"Sum of odd numbers = {Oddsum}");
        }
    }
}
```

Output:

```
Sum of even numbers = 625000025000000
Sum of odd numbers = 625000000000000
Total time in milliseconds : 696
```

As you can see it takes approximately 696 milliseconds to complete the execution.

Example: Using Multiple Threads

Let's rewrite the previous example using multiple threads and compare the output.

```
using System;
using System.Diagnostics;
using System.Threading;
namespace MultithreadingPerformanceTesting
{
    class Program
    {
        public static void Main()
        {
            Stopwatch stopwatch = Stopwatch.StartNew();
            stopwatch = Stopwatch.StartNew();
            Thread thread1 = new Thread(EvenNumbersSum);
            Thread thread2 = new Thread(OddNumbersSum);
            thread1.Start();
            thread2.Start();
            thread1.Join();
            thread2.Join();
            stopwatch.Stop();
            Console.WriteLine($"Total time in milliseconds :
{stopwatch.ElapsedMilliseconds}");
            Console.ReadKey();
        }
        public static void EvenNumbersSum()
        {
            double Evensum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 0)
                {
                    Evensum = Evensum + count;
                }
            }
            Console.WriteLine($"Sum of even numbers = {Evensum}");
        }
        public static void OddNumbersSum()
        {
            double Oddsum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 1)
```

```

        {
            Oddsum = Oddsum + count;
        }
    }
    Console.WriteLine($"Sum of odd numbers = {Oddsum}");
}
}
}

```

Output:

```

Sum of even numbers = 625000025000000
Sum of odd numbers = 625000000000000
Total time in milliseconds : 508

```

As you can see, it takes approximately 508 milliseconds to complete the execution. In the next article, I am going to discuss [Thread Pooling in C#](#) with examples. Here, in this article, I try to show you the performance testing of a multithread application with an example. I hope you enjoy this article.

Thread Pool in C# with Examples

In this article, I am going to discuss **Thread Pool in C#** with Examples. Please read our previous article where we discussed the [Performance Testing of a Multithreaded Application](#) in C#. As part of this article, we are going to discuss the following pointers.

1. [The Request Life cycle of a Thread.](#)
2. [What is Thread Pooling in C#?](#)
3. [Why do we need a C# Thread Pool?](#)
4. [Performance testing between normal thread and thread pooling](#)

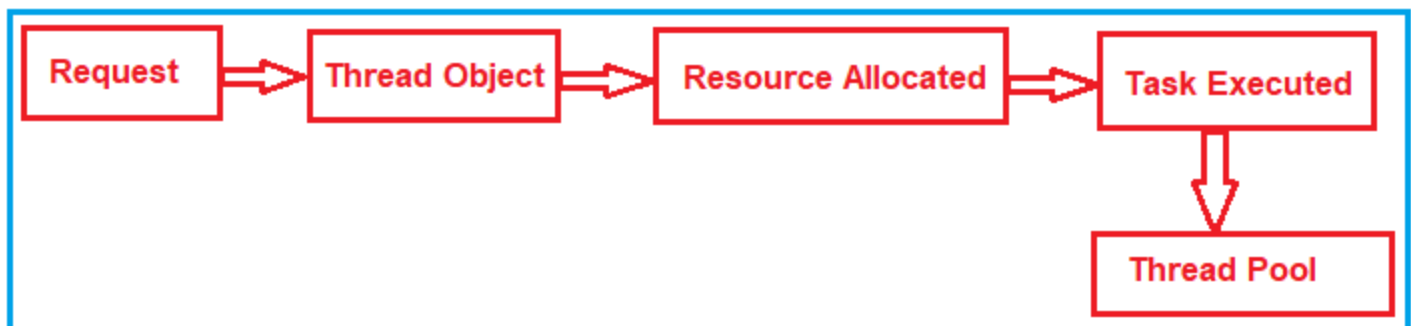
The Request Life cycle of a Thread in C# with Example.

Let us understand the life cycle of a thread in C#. In order to understand this, please have a look at the following image. When the .NET framework receives a request (the request can be a method call or function call from any kind of application). To handle that request, a thread object is created. When the thread object is created some resources are allocated to that thread object such as memory. After then the task is executed and once the task is completed then the garbage collector removes that thread object to free up memory allocation. This is the life cycle of a thread in C#.



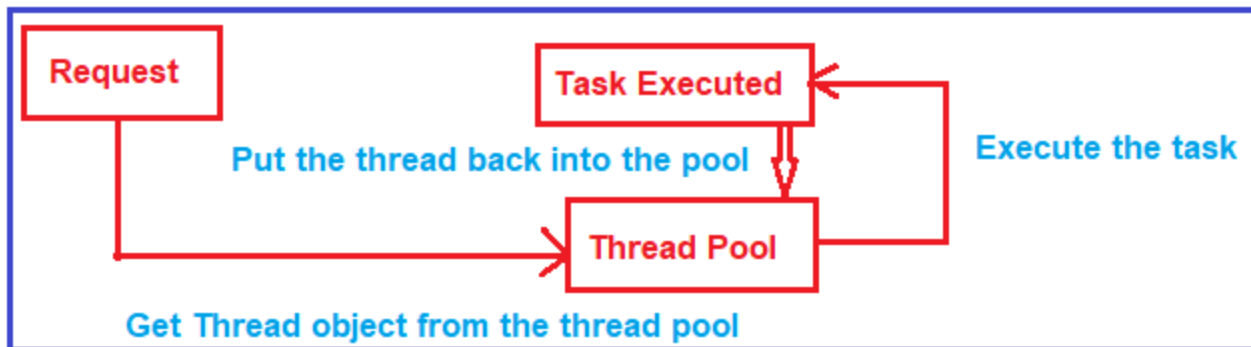
These steps are going to be repeated again and again for each request that comes in a multithread application. That means every time a new thread object is created, it gets allocated in the memory. If there are many requests then there will be many thread objects and if there are many thread objects then there will be a load on the memory which slows down your application.

There is great room for performance improvements. The Thread object is created, resources are allocated, the task is executed, and then it should not go for garbage collection, instead of how about taking the thread object and putting it into a pool as shown in the below image? This is where thread pooling comes into the picture.



Thread Pool in C#:

The Thread pool in C# is nothing but a collection of threads that can be reused to perform a number of tasks in the background. Now when a request comes, then it directly goes to the thread pool and checks whether there are any free threads available or not. If available, then it takes the thread object from the thread pool and executes the task as shown in the below image.



Once the thread completes its task then it is again sent back to the thread pool so that it can reuse. This reusability avoids an application to create a number of threads and this enables less memory consumption.

How to use Thread Pool in C#?

Let us see a simple example to understand how to use Thread Pooling in C#. Once you understand how to use thread pooling, then we will see the performance benchmark between the normal thread object and the thread pool thread object.

Step1:

In order to implement thread pooling in C#, first, we need to import the Threading namespace as ThreadPool class belongs to this Threading namespace as shown below.

```
using System.Threading;
```

Step2:

Once you import the Threading namespace, then you need to use the **ThreadPool** class, and using this class you need to call the **QueueUserWorkItem** static method. And, if you go to the definition of the **QueueUserWorkItem** method, then you will see that this method takes one parameter of the type **WaitCallback** object. While creating the object of the **WaitCallback** class, you need to pass the method name that you want to execute as shown below.

```
ThreadPool.QueueUserWorkItem(new WaitCallback(MyMethod));
```

Here, the **QueueUserWorkItem** method Queues the function for execution and that function executes when a thread becomes available from the thread pool. If no thread is available then it will wait until one thread gets freed. Here MyMethod is the method that we want to execute by a thread pool thread.

Complete Example Code to Understand Thread Pooling in C#

As you can see in the below code, here, we create one method that is MyMethod and as part of that method, we simply print the thread id, whether the thread is a background thread or not, and whether it is from a thread pool or not. And we want to execute this method 10 times using the thread pool threads. So, here we use a simple for each loop and use the ThreadPool class and call that method.

```
using System;
using System.Threading;
namespace ThreadPoolApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 10; i++)
            {
                ThreadPool.QueueUserWorkItem(new WaitCallback(MyMethod));
            }
            Console.Read();
        }
        public static void MyMethod(object obj)
        {
            Thread thread = Thread.CurrentThread;
            string message = $"Background: {thread.IsBackground}, Thread Pool: {thread.IsThreadPoolThread}, Thread ID: {thread.ManagedThreadId}";
            Console.WriteLine(message);
        }
    }
}
```

Once you execute the above code, it will give you the following output. As you can see, it shows that it is background threads and the threads are from the thread pool and the thread ids may vary in your output. Here, you can see some of the threads are reused and this is the advantage of using Thread Pooling in C#.

```
Background: True, Thread Pool: True, Thread ID: 3
Background: True, Thread Pool: True, Thread ID: 3
Background: True, Thread Pool: True, Thread ID: 9
Background: True, Thread Pool: True, Thread ID: 6
Background: True, Thread Pool: True, Thread ID: 4
Background: True, Thread Pool: True, Thread ID: 7
Background: True, Thread Pool: True, Thread ID: 9
Background: True, Thread Pool: True, Thread ID: 6
Background: True, Thread Pool: True, Thread ID: 5
Background: True, Thread Pool: True, Thread ID: 3
```

Now, let us see the same example using the normal threads. Please modify the code as follows. Here, instead of using ThreadPool, we are using the Thread class to create the threads and invoke the methods.

```
using System;
using System.Threading;
```

```

namespace ThreadPoolApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 10; i++)
            {
                Thread thread = new Thread(MyMethod)
                {
                    Name = "Thread" + i
                };
                thread.Start();
            }
            Console.Read();
        }
        public static void MyMethod(object obj)
        {
            Thread thread = Thread.CurrentThread;
            string message = $"Background: {thread.IsBackground}, Thread Pool:
{thread.IsThreadPoolThread}, Thread ID: {thread.ManagedThreadId}";
            Console.WriteLine(message);
        }
    }
}

```

Now, run the above code, and you will get the following output. Here, you can see, the threads are not backgrounded threads and they are also not coming from the thread pool, and the threads are also not reused. Each thread has a unique thread ID.

```

Background: False, Thread Pool: False, Thread ID: 3
Background: False, Thread Pool: False, Thread ID: 7
Background: False, Thread Pool: False, Thread ID: 6
Background: False, Thread Pool: False, Thread ID: 4
Background: False, Thread Pool: False, Thread ID: 5
Background: False, Thread Pool: False, Thread ID: 8
Background: False, Thread Pool: False, Thread ID: 10
Background: False, Thread Pool: False, Thread ID: 9
Background: False, Thread Pool: False, Thread ID: 11
Background: False, Thread Pool: False, Thread ID: 12

```

Performance testing using and without using Thread Pool in C# with Example:

Let us see an example to understand the performance benchmark. Here, we will compare how much time the thread object takes and how much time the thread pool thread takes to do the same task i.e. to execute the same methods.

In order to do this, what we are going to do is, we will create a method called **Test** as shown below. This method takes an input parameter of the type object and as part of that Test method, we are doing nothing that means an empty method.

```
public static void Test(object obj)
{
}
```

Then we will create two methods such as **MethodWithThread** and **MethodWithThreadPool** and inside these two methods, we will create one for loop which will execute 10 times. Within for loop, we are going to call the Test as shown below. As you can see, the **MethodWithThread** method uses the Thread object to call the Test method while the **MethodWithThreadPool** method uses the ThreadPool object to call the Test method.

```
public static void MethodWithThread(.)
{
    for (int i = 0; i < 10; i++)
    {
        Thread thread = new Thread(Test);
    }
}
```

Uses Thread object to call the Test method

```
public static void MethodWithThreadPool( )
{
    for (int i = 0; i < 10; i++)
    {
        ThreadPool.QueueUserWorkItem(new WaitCallback(Test));
    }
}
```

Uses ThreadPool thread to call the Test method

Now we need to call the above two methods (**MethodWithThread** and **MethodWithThreadPool**) from the main method. As we are going to test the performance benchmark, we are going to call these two methods between the stopwatch start and end as shown below. The Stopwatch class is available in **System.Diagnostics** namespace. The for loop within the Main method is for warm-up. This is because when we run the code for the first time, compilation happens and compilation takes some time and we don't want to measure that.

```

for (int i = 0; i < 10; i++)
{
    MethodWithThread();
    MethodWithThreadPool();
}

```

This is warm up.

```

Stopwatch stopwatch = new Stopwatch();

Console.WriteLine("Execution using Thread");
stopwatch.Start(); //Start the stopwatch
MethodWithThread(); //call the method
stopwatch.Stop(); //Stop the stopwatch
Console.WriteLine("Time consumed by MethodWithThread is : " +
    stopwatch.ElapsedTicks.ToString());

stopwatch.Reset();

Console.WriteLine("Execution using Thread Pool");
stopwatch.Start(); //Start the stopwatch
MethodWithThreadPool(); //call the method
stopwatch.Stop(); //Stop the stopwatch
Console.WriteLine("Time consumed by MethodWithThreadPool is : " +
    stopwatch.ElapsedTicks.ToString());

```

The complete code is given below.

```

using System;
using System.Diagnostics;
using System.Threading;
namespace ThreadPoolApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            //Warmup Code start
            for (int i = 0; i < 10; i++)
            {
                MethodWithThread();
                MethodWithThreadPool();
            }
            //Warmup Code start
            Stopwatch stopwatch = new Stopwatch();
            Console.WriteLine("Execution using Thread");

```

```

        stopwatch.Start();
        MethodWithThread();
        stopwatch.Stop();
        Console.WriteLine("Time consumed by MethodWithThread is : " +
                           stopwatch.ElapsedTicks.ToString());

        stopwatch.Reset();
        Console.WriteLine("Execution using Thread Pool");
        stopwatch.Start();
        MethodWithThreadPool();
        stopwatch.Stop();
        Console.WriteLine("Time consumed by MethodWithThreadPool is : " +
                           stopwatch.ElapsedTicks.ToString());

        Console.Read();
    }
    public static void MethodWithThread()
    {
        for (int i = 0; i < 10; i++)
        {
            Thread thread = new Thread(Test);
            thread.Start();
        }
    }
    public static void MethodWithThreadPool()
    {
        for (int i = 0; i < 10; i++)
        {
            ThreadPool.QueueUserWorkItem(new WaitCallback(Test));
        }
    }
    public static void Test(object obj)
    {
    }
}
}

```

Output:

```

Execution using Thread
Time consumed by MethodWithThread is : 272588
Execution using Thread Pool
Time consumed by MethodWithThreadPool is : 692

```

As you can see in the above output, the Time consumed by MethodWithThread is 272588 and the Time consumed by MethodWithThreadPool is 692. If you observe there is a vast time difference between these two.

So it proves that the thread pool gives better performance as compared to the thread class object. If there are need to create one or two threads then you need to use the Thread class object while if there is a need to create more than 5 threads then you need to go for the thread pool class in a multithreaded environment.

That's it for today. In the next article, I am going to discuss [Foreground and Background Threads in C#](#) with examples. Here, in this article, I try to explain **Thread Pool in C#** with examples. I hope you enjoy this article and understood C# Thread Pooling with Examples.

Foreground and Background Threads in C# with Examples

In this article, I am going to discuss **Foreground and Background Threads in C#** with Examples. Please read our previous article where we discussed [Thread Pool in C#](#) with Examples.

Types of Threads in C#

Threading is one of the most important concepts and as a developer, we all need to understand this concept because in most real-time applications we use threading. In C#, by default, the execution of an application starts with a thread which is called the main thread which is automatically run by the CLR and the Operating System.

From the main thread, we can also create other threads for doing the desired tasks simultaneously or concurrently in the application. In C#, we can create two types of threads. They are as follows.

1. **Foreground Thread**
2. **Background Thread**

Foreground Thread in C#:

Foreground threads are those threads that keep running even after the main application exits or quits. That means if the Main thread leaves the application, then still the foreground threads are running to complete their assigned task. So, the foreground threads in C# do not care whether the main thread is alive or not, it completes only when it finishes its assigned work. That means the life of a foreground thread in C# does not depend upon the main thread. The foreground thread has the ability to prevent the current application from terminating. The CLR will not shut down the application until all the Foreground Threads have finished their assigned work. The main thread is a foreground thread.

Background Thread in C#:

Background Threads are those threads that will quit if our main application quits. Or in simple words, we can say that the life of the background thread will depend upon the life of the main thread. In short, if our main application quits, then the background threads will also quit. Background threads are viewed by the CLR and if all foreground threads have terminated, then all the background threads are automatically stopped when the application quits.

Note: By default, when we created a thread in C#, it is a Foreground thread.

How to Make a Thread as Background Thread in C#?

As we already discussed, in C#, the thread is created and managed by the Thread class. The Thread class provides a property called `IsBackground` to check whether the given thread is running in the background or in the foreground. If the value of `IsBackground` property is set to true, then the thread is a background thread i.e. running in the background. Or if the value of `IsBackground` is set to false, then the thread is a foreground thread i.e. running in the foreground.

1. **`IsBackground {get; set;}`**: This property is used to get or set a value indicating whether or not a thread is a background thread. It returns true if this thread is or is to become a background thread; otherwise, false. It throws `ThreadStateException` if the thread is dead.

By default, when we create a thread in C# by using the Thread class, the IsBackground property is false, and if you want to make a background thread in your program, then you need to set the value of IsBackground property of the thread to true.

Example to Understand Foreground Thread in C#:

By default when we created a thread in C#, it is a Foreground Thread. As we discussed Foreground Threads are the threads that keep running even when the main application or main thread quits. Let us understand this with an example.

```
using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // A thread created here to run Method1 Parallely
            Thread thread1 = new Thread(Method1);
            Console.WriteLine($"Thread1 is a Background thread:
{thread1.IsBackground}");
            thread1.Start();
            //The control will come here and will exit
            //the main thread or main application
            Console.WriteLine("Main Thread Exited");
        }
        // Static method
        static void Method1()
        {
            Console.WriteLine("Method1 Started");
            for (int i = 0; i <= 5; i++)
            {
                Console.WriteLine("Method1 is in Progress!!");
                Thread.Sleep(1000);
            }
            Console.WriteLine("Method1 Exited");
            Console.WriteLine("Press any key to Exit.");
            Console.ReadKey();
        }
    }
}
```

Output:

```
Thread1 is a Background thread:  False
Main Thread Exited
Method1 Started
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 Exited
Press any key to Exit.
```

As you can see in the above output, after the main thread or main method completes its execution, still the Foreground Thread is running. So, this proves that Foreground threads keep running and complete their tasks even after the application exits or quits i.e. even after the main thread quits.

Example to Understand Background Thread in C#:

By default when we created a thread in C#, it is a Foreground Thread. And, if you want to make a thread as a background thread, then you need to set the `IsBackground` property of the thread object to true. Background Threads are the threads that will quit if our main thread quits i.e. if all the foreground threads quit, then automatically the background thread quits. Let us understand this with an example. We are rewriting the previous example and here we simply set the `IsBackground` property to true of the thread object.

```
using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // A thread created here to run Method1 Parallely
            Thread thread1 = new Thread(Method1)
            {
                //Thread becomes background thread
                IsBackground = true
            };

            Console.WriteLine($"Thread1 is a Background thread:
{thread1.IsBackground}");
            thread1.Start();
            //The control will come here and will exit
            //the main thread or main application
            Console.WriteLine("Main Thread Exited");
            //As the Main thread (i.e. foreground thread exits the application
            //Automatically, the background thread quits the application
        }
        // Static method
```

```

static void Method1()
{
    Console.WriteLine("Method1 Started");
    for (int i = 0; i <= 5; i++)
    {
        Console.WriteLine("Method1 is in Progress!!");
        Thread.Sleep(1000);
    }
    Console.WriteLine("Method1 Exited");
    Console.WriteLine("Press any key to Exit.");
    Console.ReadKey();
}
}
}

```

Output:

```

Thread1 is a Background thread: True
Main Thread Exited

```

In the above example, as soon as the main thread quits the background thread also quit.

Multiple Foreground Threads and one Background Thread in C#

In the below example, the main thread is by default a foreground thread and the main thread creates a thread1 object to call Method1. Here, thread1 is also a foreground thread. Then from Method1, we created a foreground thread to call Method2. Here, once all the foreground threads i.e. Main thread and thread1 quit, then automatically the background thread i.e. thread2 quits the application without completing its task (sometimes the task might be completed).

```

using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // A thread created here to run Method1 Parallely
            Thread thread1 = new Thread(Method1)
            {
            };
            Console.WriteLine($"Thread1 is a Background thread:
{thread1.IsBackground}");
            thread1.Start();
            //The control will come here and will exit
            //the main thread or main application
            Console.WriteLine("Main Thread Exited");
            //As the Main thread (i.e. foreground thread exits the applicatio

```

```

        //Automatically, the background thread quits the application
    }
    // Static method
    static void Method1()
    {
        Console.WriteLine("Method1 Started");
        Thread thread2 = new Thread(Method2)
        {
            IsBackground = true
        };
        thread2.Start();
        Thread.Sleep(3000);
        Console.WriteLine("Method1 Exited");
    }
    // Static method
    static void Method2()
    {
        Console.WriteLine("Method2 Started");
        for (int i = 0; i <= 10; i++)
        {
            Console.WriteLine("Method2 is in Progress!!");
            Thread.Sleep(1000);
        }
        Console.WriteLine("Method2 Exited");
        Console.WriteLine("Press any key to Exit.");
        Console.ReadKey();
    }
}

```

So, the point that you need to remember is once all the foreground thread including the main thread quits the application, then automatically all the background threads quit the application.

Points to Remember:

In C#, a thread is either a background thread or a foreground thread. Background threads are similar to the foreground threads, except that the background threads do not prevent a process from terminating. Once all the Foreground threads belonging to a process have terminated, then the CLR ends the process. Any remaining background threads are stopped and not completed.

By default, the following threads execute in the foreground (that is, their `IsBackground` property returns false):

1. The primary thread (or main application thread).
2. All threads are created by calling a `Thread` class constructor.

By default, the following threads execute in the background (that is, their `IsBackground` property returns true):

1. Thread pool threads are a pool of worker threads maintained by the runtime. You can configure the thread pool and schedule work on thread pool threads by using the `ThreadPool` class.

Example for Better understanding Background and Foreground Threads in C#:

In the below example, we are showing the behavior of foreground and background threads in C#. The code example will create a foreground thread and a background thread. The foreground thread keeps the process running until completes its for loop and terminates. The foreground thread has finished execution, the process is terminated before the background thread has completed execution.

```
using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            ThreadingTest foregroundTest = new ThreadingTest(5);
            //Creating a Foreground Thread
            Thread foregroundThread = new Thread(new
ThreadStart(foregroundTest.RunLoop));
            ThreadingTest backgroundTest = new ThreadingTest(50);
            //Creating a Background Thread
            Thread backgroundThread = new Thread(new ThreadStart(backgroundTest
            {
                IsBackground = true
            });
            foregroundThread.Start();
            backgroundThread.Start();
        }
    }
    class ThreadingTest
    {
        readonly int maxIterations;
        public ThreadingTest(int maxIterations)
        {
            this.maxIterations = maxIterations;
        }
        public void RunLoop()
        {
            for (int i = 0; i < maxIterations; i++)
            {
                Console.WriteLine($"{0} count: {1}", Thread.CurrentThread.IsBack
"Background Thread" : "Foreground Thread", i);
                Thread.Sleep(250);
            }
            Console.WriteLine("{0} finished counting.", Thread.CurrentThread.I
? "Background Thread" : "Foreground Thread");
        }
    }
}
```

Note1: In C#, the foreground threads have the ability to prevent the current application from terminating. The CLR will not shut down an application (which is to say, unload the hosting AppDomain) until all foreground threads have ended.

Note2: In C#, the Background threads are viewed by the CLR as expendable paths of execution that can be ignored at any point in time (even if they are currently doing some unit of work). Thus, if all foreground threads have terminated, any and all background threads are automatically killed when the application domain unloads.

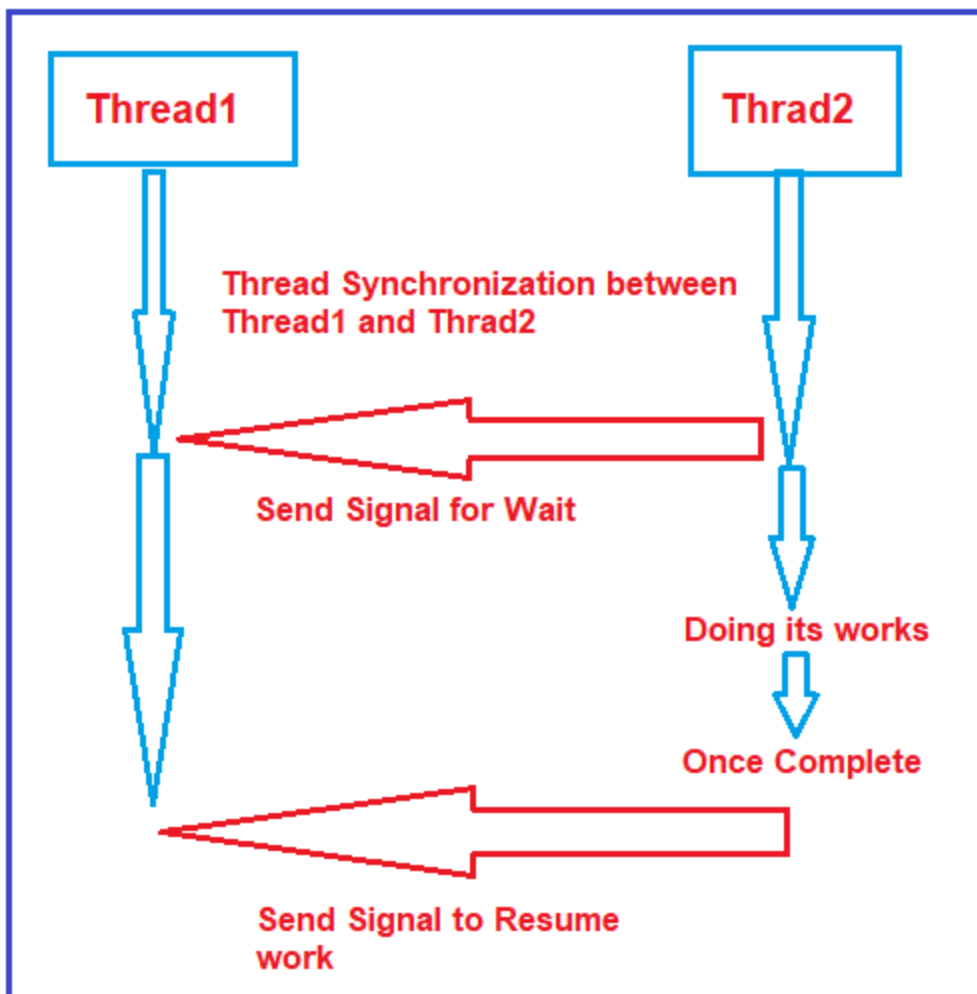
In the next article, I am going to discuss [AutoResetEvent and ManualResetEvent in C#](#) with Examples. Here, in this article, I try to explain **Foreground and Background Threads in C#** with Examples. I hope you enjoy this Foreground and Background Threads in C# article.

AutoResetEvent and ManualResetEvent in C# with Examples

In this article, I am going to discuss two important concepts of threading i.e. **AutoResetEvent** and **ManualResetEvent** in C# with Examples. In our previous articles, we already saw how to implement thread synchronization using **Lock**, **Monitor**, **Mutex**, **Semaphore**, etc. And there is another way of doing thread synchronization i.e. by using signaling methodology. And both AutoResetEvent and ManualResetEvent in C# help us to implement thread synchronization using signaling methodology.

What is Signaling Methodology?

Let us first understand what is Signalling Methodology and then we will understand how to implement the Signalling Methodology using AutoResetEvent and ManualResetEvent in C#. Let us understand this with an example. Please have a look at the below image. Let's say we have two threads Thread1 and Thread2. And we need to implement thread synchronization between these two threads. For thread synchronization, here thread2 sends a signal to thread1 saying that please go under Wait mode. And then thread2 continues doing its work. And once thread2 finishes its work, again it sends a signal to thread1 saying that you can resume your work from where you halted.



So, in this way, by using the Signalling Methodology we can implement thread synchronization between multiple threads in C#. And both AutoResetEvent and ManualResetEvent in C# help us to achieve this. So, here, first, we will see an example using AutoResetEvent, then we will see an example using ManualResetEvent, and finally, we will see the differences between them.

AutoResetEvent Class in C#:

AutoResetEvent is used to send signals between two threads. This class Notifies a waiting thread that an event has occurred. If you go to the definition of AutoResetEvent class, then you will see the following. This is a sealed class and hence cannot be inherited. And it is inherited from the EventWaitHandle class.

```
namespace System.Threading
{
    ...public sealed class AutoResetEvent : EventWaitHandle
    {
        ...public AutoResetEvent(bool initialState);
    }
}
```

This class provides the following constructor that we can use to create an instance of the AutoResetEvent class in C#.

1. **AutoResetEvent(bool initialState):** It initializes a new instance of the AutoResetEvent class with a Boolean value indicating whether to set the initial state to signaled. Here, if the parameter initialState is true then it set the initial state to signaled; false to set the initial state to non-signaled.

The AutoResetEvent is inherited from the EventWaitHandle class and if you go to the definition of the EventWaitHandle class, then you will see that this EventWaitHandle class implements the WaitHandle class as shown in the below image, and the EventWaitHandle class also has the Set and Reset method that we are going to use with AutoResetEvent object.

```
namespace System.Threading
{
    ...public class EventWaitHandle : WaitHandle
    {
        ...public EventWaitHandle(bool initialState, EventResetMode mode);
        ...public EventWaitHandle(bool initialState, EventResetMode mode, string name);
        ...public EventWaitHandle(bool initialState, EventResetMode mode, string name, out bool createdNew);
        ...public EventWaitHandle(bool initialState, EventResetMode mode, string name, out bool createdNew,
        ...public static EventWaitHandle OpenExisting(string name);
        ...public static EventWaitHandle OpenExisting(string name, EventWaitHandleRights rights);
        ...public static bool TryOpenExisting(string name, out EventWaitHandle result);
        ...public static bool TryOpenExisting(string name, EventWaitHandleRights rights, out EventWaitHandle result);
        ...public EventWaitHandleSecurity GetAccessControl();
        ...public bool Reset();
        ...public bool Set();
        ...public void SetAccessControl(EventWaitHandleSecurity eventSecurity);
    }
}
```

The following two methods of this class, we are going to use in our example.

1. **Set():** This method is used to set the state of the event to signaled, allowing one or more waiting threads to proceed. It returns true if the operation succeeds; otherwise, false.

2. **Reset()**: This method is used to set the state of the event to non signaled, causing threads to block. It returns true if the operation succeeds; otherwise, false.

Again, the `EventWaitHandle` class is inherited from the `WaitHandle` and if you go to the definition of `WaitHandle` class, then you will see that it is an abstract class and this class has some overloaded versions of the `WaitOne` method as shown in the below image. The `WaitOne` method we are going to use with the `AutoResetEvent` object.

```
namespace System.Threading
{
    ...public abstract class WaitHandle : MarshalByRefObject, IDisposable
    {
        ...public const int WaitTimeout = 258;
        ...protected static readonly IntPtr InvalidHandle;

        ...protected WaitHandle();

        ...public virtual IntPtr Handle { get; set; }
        ...public SafeWaitHandle SafeWaitHandle { get; set; }

        ...public static bool SignalAndWait(WaitHandle toSignal, WaitHandle toWaitOn, int millisecondsTimeout, bool exitContext);
        ...public static bool SignalAndWait(WaitHandle toSignal, WaitHandle toWaitOn, TimeSpan timeout, bool exitContext);
        ...public static bool SignalAndWait(WaitHandle toSignal, WaitHandle toWaitOn);
        ...public static bool WaitAll(WaitHandle[] waitHandles, TimeSpan timeout);
        ...public static bool WaitAll(WaitHandle[] waitHandles, int millisecondsTimeout, bool exitContext);
        ...public static bool WaitAll(WaitHandle[] waitHandles, TimeSpan timeout, bool exitContext);
        ...public static bool WaitAll(WaitHandle[] waitHandles);
        ...public static bool WaitAll(WaitHandle[] waitHandles, int millisecondsTimeout);
        ...public static int WaitAny(WaitHandle[] waitHandles, int millisecondsTimeout);
        ...public static int WaitAny(WaitHandle[] waitHandles);
        ...public static int WaitAny(WaitHandle[] waitHandles, TimeSpan timeout);
        ...public static int WaitAny(WaitHandle[] waitHandles, TimeSpan timeout, bool exitContext);
        ...public static int WaitAny(WaitHandle[] waitHandles, int millisecondsTimeout, bool exitContext);
        ...public virtual void Close();
        ...public void Dispose();
        ...public virtual bool WaitOne(int millisecondsTimeout, bool exitContext);
        ...public virtual bool WaitOne(TimeSpan timeout);
        ...public virtual bool WaitOne(int millisecondsTimeout);
        ...public virtual bool WaitOne(TimeSpan timeout, bool exitContext);
        ...public virtual bool WaitOne();
        ...protected virtual void Dispose(bool explicitDisposing);
    }
}
```

We are going to use the following method in our example.

1. **WaitOne()**: The `WaitOne()` method blocks the current thread until the current `WaitHandle` receives a signal. It returns true if the current instance receives a signal. If the current instance is never signaled, `WaitHandle.WaitOne(System.Int32, System.Boolean)` never returns.

How AutoResetEvent Work in C#?

The `AutoResetEvent` in C# maintains a boolean variable in memory. If the boolean variable is false then it blocks the thread and if the boolean variable is true it unblocks the thread. So, when we create an instance of `AutoResetEvent` class, we need to pass the default value of the boolean value to the constructor of `AutoResetEvent` class. The following is the syntax to instantiate an `AutoResetEvent` object.

```
AutoResetEvent autoResetEvent = new AutoResetEvent(false);
```

WaitOne Method

The WaitOne method blocks the current thread and waits for the signal by another thread. That means the WaitOne method puts the current thread into a Sleep state of the thread. The WaitOne method returns true if it receives the signal else returns false. We need to call the WaitOne method on the AutoResetEvent object as follows.

```
autoResetEvent.WaitOne();
```

Another overloaded version of the WaitOne method takes seconds as a parameter and will wait for the specified number of seconds. If it does not get any signal, then the thread will continue its work. Following is the syntax.

```
autoResetEvent.WaitOne(TimeSpan.FromSeconds(2))
```

Set Method

The Set method sent the signal to the waiting thread to proceed with its work. Following is the syntax to call the Set method.

```
autoResetEvent.Set();
```

Note: The most important point that you need to remember is both threads will share the same AutoResetEvent object. Any thread can enter into a wait state by calling the WaitOne() method of the AutoResetEvent object. When the other thread calls the Set() method it unblocks the waiting thread.

Example to Understand AutoResetEvent in C#:

Let us see an example to understand the AutoResetEvent in C#. In the below example, we have two threads. The Main thread will invoke the main method and the NewThread which will invoke the SomeMethod method. The main method will invoke the new thread and the new thread actually go and run the SomeMethod. And the SomeMethod will first print the first statement i.e. Starting..... and then it invokes the WaitOne() method which Put the current thread i.e.NewThread into the waiting state until it receives the signal. Then inside the static void Main method, when we press the enter key, it will invoke the Set method which will send a signal to other threads to resume their work i.e. send the signal to NewThread to resume its work, and the new thread then prints Finishing..... on the console window.

```
using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static AutoResetEvent autoResetEvent = new AutoResetEvent(false);

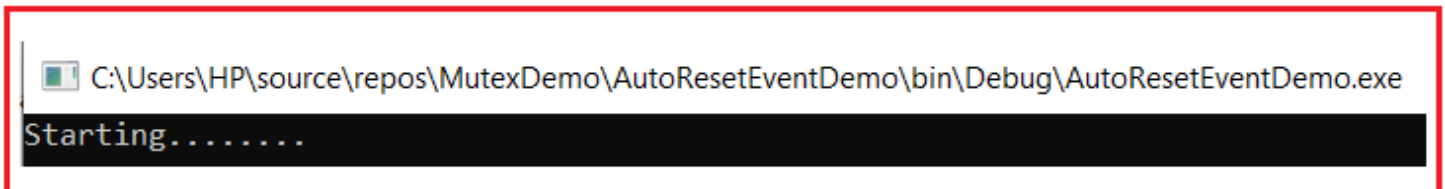
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt mode
            //Once we enter any key it will call set method and the SomeMethod
            Resume its work
        }
    }
}
```

```

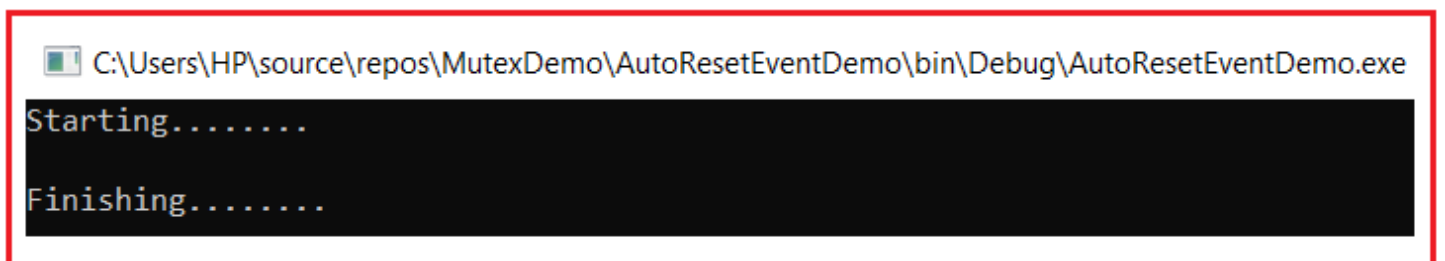
        Console.ReadLine();
        //It will send a signal to other threads to resume their work
        autoResetEvent.Set();
    }
    static void SomeMethod()
    {
        Console.WriteLine("Starting.....");
        //Put the current thread into waiting state until it receives the
        autoResetEvent.WaitOne(); //It will make the thread in halt mode
        Console.WriteLine("Finishing.....");
        Console.ReadLine(); //To see the output in the console
    }
}
}

```

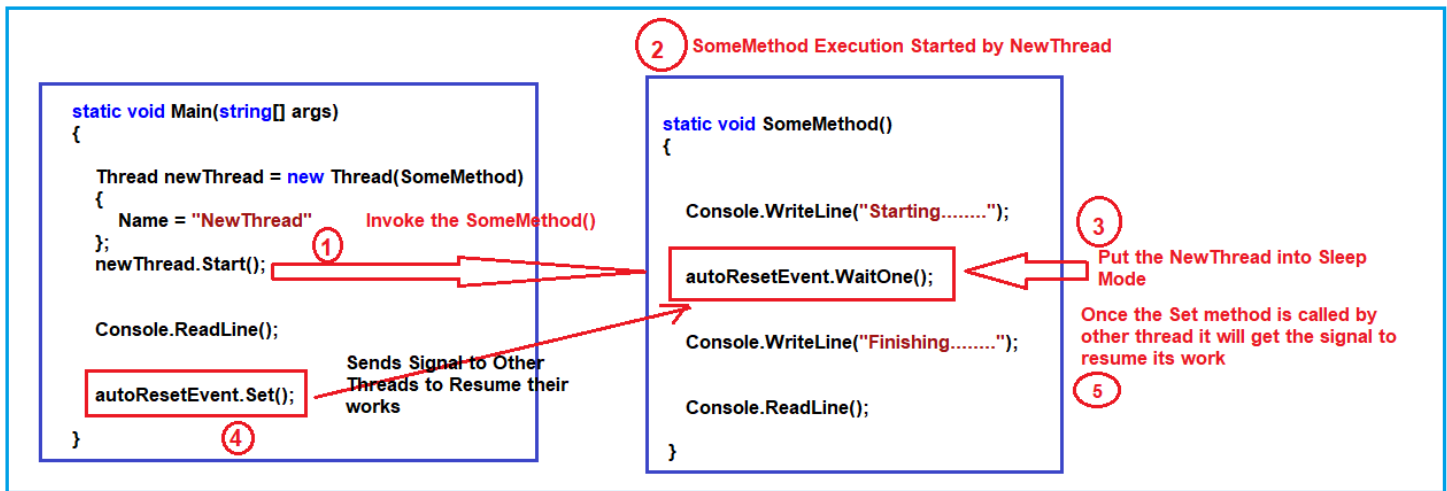
Now, run the application and you will get the following message.



At this point, the main thread is called the New Thread and the new thread executes the first statement i.e. printing the first message on the console and then invoking the WaitOne method. Once it invokes the WaitOne method the new thread goes into the sleep state. Next, when we press the enter key, then the main method invokes the Set method which will send a signal to other threads to resume their work. At this point, SomeMethod will resume its work and continue and you will see the Finishing message on the console window as shown below.



For a better understanding of the above program workflow please have a look at the below image.



Note: There is no guarantee that every call to the Set method will release a thread. If two calls are too close together, so that the second call occurs before a thread has been released, only one thread is released. It's as if the second call did not happen. Also, if Set is called when there are no threads waiting and the AutoResetEvent is already signaled, the call has no effect.

ManualResetEvent Class in C#:

The ManualResetEvent Class in C# works exactly the same as the AutoResetEvent Class in C#. Let us rewrite the same example using ManualResetEvent and then we will discuss the differences between them. Simply, replace the AutoResetEvent class with ManualResetEvent class in the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static ManualResetEvent manualResetEvent = new ManualResetEvent(false);

        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt mode
            //Once we enter any key it will call set method and the SomeMethod
            Resume its work
            Console.ReadLine();
            //It will send a signal to other threads to resume their work
            manualResetEvent.Set();
        }

        static void SomeMethod()
        {
            Console.WriteLine("Starting.....");
            //Put the current thread into waiting state until it receives the

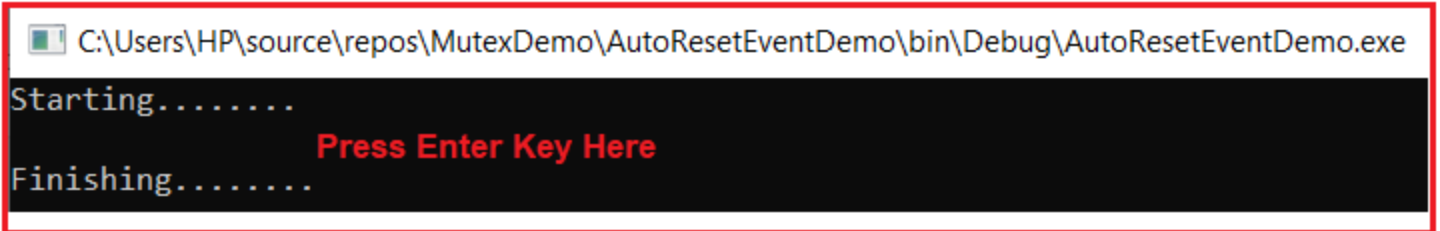
```

```

        manualResetEvent.WaitOne(); //It will make the thread in halt mode
        Console.WriteLine("Finishing.....");
        Console.ReadLine(); //To see the output in the console
    }
}
}

```

Output:



```

C:\Users\HP\source\repos\MutexDemo\AutoResetEventDemo\bin\Debug\AutoResetEventDemo.exe
Starting.....
Press Enter Key Here
Finishing.....

```

What are the Differences between AutoResetEvent and ManualResetEvent in C#?

Let us understand the differences with some examples. In AutoResetEvent, for every WaitOne method, there should be a Set method. That means if we are using the WaitOne method 2 times, then we should use the Set method 2 times. If we use the Set method 1 time, then the 2nd WaitOne method will be hung in the waiting state and will not be released. To understand this better please have a look at the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static AutoResetEvent manualResetEvent = new AutoResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt state let sleep the main
            3 secs
            Thread.Sleep(3000);
            Console.WriteLine("Releasing the WaitOne 1 by Set 1");
            manualResetEvent.Set(); //Set 1 will relase the Wait 1
            //To See how the SomeMethod goes in halt state let sleep the main
            3 secs
            Thread.Sleep(5000);
            Console.WriteLine("Releasing the WaitOne 2 by Set 2");
            manualResetEvent.Set(); //Set 2 will relase the Wait 2
            Console.ReadKey();
        }
    }
}

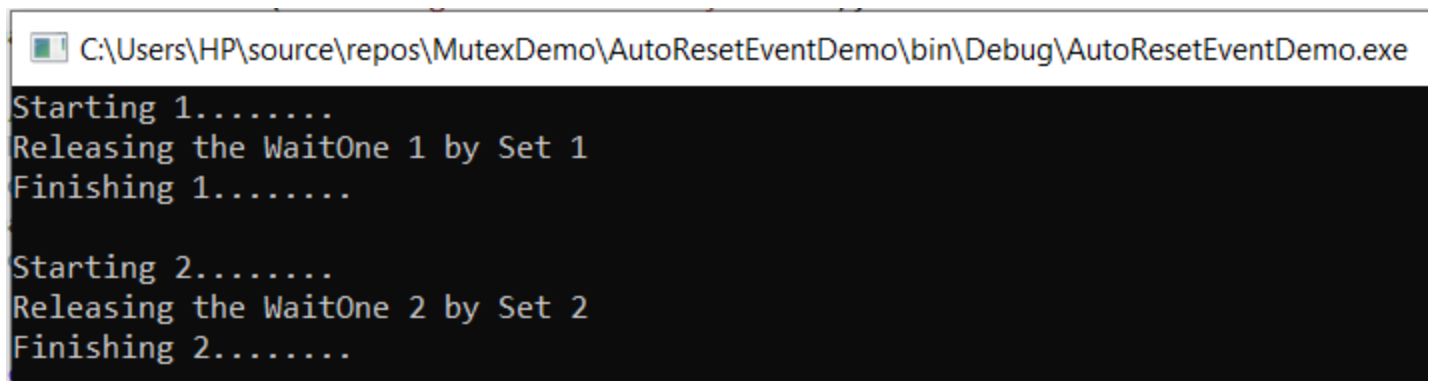
```

```

static void SomeMethod()
{
    Console.WriteLine("Starting 1.....");
    manualResetEvent.WaitOne(); //Wait 1
    Console.WriteLine("Finishing 1.....");
    Console.WriteLine();
    Console.WriteLine("Starting 2.....");
    manualResetEvent.WaitOne(); //Wait 2
    Console.WriteLine("Finishing 2.....");
}
}
}

```

Output:



```

C:\Users\HP\source\repos\MutexDemo\AutoResetEventDemo\bin\Debug\AutoResetEventDemo.exe
Starting 1.....
Releasing the WaitOne 1 by Set 1
Finishing 1.....

Starting 2.....
Releasing the WaitOne 2 by Set 2
Finishing 2.....

```

For each WaitOne method, we should and must have a Set method if we are using AutoResetEvent in C#. If we have two WaitOne methods and we have one Set method, then the second WaitOne method will hang in the sleep mode and will not release. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static AutoResetEvent manualResetEvent = new AutoResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt state let sleep the main
            Thread.Sleep(3000);
            Console.WriteLine("Releasing the WaitOne 1 by Set 1");
            manualResetEvent.Set(); //Set 1 will relase the Wait 1

```

3 secs

```

        Console.ReadKey();
    }
    static void SomeMethod()
    {
        Console.WriteLine("Starting 1.....");
        manualResetEvent.WaitOne(); //Wait 1
        Console.WriteLine("Finishing 1.....");
        Console.WriteLine();
        Console.WriteLine("Starting 2.....");
        manualResetEvent.WaitOne(); //Wait 2
        Console.WriteLine("Finishing 2.....");
    }
}

```

Output: Finishing 2..... statement will never be executed; you will get the following output.

```

Starting 1.....
Releasing the WaitOne 1 by Set 1
Finishing 1.....

Starting 2.....

```

But if we write the previous example using ManualResetEvent then it will work. That is one Set method in ManualResetEvent can release all the WaitOne methods. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static ManualResetEvent manualResetEvent = new ManualResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt state let sleep the main

            Thread.Sleep(3000);
            Console.WriteLine("Releasing the WaitOne 1 by Set 1");
            manualResetEvent.Set(); //Set will release all the WaitOne

            Console.ReadKey();
        }
        static void SomeMethod()
    }
}

```

3 secs

```

    {
        Console.WriteLine("Starting 1.....");
        manualResetEvent.WaitOne(); //Wait 1
        Console.WriteLine("Finishing 1.....");
        Console.WriteLine();
        Console.WriteLine("Starting 2.....");
        manualResetEvent.WaitOne(); //Wait 2
        Console.WriteLine("Finishing 2.....");
    }
}

```

Output:

```

Starting 1.....
Releasing the WaitOne 1 by Set 1
Finishing 1.....

Starting 2.....
Finishing 2.....

```

So, the one and the only difference between `AutoResetEvent` and `ManualResetEvent` in C# is that for each `WaitOne` method there should be a corresponding `Set` method in `AutoResetEvent` while for all the `WaitOne` methods, one `Set` method is enough to release in the case of `ManualResetEvent`.

In the next article, I am going to discuss [Thread Life Cycle in C#](#) with Examples. Here, in this article, I try to explain **AutoResetEvent and ManualResetEvent in C#** with Examples. I hope you enjoy this `AutoResetEvent and ManualResetEvent in C# with Examples` article.

Thread Life Cycle in C# with Examples

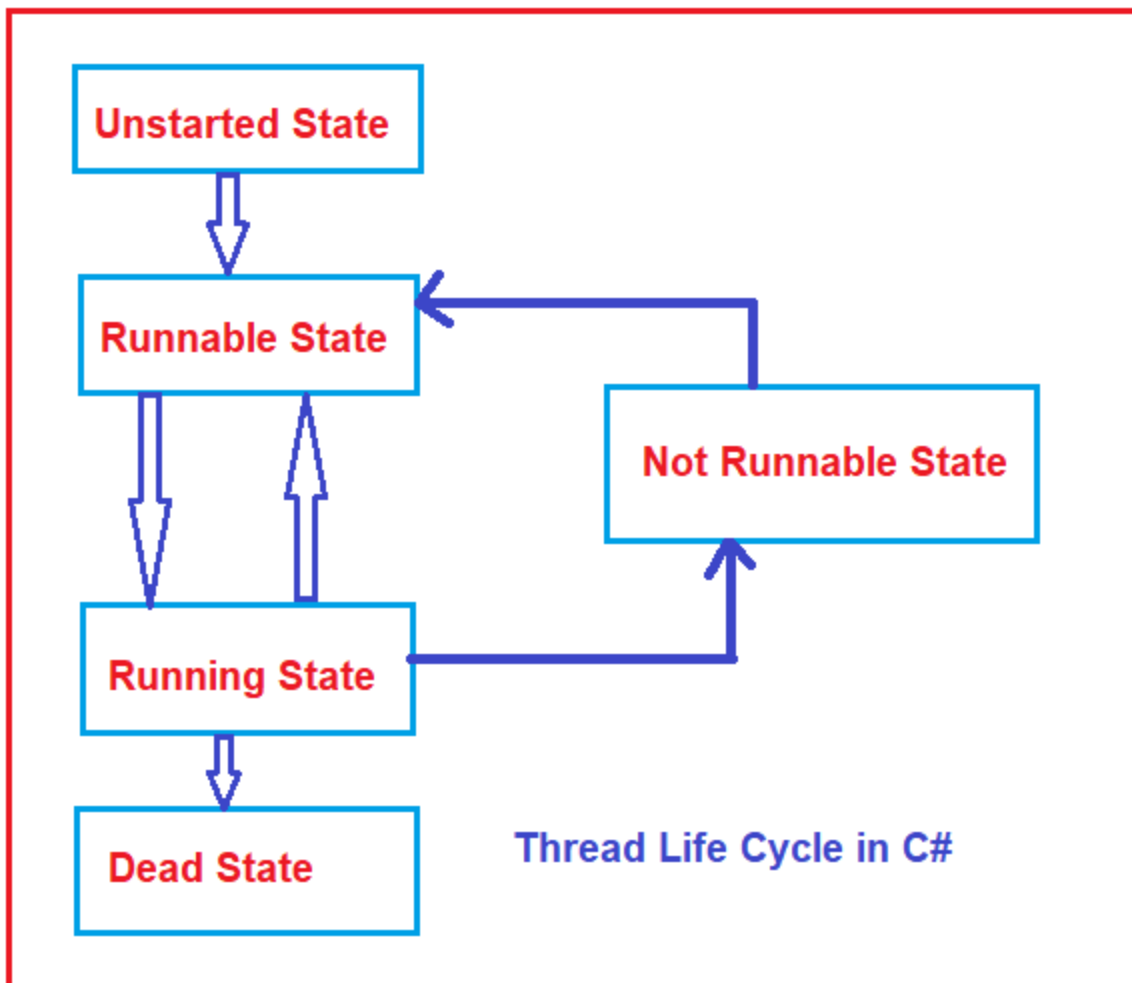
In this article, I am going to discuss **Thread Life Cycle in C#** with Examples. Please read our previous article where we discussed [AutoResetEvent and ManualResetEvent in C#](#) with Examples.

Thread Life Cycle in C#

A thread in C# has a life cycle, which will start when we create an instance of the Thread class, and the thread life cycle ends when the task execution of the thread is completed or when the thread is terminated. A thread in C# at any point in time exists in any one of the following states.

1. **Unstarted (New) State**
2. **Runnable State (Ready to Run)**
3. **Running**
4. **Not Runnable State**
5. **Dead State**

For a better understanding, please have a look at the below diagram which shows the different states of the thread in C#.



Now, let us understand each state in detail.

Unstarted State or New State:

When we create an instance of a Thread class, it is in an unstarted state. That means the thread is just created but it is not yet started. That means we have created the thread object but not called Start() method. The following is an example to create a thread instance in C#,

```
Thread thread = new Thread(SomeMethod);
```

Once you execute the above statement, the thread is created but it is in the unstarted state. In the unstarted state, a thread is not considered alive. Once the Start() method is called on the thread instance, it leaves the unstarted state and enters the next state but once it leaves the unstarted state, it is not impossible for a thread to return back to this unstarted state in its lifetime. The unstarted state can also be called the new state.

Runnable State (Ready to Run):

When we call the start() method, then the thread will move from the unstarted state to the runnable or ready-to-run state. That means now the thread is eligible to run, but it is not running yet, because the thread scheduler has not selected it to run. It might be possible that there could be multiple threads in the runnable state. If there are multiple threads in the runnable state, then the thread scheduler will decide which thread from the runnable state should be moved to the next state i.e. Running state. A thread in the runnable state is considered to be alive. A thread can return to a runnable state after coming back from sleeping, waiting/blocked, or running state.

Running State:

A thread will enter into the running state, when the thread scheduler selected it to run (out of all the threads which are ready to run or in the runnable state), only one thread within a process can be executed at a time. At the time of execution, the thread is in the running state. A thread in the Running state is alive. From the running state, a thread can enter into the Not Runnable state, Runnable State, or Dead state.

Not Runnable State:

A thread in C# enters into the Not Runnable State in the following scenarios. When a thread finds itself in any of the below situations, then the thread will move into the not runnable state and the thread is no longer eligible to run, but even in this state, the thread is still considered to be alive. Some people also refer to this state as the WaitSleepJoin state.

1. When we called the Wait() method on the thread object and it is waiting for other threads to notify it or wake it up.
2. When we called the Sleep() method on the thread object and asked it to sleep for some duration of time.
3. When a thread has called the Join() method on another thread, which makes the first thread wait until another thread has finished its execution.
4. When a thread is waiting for an Input/Output or other resources to be free.

Note: When the thread gets out of this Not Runnable state, then it will re-enter into a runnable state.

Dead State:

When the thread completed its task, the thread enters into a dead, terminates, or abort state. That means the execution of the thread is finished. This is the last state in a thread's life cycle. A thread enters the dead state

after it has successfully completed executing its entry point method i.e. Start(), or when the Abort() method has been called on it to abort its execution. In this state, a thread is considered to be not alive, and hence if you try to call Start() method on a dead thread, it raises the ThreadStateException exception.

Important Properties and Methods of Thread Class in C#:

The Thread Class in C# provides various properties that allow us to perform different tasks such as obtaining the status of a thread and specifying a name for the thread. Following are the properties of the Thread class in C#.

1. **CurrentThread**: It gets the currently running thread instance. That means it returns a Thread instance that is the currently running thread.
2. **IsAlive**: It gets a value indicating the execution status of the current thread. It returns true if this thread has been started and has not terminated normally or aborted; otherwise, false.
3. **Name**: It is used to get or set the name of the thread. It returns a string containing the name of the thread, or null if no name was set.
4. **ThreadState**: It gets a value containing the states of the current thread. It returns one of the System.Threading.ThreadState values indicate the state of the current thread. The initial value is Unstarted.

The Thread Class in C# provides the following methods to implement the states of the threads.

1. **Sleep()**: This method Suspends the current thread for the specified amount of time.
2. **Join()**: This method blocks the calling thread until the thread represented by this instance terminates while continuing to perform standard COM and SendMessage pumping.
3. **Abort()**: This method Raises a System.Threading.ThreadAbortException in the thread on which it is invoked, to begin the process of terminating the thread. Calling this method usually terminates the thread.
4. **Suspend()**: This method either suspends the thread or if the thread is already suspended, has no effect.
5. **Resume()**: This method resumes a thread that has been suspended.
6. **Start()**: This method causes the operating system to change the state of the current instance to the Running state.

Example to understand Thread Life Cycle States in C#:

The following example shows the different states of the thread. These states of the thread are determined by using the ThreadState property of the Thread class. Also, we use Suspend() and Resume() methods to suspend the current execution of the thread and resume the suspended thread by using the Resume() method. We have also called the Abort method to terminate the thread and after terminating the thread, again, we are calling the Start method on the thread instance which will throw an exception.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

    try
    {
        // Creating and initializing threads Unstarted state
        Thread thread1 = new Thread(SomeMethod);
        Console.WriteLine($"Before Start, IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
        // Running State
        thread1.Start();
        Console.WriteLine($"After Start(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
        // thread1 is in suspended state
        thread1.Suspend();
        Console.WriteLine($"After Suspend(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
        // thread1 is resume to running state
        thread1.Resume();
        Console.WriteLine($"After Resume(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
        // thread1 is in Abort state
        //In this case, it will start the termination, IsAlive still {
true
        thread1.Abort();
        Console.WriteLine($"After Abort(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
        //Calling the Start Method on a dead thread will result an Exce
        thread1.Start();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception Occurred: {ex.Message}");
    }

    Console.ReadKey();
}
public static void SomeMethod()
{
    for (int x = 0; x < 3; x++)
    {
        Thread.Sleep(1000);
    }
    Console.WriteLine("SomeMethod Completed...");
}
}
}

```

Output:

```
Before Start, IsAlive: False, ThreadState: Unstarted  
After Start(), IsAlive: True, ThreadState: Running  
After Suspend(), IsAlive: True, ThreadState: SuspendRequested  
After Resume(), IsAlive: True, ThreadState: Running  
After Abort(), IsAlive: True, ThreadState: AbortRequested  
Exception Occurred: Thread is running or terminated; it cannot restart.
```

In the next article, I am going to discuss [Thread Priority in C#](#) with Examples. Here, in this article, I try to explain **Thread Life Cycle in C#** with Examples. I hope you enjoy this Thread Life Cycle in C# with Examples article.

Threads Priorities in C# with Examples

In this article, I am going to discuss **Threads Priorities in C#** with Examples. Please read our previous article where we discussed [Thread Life Cycle in C#](#) with Examples.

Threads Priorities in C#

In C# Programming Language, each and every thread has a priority that determines how often the thread gets access to the CPU. In general, a Low-Priority Thread will get less CPU time than a High-Priority Thread. The important point that we need to understand is how much CPU time a thread will get, it doesn't only depend on its priority, but also depends on the kind of operation it is performing.

For example, if a high-priority thread is waiting for some shared I/O resources to complete its task, then it will be blocked and taken off by the CPU. And at the same time, a lower-priority thread may get the CPU time and finish its execution if it doesn't require such shared I/O resources. In scenarios like this, a High-Priority Thread may get less CPU time than a Low-Priority Thread over a specific period of time.

Another factor that determines how much CPU time is allocated to a thread is how the task scheduling is implemented by the operating system.

How to Set the Priority of a Thread in C#?

When we created an instance of the Thread class, the thread object gets a default priority setting. We can get or set the Priority of a thread by using the following Priority property of the Thread class.

1. **ThreadPriority Priority {get; set;}:** This property is used to get or set a value indicating the scheduling priority of a thread. It returns one of the ThreadPriority values. The default value is ThreadPriority.Normal.

That means we can set the Priority property with one of the fields of the ThreadPriority Enum. If you go to the definition of the ThreadPriority enum. Then you will see the following signature.

```
namespace System.Threading
{
    ...public enum ThreadPriority
    {
        ...Lowest = 0,
        ...BelowNormal = 1,
        ...Normal = 2,
        ...AboveNormal = 3,
        ...Highest = 4
    }
}
```

The ThreadPriority enum provides the following 5 properties:

1. **Lowest = 0**: The Thread can be scheduled after threads with any other priority. That means threads with the Lowest Priority can be scheduled after threads with any other higher priority.
2. **BelowNormal = 1**: The Thread can be scheduled after threads with Normal priority and before those with Lowest Priority. That means threads with BelowNormal priority can be scheduled after threads with Normal priority and before threads with Lowest priority.
3. **Normal = 2**: The Thread can be scheduled after threads with AboveNormal priority and before those with BelowNormal Priority. Threads have Normal priority by default. That means threads with Normal priority can be scheduled after threads with AboveNormal priority and before threads with BelowNormal and Lowest priority.
4. **AboveNormal = 3**: The Thread can be scheduled after threads with the Highest priority and before those with Normal Priority. That means threads with AboveNormal priority can be scheduled after the thread with the Highest priority and before threads with Normal, BelowNormal, and Lowest priority.
5. **Highest = 4**: The Thread can be scheduled before threads with any other priority. That means threads with the Highest Priority can be scheduled before threads with any other priority.

Note: By default, when we created a thread, it gets a default priority of 2 i.e. ThreadPriority.Normal

How can we Set and Get the Priority of a Thread in C#?

Let us see an example to understand how to set and get the priorities of a Thread in C# by using the Priority property of the Thread class. For a better understanding, please have a look at the below example.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            //Setting the thread Priority as Normal
            thread1.Priority = ThreadPriority.Normal;
            Thread thread2 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            //Setting the thread Priority as Lowest
            thread2.Priority = ThreadPriority.Lowest;
            Thread thread3 = new Thread(SomeMethod)
            {
                Name = "Thread 3"
            };
            //Setting the thread Priority as Highest
            thread3.Priority = ThreadPriority.Highest;
```

```

        //Getting the thread Priority
        Console.WriteLine($"Thread 1 Priority: {thread1.Priority}");
        Console.WriteLine($"Thread 2 Priority: {thread2.Priority}");
        Console.WriteLine($"Thread 3 Priority: {thread3.Priority}");
        thread1.Start();
        thread2.Start();
        thread3.Start();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        for (int i = 0; i < 3; i++)
        {
            Console.WriteLine($"Thread Name: {Thread.CurrentThread.Name} {i}");
        }
    }
}

```

Output:

```

Thread 1 Priority: Normal
Thread 2 Priority: Lowest
Thread 3 Priority: Highest
Thread Name: Thread 3 Printing 0
Thread Name: Thread 3 Printing 1
Thread Name: Thread 3 Printing 2
Thread Name: Thread 1 Printing 0
Thread Name: Thread 2 Printing 0
Thread Name: Thread 2 Printing 1
Thread Name: Thread 2 Printing 2
Thread Name: Thread 1 Printing 1
Thread Name: Thread 1 Printing 2

```

Note: The output is unpredictable because threads are highly system-dependent. The thread scheduler of the operating system schedule threads with no guarantee but tries to consider them. For long-running task threads get the benefit of priority setting.

Why do we need Thread Priority in C#?

Well, this is not required in common cases. However, in some cases where you may want to elevate the priority of some threads. One such example could be when you want certain tasks to be completed first over others.

Points to Remember:

1. A programmer can explicitly assign priority to a thread.

2. The default value is ThreadPriority.Normal
3. The operating system does not assign priority to threads.
4. It will throw ThreadStateException if the thread has reached a final state, such as Aborted.
5. It will throw ArgumentException if the value specified for a set operation is not a valid ThreadPriority value.
6. It is not guaranteed that the thread whose priority is high will executes first and the thread whose priority is low will executes after. Due to context switching the highest priority thread may execute after the lowest priority thread.

In the next article, I am going to discuss [How to Terminate a Thread in C#](#) with Examples. Here, in this article, I try to explain **Threads Priorities in C#** with Examples. I hope you enjoy this Thread Priority in C# with Examples article.

How to Terminate a Thread in C# with Examples?

In this article, I am going to discuss **How to Terminate a Thread in C#** with Examples. Please read our previous article where we discussed [Threads Priorities in C#](#) with Examples.

How to Terminate a Thread in C#?

In C#, a thread can be terminated by calling the `Abort()` method of the thread instance. The `Abort()` method throws `ThreadAbortException` to the thread in which it is called. Due to this exception, the thread is terminated.

If the `Abort()` method is called on a thread before calling the `Start()` method on it, calling the `Start()` method on such a thread, later on, will not start it, but it will throw the `ThreadStateException` in the thread which called the `Abort()` method and abort both the threads.

If the `Abort()` method is called on a thread that has started and is in either of the blocked states i.e. Waiting State, Sleep State, or Join State, will first interrupt the thread and then abort it.

There are two overloaded versions of the `Abort()` method available in the `Thread` class. They are as follows:

1. **`Abort()`**: This method raises a `System.Threading.ThreadAbortException` in the thread on which it is invoked, to begin the process of terminating the thread. Calling this method usually terminates the thread. It will throw `ThreadStateException` if the thread that is being aborted is currently suspended. It will throw `SecurityException` if the caller does not have the required permission.
2. **`Abort(object stateInfo)`**: This method raises a `System.Threading.ThreadAbortException` in the thread on which it is invoked, to begin the process of terminating the thread while also providing exception information about the thread termination. Calling this method usually terminates the thread. Here, the parameter `stateInfo` specifies an object that contains application-specific information, such as the state, which can be used by the thread being aborted. It will throw `ThreadStateException` if the thread that is being aborted is currently suspended. It will throw `SecurityException` if the caller does not have the required permission.

Example to Understand `Abort()` Method of Thread Class in C#:

Let us see an example to Understand the **`Abort()`** Method of Thread Class in C# to terminate a thread. This method raises a `ThreadAbortException` in the thread on which it is invoked, to begin the process of terminating the thread. Generally, this method is used to terminate the thread. For a better understanding, please have a look at the below example.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating and initializing threads
            Thread thread = new Thread(SomeMethod);
```

```

        thread.Start();
        Console.WriteLine("Thread is Abort");
        // Abort thread Using Abort() method
        thread.Abort();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        for (int x = 0; x < 3; x++)
        {
            Console.WriteLine(x);
        }
    }
}

```

Output: Thread is Abort

The above example shows the use of the Abort() method which is provided by the Thread class. By using thread.Abort(); statement, we can terminate the execution of the thread.

Example to Understand Abort(object stateInfo) Method of Thread Class in C#:

Let us see an example to Understand the **Abort(object stateInfo)** Method of Thread Class in C# to terminate a thread. This method raises a ThreadAbortException in the thread on which it is invoked, to begin the process of terminating the thread while also providing exception information about the thread termination. Generally, this method is used to terminate the thread. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread thread = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            thread.Start();
            Thread.Sleep(1000);
            Console.WriteLine("Abort Thread Thread 1");
            thread.Abort(100);
            // Waiting for the thread to terminate.
            thread.Join();
            Console.WriteLine("Main thread is terminating");
            Console.ReadKey();
        }
    }
}

```

```

public static void SomeMethod()
{
    try
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} is Starting");
        for (int j = 1; j <= 100; j++)
        {
            Console.Write(j + " ");
            if ((j % 10) == 0)
            {
                Console.WriteLine();
                Thread.Sleep(200);
            }
        }
        Console.WriteLine($"{Thread.CurrentThread.Name} Exiting Normal");
    }
    catch (ThreadAbortException ex)
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} is aborted and the code is {ex.ExceptionState}");
    }
}
}
}

```

Output:

```

Thread 1 is Starting
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
Abort Thread Thread 1
Thread 1 is aborted and the code is 100
Main thread is terminating

```

Calling the Abort() method on a Running Thread in C#:

In the below example, we are calling the Abort() method on the running thread. This will throw the ThreadAbortException and abort the thread on which the Abort() method is called. Calling the Abort() method will throw a ThreadAbortException, so we will enclose its statements within a try-catch block to catch the exception.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Creating an object Thread class
        Thread thread = new Thread(SomeMethod)
        {
            Name = "My Thread1"
        };
        thread.Start();
        //Making the main Thread sleep for 1 second
        //Giving the child thread enough time to start its execution
        Thread.Sleep(1000);
        //Calling the Abort() on thread object
        //This will abort the new thread and throw ThreadAbortException in
        thread.Abort();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        try
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Started its
Execution");
            for (int i = 0; i < 3; i++)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} is printing {i}");
                //Calling the Sleep() method to make it sleep and
                //suspend for 2 seconds after printing a number
                Thread.Sleep(1000);
            }
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Finished its
Execution");
        }
        catch (ThreadAbortException e)
        {
            Console.WriteLine($"ThreadAbortException Occurred, Message : {
e.Message}");
        }
    }
}

```

Output:

```

My Thread1 Has Started its Execution
My Thread1 is printing 0
My Thread1 is printing 1
ThreadAbortException Occurred, Message : Thread was being aborted.

```

Calling the Abort() method on a thread that hasn't started yet.

In the below example, we are calling the Abort() method on a thread before calling the Start() method on it. Calling the Start() method on such a thread, later on, will not start it, but throw the ThreadStartException. For a better understanding, please have a look at the below example.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                //Creating an object Thread class
                Thread MyThread = new Thread(SomeMethod)
                {
                    Name = "My Thread1"
                };
                //Calling the Abort() method on MyThread which hasn't started
                //This will leads to the ThreadStartException
                //And calling the Start() method on the same thread later on v
it and throw ThreadStartException
                MyThread.Abort();
                //Calling the Start() method will not start the thread
                //but throw ThreadStartException and abort it.
                //Because the Abort() method was called on it before it could
                MyThread.Start();
                Console.WriteLine("Main Thread has terminated");
            }
            catch (ThreadStartException e)
            {
                Console.WriteLine($"ThreadStartException Occurred, Message : {
e.Message}");
            }

            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            try
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} Has Started it
Execution");
                for (int i = 0; i < 3; i++)
                {
                    Console.WriteLine($"{Thread.CurrentThread.Name} is printin
                    //Calling the Sleep() method to make it sleep and
```

```

        //suspend for 2 seconds after printing a number
        Thread.Sleep(1000);
    }
    Console.WriteLine($"{Thread.CurrentThread.Name} Has Finished its
Execution");
}
catch (ThreadAbortException e)
{
    Console.WriteLine($"ThreadAbortException Occurred, Message : {
e.Message}");
}
}
}
}
}

```

Output:

```
ThreadStartException Occurred, Message : Thread failed to start.
```

Calling the Abort() method on a Thread that is in Blocked State in C#:

When we call the Abort() method on a thread, which has started its execution, but presently it is in either of the blocked states i.e. Wait State, Sleep State, or Join State, will first interrupt the thread and then abort it by throwing ThreadAbortException. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating an object Thread class
            Thread MyThread = new Thread(SomeMethod)
            {
                Name = "My Thread1"
            };
            MyThread.Start();
            //Making the Main thread sleep for 500 milliseconds
            //Which gives enough time for its child start to start its execution
            Thread.Sleep(500);
            //Main thread calling Abort() on the child Thread which is in a blocked state
            //will throw ThreadAbortException
            MyThread.Abort();

            //Main thread has called Join() method on the new thread
            //To wait until its execution is complete
            MyThread.Join();
        }
    }
}

```

```

        Console.WriteLine("Main Thread has terminated");
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        try
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Started its
Execution");
            for (int i = 0; i < 3; i++)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} is printing {i}");
                //Calling the Sleep() method on newly created thread
                //To make it sleep and suspend for 3 seconds after printing
                Thread.Sleep(3000);
            }
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Finished its
Execution");
        }
        catch (ThreadAbortException e)
        {
            Console.WriteLine($"ThreadAbortException Occurred, Message : {
e.Message}");
        }
    }
}

```

Output:

```

My Thread1 Has Started its Execution
My Thread1 is printing 0
ThreadAbortException Occurred, Message : Thread was being aborted.
Main Thread has terminated

```

In the next article, I am going to discuss [Inter Thread Communication in C#](#) with Examples. Here, in this article, I try to explain **How to Terminate a Thread in C#** with Examples. I hope you enjoy this How to Terminate a Thread in C# with Examples article.

Inter Thread Communication in C# with Examples

In this article, I am going to discuss **Inter Thread Communication in C#** with Examples. Please read our previous article where we discussed [How to Terminate a Thread in C#](#) with Examples.

Interthread Communication in C#:

Interthread Communication in C# is a mechanism of communication between two or more threads that work on the shared resource. To perform multiple actions at the same time, we need Inter-thread communication. In order to have smooth interthread communication between multiple threads in C#, we can use the Monitor Class. The Monitor class in C# provides some static methods such as `Wait()`, `Pulse()`, and `PulseAll()` which we can use to perform Interthread Communication between multiple threads. The Monitor class belongs to `System.Threading` namespace.

If you want to learn and understand the C# Monitor Class in detail, then please click on the below URL.

<https://dotnettutorials.net/lesson/multithreading-using-monitor/>

Methods of Monitor Class for Interthread communication in C#:

1. **`public static bool Wait(Object obj)`**: This method releases the lock on an object and blocks the current thread until it reacquires the lock. In other words, this method makes the thread that has called the `Wait()` method wait for the other thread to complete its work on the same object. Here, the parameter `obj` specifies the object on which to wait. It returns true if the call is returned because the caller reacquired the lock for the specified object. This method does not return if the lock is not reacquired. It will throw `ArgumentNullException` if the `obj` parameter is null. It will throw `SynchronizationLockException` if the calling thread does not own the lock for the specified object. And this method will throw `ThreadInterruptedException` if the thread that invokes `Wait` is later interrupted from the waiting state. This happens when another thread calls this thread's `System.Threading.Thread.Interrupt` method.
2. **`public static void Pulse(object obj)`**: This method notifies a thread in the waiting queue of a change in the locked object's state. That means The thread was waiting(after calling `Wait()` method) on the same object. Here, the parameter `obj` specifies the object a thread is waiting for. It will throw `ArgumentNullException` if the `obj` parameter is null. It will throw `SynchronizationLockException` if the calling thread does not own the lock for the specified object.
3. **`public static void PulseAll(object obj)`**: This method notifies all waiting threads of a change in the object's state i.e. about the release of lock over the object. Here, the parameter `obj` specifies the object that sends the pulse. It will throw `ArgumentNullException` if the `obj` parameter is null. It will throw `SynchronizationLockException` if the calling thread does not own the lock for the specified object.

Note: The Calling of `Wait()`, `Pulse()`, `PulseAll()` method is only possible from within the synchronized context i.e. from within a synchronized block with a lock.

Example to Understand Interthread Communication in C#:

The `Wait()` Method of Monitor Class is used to Release the lock on an object in order to permit other threads to lock and access the object. The calling thread waits while another thread accesses the object. The `Pulse` signals are used to notify waiting threads about changes to an object's state. For a better understanding, please have a look at the below example.

Let us understand this with a real-time example. Our business requirement is to print the Even and Odd number sequence using 2 different threads. So, One thread will print the even numbers and another thread will print the odd numbers.

Thread T1: 0,2,4,6,8...

Thread T2: 1,3,5,7,9...

Output: 0,1,2,3,4,5,6,7,8,9...

To solve the problem let's use the signaling mechanism using the Monitor Class Wait() and Pulse() Methods in C#. In the following example, we use the Monitor.Wait() method to make the thread waiting and Monitor.Pulse() method to signal other thread. The process is as follows:

1. First, the Even thread will start to print the number on the console.
2. Then the Even thread will signal the Odd thread to print the number using the Monitor.Pulse() method.
3. Again, the Even thread will wait for a signal from the Odd thread using Monitor.Wait() method.
4. The same thing will also be done by the Odd Thread.
5. The Odd thread will start to print the number on the console.
6. Then the Odd thread will signal the Even thread to print the number using Monitor.Pulse() method.
7. Again, the Odd thread will wait for the signal from the Even thread using Monitor.Wait() method.
8. The same process is going on.

Since both the Odd and Even threads are sharing the same console window to print the number, we need to put a lock on the console IO. We want the sequence to be started with the even number, So, the even thread must run first. Once, we will start the Even thread, then we need to pause for a moment before starting the Odd thread using the Sleep() method of the Thread class in C# to avoid any chance to start the Odd thread first.

```
using System;
using System.Threading;
namespace InterthreadCommunications
{
    class Program
    {
        //Limit numbers will be printed on the Console
        const int numberLimit = 10;
        static readonly object _lockObject = new object();
        static void Main(string[] args)
        {
            Thread EvenThread = new Thread(PrintEvenNumbers);
            Thread OddThread = new Thread(PrintOddNumbers);
            //First Start the Even thread.
            EvenThread.Start();
            //Pause for 10 ms, to make sure Even thread has started
            //or else Odd thread may start first resulting different sequence.
            Thread.Sleep(100);
            //Next, Start the Odd thread.
            OddThread.Start();
            //Wait for all the childs threads to complete
            OddThread.Join();
            EvenThread.Join();
            Console.ReadKey();
        }
    }
}
```

```

}
//Printing of Even Numbers Function
static void PrintEvenNumbers()
{
    try
    {
        //Implement lock as the Console is shared between two threads
        Monitor.Enter(_lockObject);
        for (int i = 0; i <= numberLimit; i = i + 2)
        {
            //Printing Even Number on Console)
            Console.Write($"{i} ");
            //Notify Odd thread that I'm done, you do your job
            Monitor.Pulse(_lockObject);
            //I will wait here till Odd thread notify me
            // Monitor.Wait(monitor);
            //Without this logic application will wait forever
            bool isLast = false;
            if (i == numberLimit)
            {
                isLast = true;
            }
            if (!isLast)
            {
                //I will wait here till Odd thread notify me
                Monitor.Wait(_lockObject);
            }
        }
    }
    finally
    {
        //Release the lock
        Monitor.Exit(_lockObject);
    }
}

//Printing of Odd Numbers Function
static void PrintOddNumbers()
{
    try
    {
        //Hold lock as the Console is shared between two threads
        Monitor.Enter(_lockObject);
        for (int i = 1; i <= numberLimit; i = i + 2)
        {
            //Printing the odd numbers on the console
            Console.Write($"{i} ");
            //Notify Even thread that I'm done, you do your job
            Monitor.Pulse(_lockObject);
            //I will wait here till even thread notify me
            // Monitor.Wait(monitor);
            // without this logic application will wait forever

```

```

        bool isLast = false;
        if (i == numberLimit - 1)
        {
            isLast = true;
        }
        if (!isLast)
        {
            //I will wait here till Even thread notify me
            Monitor.Wait(_lockObject);
        }
    }
}
finally
{
    //Release lock
    Monitor.Exit(_lockObject);
}
}
}
}

```

Output: 0 1 2 3 4 5 6 7 8 9 10

How do Wait() and Pulse() methods help to Implement Interthread Communication in C#?

Let us see some more examples to understand the importance of Wait() and Pulse() methods in order to implement the Interthread Communication in C#. Suppose we have two threads i.e. Thread1 and Thread2, where, Thread1 has to print the table of 4, and Thread2 has to print the table of 5 and the table of 4 should be printed before the table of 5, in order to keep an ascending order of tables.

By using Wait() and Pulse() methods for inter-thread communication, Thread2 will call the Wait() method, to wait for Thread1 to finish printing the table of 4. On completing its task, the Thread1 will call Pulse() method, which notifies Thread2 to continue its work of printing the table of 5.

Without the use of Wait() and Pulse() methods, when both threads start at the same time, Thread2 may print the table of 5, before Thread1 has printed the table of 4. Let's see code examples proving each of these cases.

Example without using the Wait and Pulse method in C#:

In the below example, We are going to have two threads. The first thread i.e. Main Thread is given to us by default. And then We manually create a new thread. The manually created new thread will print a table of 4 and the Main Thread will print a table of 5. But, without using the Wait() and Pulse() methods, when two threads start at almost the same time, The Main Thread may print the table of 5 before the manually created thread has printed the table of 4. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;

```

```

namespace InterthreadCommunications
{
    class Program
    {
        static readonly object _lockObject = new object();
        static void Main(string[] args)
        {
            //Creating an object of Thread class to Execute the PrintTable method
            Thread thread = new Thread(PrintTable)
            {
                Name = "Manual Thread"
            };
            thread.Start();
            //Locking the _lockObject
            lock (_lockObject)
            {
                Thread th = Thread.CurrentThread;
                th.Name = "Main Thread";
                Console.WriteLine($"{th.Name} Running and Printing the Table of 5");
                for (int i = 1; i <= 10; i++)
                {
                    Console.WriteLine("5 x " + i + " = " + (5 * i));
                }
            } //synchronized block ends
            Console.ReadKey();
        }

        public static void PrintTable()
        {
            //Synchronizing or locking the _lockObject
            //Doing so, restricts any other thread to access a block of code using
            _lockObject at the same time.
            lock (_lockObject)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} Running and Printing the
                Table of 4");
                for (int i = 1; i <= 10; i++)
                {
                    Console.WriteLine("4 x " + i + " = " + (4 * i));
                }
            }
        }
    }
}

```

Output:

```

Main Thread Running and Printing the Table of 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
Manual Thread Running and Printing the Table of 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40

```

In the above example, the Main thread enters the synchronized block, gets the lock on the `_lockObject`, and gets its synchronized sole access to print the table of 5. It then releases the lock on the `_lockObject`.

In the `PrintTable` method, Manual Thread enters the synchronized block and gets the lock on the same `_lockObject`. It then prints the table of 4. Hence, table 5 is printed before table 4, bothering our ascending order, because there was no interthread communication.

Interthread Communication Example using `Wait()` and `Pulse()` Methods in C#

We are going to have two threads in the below example. The first thread i.e. Main Thread is given to us by default. And we will manually create a new thread, based on an object of the class. The manually created new thread will print a table of 4 and the Main Thread will print a table of 5. We will use `Wait()` and `Pulse()` methods for communication between these two threads, in such a way that table 4 is printed before table 5, to maintain an ascending order.

```

using System;
using System.Threading;
namespace InterthreadCommunications
{
    class Program
    {
        static readonly object _lockObject = new object();
        static void Main(string[] args)
        {
            //Creating an object of Thread class to Execute the PrintTable method

```

```

Thread thread = new Thread(PrintTable)
{
    Name = "Manual Thread"
};
thread.Start();
//Locking the _lockObject
lock (_lockObject)
{
    //Calling the Wait() method in a synchronized context
    //Doing so, makes the Main Thread stops its execution and wait
    //until it is notified by the Pulse() method
    //on the same object _lockObject
    Monitor.Wait(_lockObject);
    Thread th = Thread.CurrentThread;
    th.Name = "Main Thread";
    Console.WriteLine($"{th.Name} Running and Printing the Table of 5");
    for (int i = 1; i <= 10; i++)
    {
        Console.WriteLine("5 x " + i + " = " + (5 * i));
    }
} //synchronized block ends
Console.ReadKey();
}
//The entry-point method of the thread
public static void PrintTable()
{
    //Synchronizing or locking the _lockObject
    //Doing so, restricts any other thread to access a block of code u
    _lockObject at the same time.
    lock (_lockObject)
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} Running and Pr
        Table of 4");
        for (int i = 1; i <= 10; i++)
        {
            Console.WriteLine("4 x " + i + " = " + (4 * i));
        }
        //The manually created thread is calling the Pulse() method
        //To notifying the Main thread that it is releasing the lock c
        _lockObject
        //And Main Thread could lock the object to continue its work
        Monitor.Pulse(_lockObject);
    } //synchronized block ends
}
}
}

```

Output:

```
Manual Thread Running and Printing the Table of 4
```

```
4 x 1 = 4  
4 x 2 = 8  
4 x 3 = 12  
4 x 4 = 16  
4 x 5 = 20  
4 x 6 = 24  
4 x 7 = 28  
4 x 8 = 32  
4 x 9 = 36  
4 x 10 = 40
```

```
Main Thread Running and Printing the Table of 5
```

```
5 x 1 = 5  
5 x 2 = 10  
5 x 3 = 15  
5 x 4 = 20  
5 x 5 = 25  
5 x 6 = 30  
5 x 7 = 35  
5 x 8 = 40  
5 x 9 = 45  
5 x 10 = 50
```

In the above code, we manually created a thread called Manual Thread. The Main thread enters the synchronized block, gets the lock on the `_lockObject`, and gets its synchronized sole access to print the table of 5. However, the Main Thread calls the `Wait()` method, doing so, it releases the lock on the `_lockObject` and stops its execution.

In the `PrintTable` method, Manual Thread enters the synchronized block and gets the lock on the same `_lockObject`. It then prints the table of 4 and once it prints the table of 4, then it calls the `Pulse()` method to notify the waiting Main Thread and releases the lock on `_lockObject`. On being notified, the Main Thread wakes up, locks the `_lockObject`, and completes its execution by printing the table of 5.

Note: In our example, we call the `Wait()` or `Pulse()` methods from within a synchronized context, otherwise such interthread communication would not have been possible.

In the next article, I am going to discuss [How to Debug a Multi-threaded Application in C#](#) with Examples. Here, in this article, I try to explain **Inter Thread Communication in C#** with Examples. I hope you enjoy this **Inter Thread Communication in C# with Examples** article.

How to Debug a Multi-Threaded Application in C#

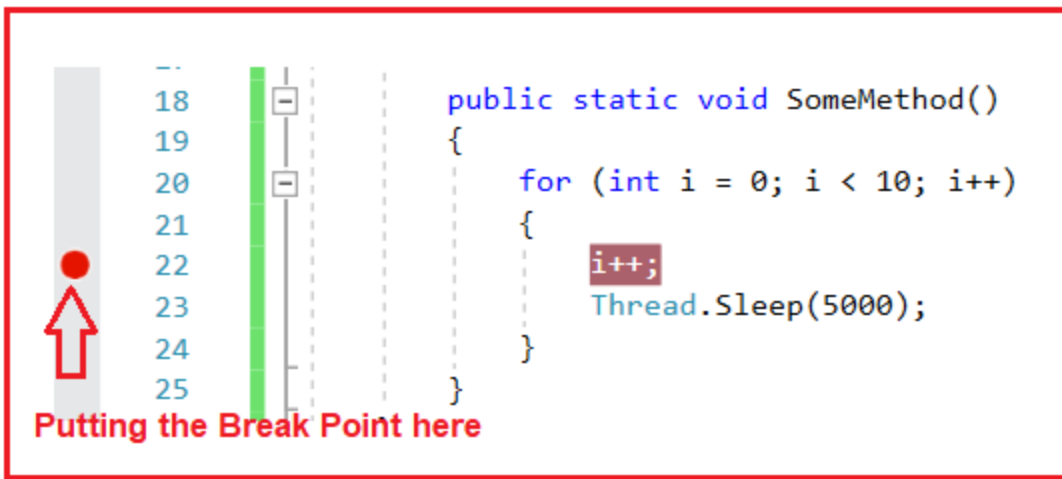
In this article, I am going to discuss **How to Debug a Multi-Threaded Application in C#** with Examples. Please read our previous article where we discussed [Inter Thread Communication in C#](#) with Examples.

How to Debug a Multi-threaded Application in C#?

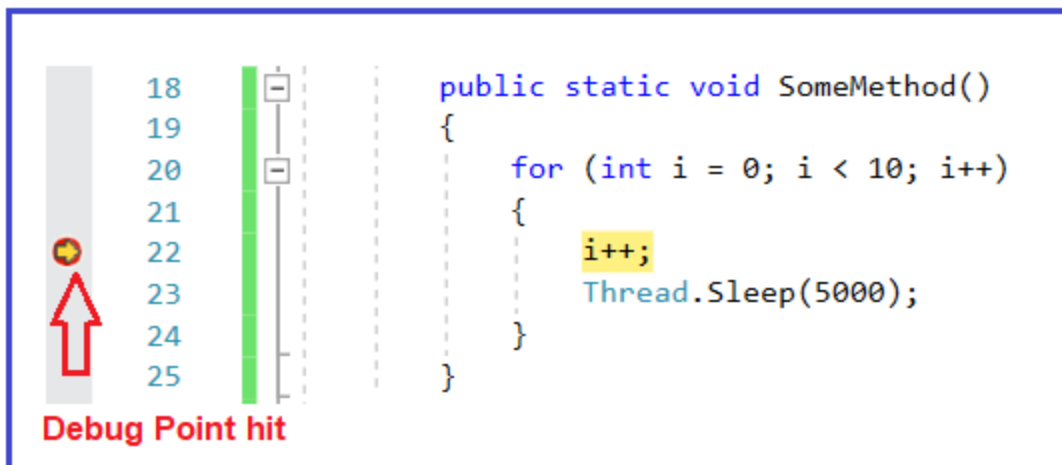
Let us understand how to debug threads in C# using Visual Studio. Please have a look at the below example. In the below example, we have a method called SomeMethod and this SomeMethod contains a for loop which will run 10 times. As part of the method body, it just manipulates the i variable and then sleeps for 5 seconds. Then we invoke the SomeMethod from the Main method by creating two different threads i.e. thread1 and thread2 and then call the Start method.

```
using System;
using System.Threading;
namespace DebugThreadsDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod);
            Thread thread2 = new Thread(SomeMethod);
            thread1.Start();
            thread2.Start();
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            for (int i = 0; i < 10; i++)
            {
                i++;
                Thread.Sleep(5000);
            }
        }
    }
}
```

Now, we want to debug the for loop of the SomeMethod of the above Console application. So, normally what we are intended to do is, we need to put the breakpoint inside the for loop as shown in the below image.

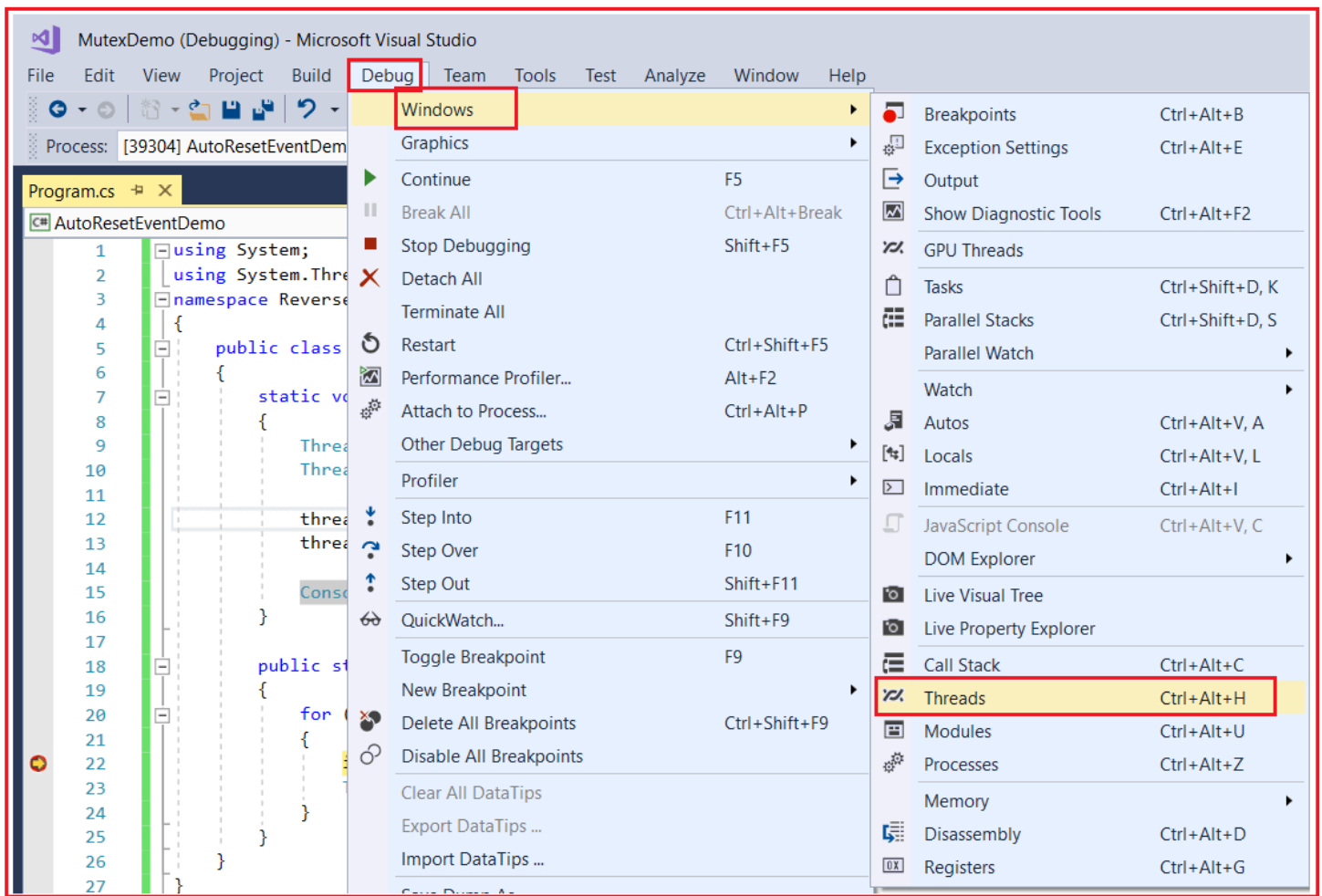


Once you put the debug point, run the application. Once you run the application it will hit the debug point as shown in the below image.



But this debugging has problems. The first problem with this debugging approach is that I am unable to identify currently my debugger is debugging for which thread, whether it is debugging thread1 or thread2, I am unable to identify.

In order to identify the debugger is debugging which thread, just select **Debug => Windows => Threads** options from the context menu as shown in the below image.



So, once you select the **Debug => Windows => Threads** options, it will open the following window. The yellow symbol shows where the current debugger is debugging. In the Location header, you can see the name of the method and from there you can identify the threads. Here, you can see in the Location header, that it is showing three method names i.e. Main and two times SomeMethod. The Main thread executes the Main method and the Worker Threads execute the SomeMethod.

	ID	Managed ID	Category	Name	Location
^ Process ID: 34924 (3 threads)					
▼	33024	1	Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	31096	3	Worker Thread	<No Name>	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	22888	4	Worker Thread	<No Name>	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod


This Worker Thread is Currently Debugging

You can see in the above image that the name for worker threads is showing **<No Name>**. For a better understanding, let us give some meaningful names to our threads. So, modify the example code as follows. Here, we have provided the thread1 name as Thread One and the thread2 name as Thread Two.

```
using System;
using System.Threading;
namespace DebugThreadsDemo
{
    public class Program
```

```

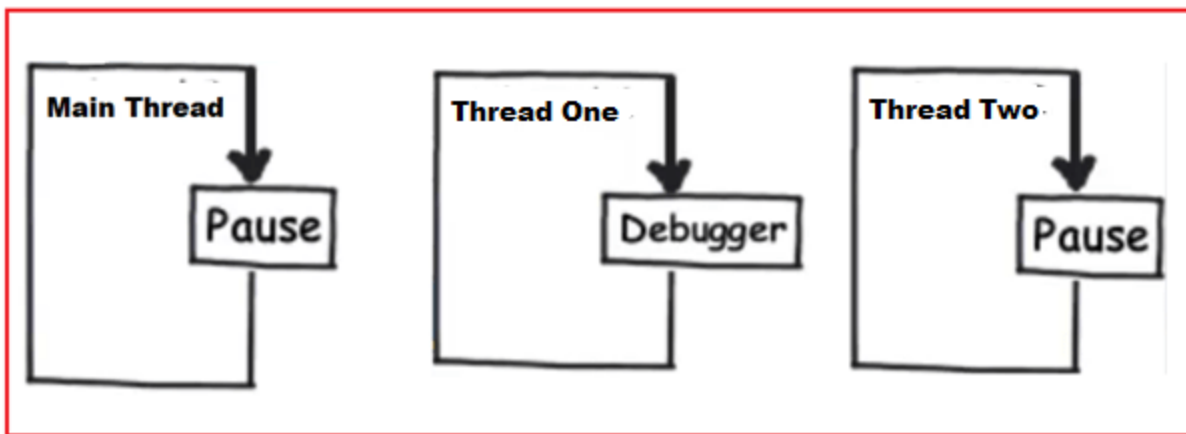
{
    static void Main(string[] args)
    {
        Thread thread1 = new Thread(SomeMethod)
        {
            Name = "Thread One"
        };
        Thread thread2 = new Thread(SomeMethod)
        {
            Name = "Thread Two"
        };
        thread1.Start();
        thread2.Start();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        for (int i = 0; i < 10; i++)
        {
            i++;
            Thread.Sleep(5000);
        }
    }
}
}

```

Now, save the changes. Put the debugger point and run the application. Once you run the application and once the debugger point is hit, open **Debug => Windows => Threads**, and this time it will show you the name of the worker's thread as shown in the below image. Now, the yellow symbol is on Thread two which means it is debugging Thread Two currently. In your case, it might be pointed to Thread One. If you see only one thread, then simply click on F5, and then you will see both threads.

	ID	Managed ID	Category	Name	Location
^ Process ID: 33208 (3 threads)					
▼	35648	1	✖ Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	21912	3	⚙ Worker Thread	Thread One	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	29888	4	⚙ Worker Thread	Thread Two	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod

When our debugger is debugging a thread, then all other threads including the main thread go into the halt mode or you can say pause mode. Suppose, our debugger is debugging Thread One, then don't think that Thread two is running parallelly. As the debugger halted at the SomeMethod i.e. our application code, then all other threads including the main thread will be halted. For a better understanding, please have a look at the below diagram.



As you can see in the **Debug => Windows => Threads** window, currently thread two is debugging. If you want to switch i.e. you want to debug Thread One, then simply select Thread One and double-click on it as shown in the below image.

	ID	Managed ID	Category	Name	Location
^ Process ID: 33208 (3 threads)					
▼	35648	1	Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	21912	3	Worker Thread	Thread One	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	29888	4	Worker Thread	Thread Two	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod

First Select this one and then double click on it to Debugging Thread One

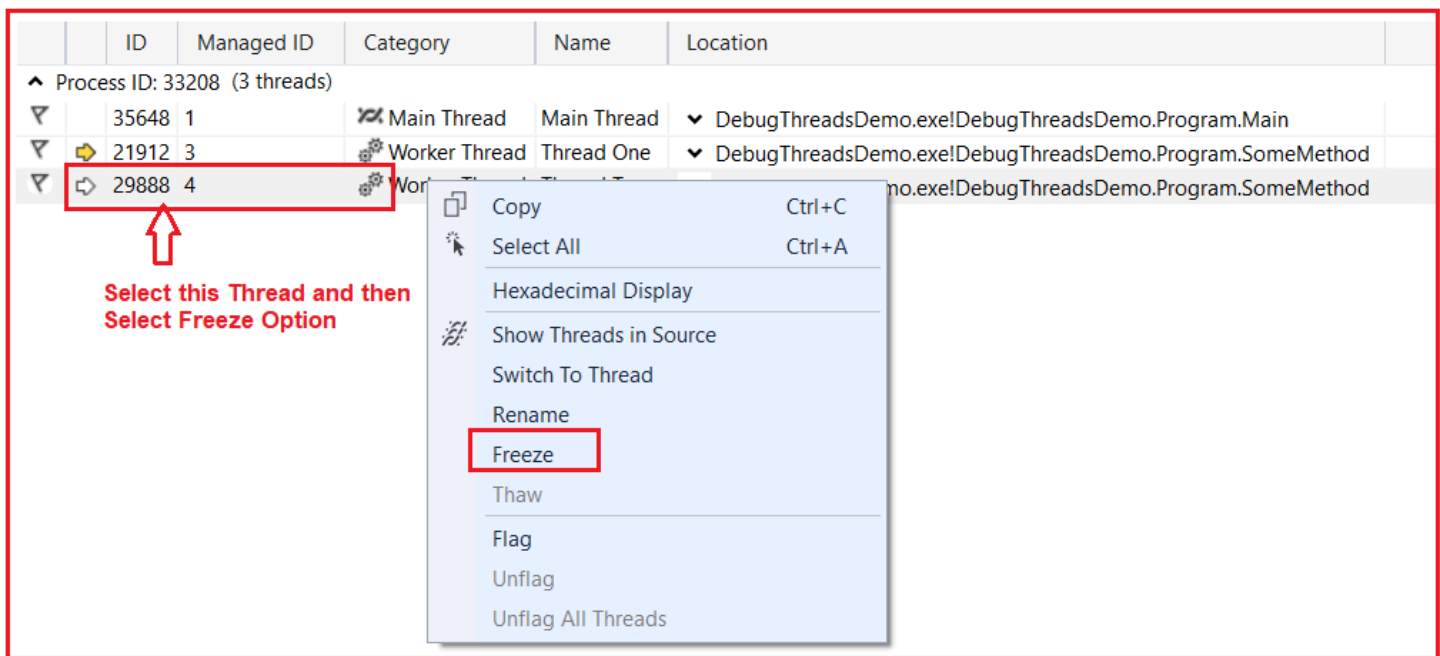
Once you select and double-click on Thread One, then you will see that the yellow symbol is switched to Thread One as shown in the below image which means currently it is debugging Thread One.

	ID	Managed ID	Category	Name	Location
^ Process ID: 33208 (3 threads)					
▼	35648	1	Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	21912	3	Worker Thread	Thread One	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	29888	4	Worker Thread	Thread Two	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod

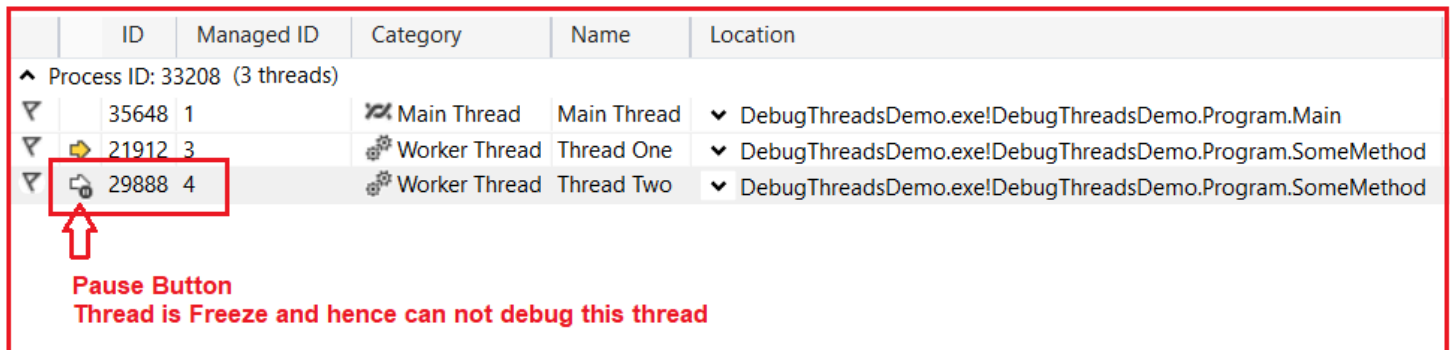
How to Debug a Specific Thread in C#?

Suppose you want to debug only Thread One. You don't want to debug Thread Two. This is also possible. Visual Studio provides two options i.e. **Freeze** and **Thaw**. So, when you select the Freeze option of a thread, then that thread is not going to be debugged by a debugger. Similarly, if you select the Thaw option of a thread, then again it is going to be debugged by the debugger. When a thread is created in C#, by default it is created with the Thaw option.

Now, let us see how to pause Thread Two so that our debugger only debugs Thread One. To do so, right-click on Thread One and then select the Freeze option from the context menu as shown in the below image.

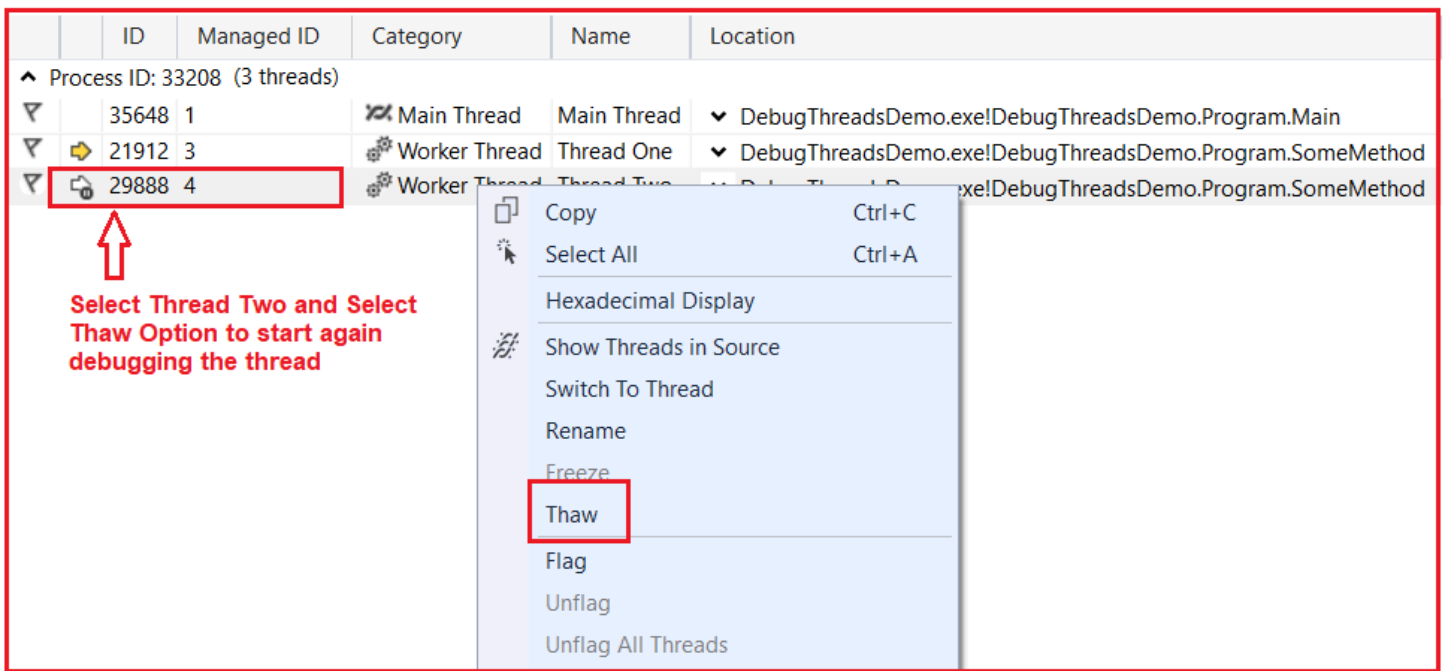


Once you select the Thread and click on the Freeze, then it will Freeze the thread and you can also see the Pause button as shown in the below image.



With the above changes in place, now if you continue your debugging, then you will see that it will only Debug Thread One, it will not debug Thread Two.

Now, again if you want to debug Thread Two, then simply you need to select Thread two, and this time you need to select the Thaw option from the context menu as shown in the below image.

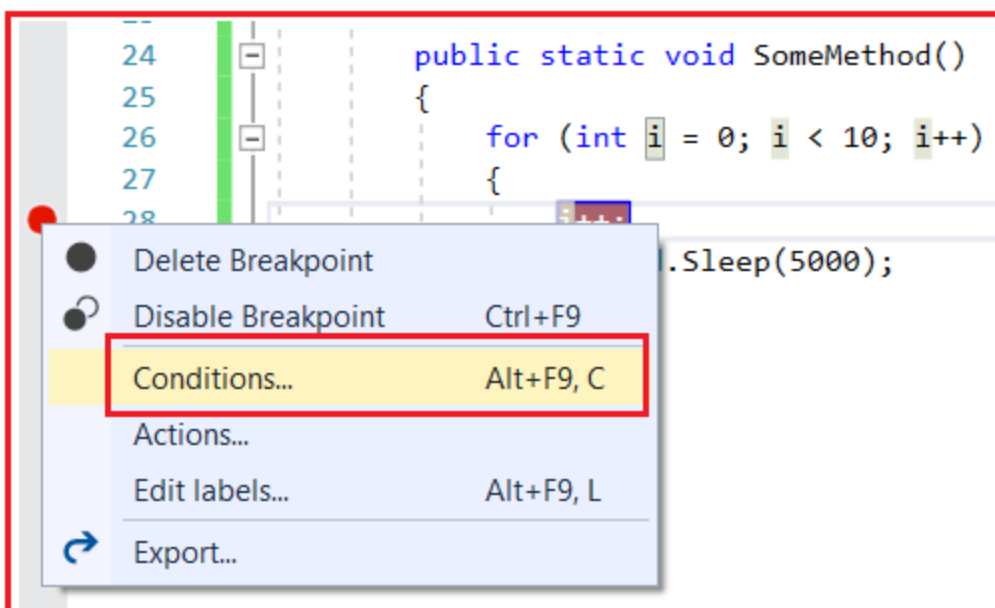


Note: You can use the Freeze option to stop debugging and the Thaw option to allow debugging.

What is Debugger Point with Condition in C#?

Sometimes you want to debug specifics. Let us understand this in detail. In our example, when we run the application in debug mode, sometime it will halt Thread Two if Thread One is debugging or it will halt Thread One if Thread Two is debugging. But what about if you want to only halt the debugger on Thread One, I am not interested to halt the debugger on Thread two. If you want to do so, then you need to create the debugger point with a condition. Let us see the procedure of how to create a debugger point with the condition.

First, right-click on the debugger point and then select the Conditions option from the context menu as shown in the below image.



Once you click on the Conditions option, then it will open the below window. In conditions, we are writing **System.Threading.Thread.CurrentThread.Name == "Thread One"** which means it will only hit the debug point if the thread is Thread One.

Location: Program.cs, Line: 28, Character: 17, Must match source

☒ Conditions

Conditional Expression Is true System.Threading.Thread.CurrentThread.Name == "Thread One" Cancel

Add condition

☐ Actions

Close

With the above changes in place, now if you run the application, then the debugger point will halt only for Thread One. So, by giving a name to our thread how easier it is for us to debug a multithread application in C#.

In the next article, I am going to discuss [Array in C#](#) with Examples. Here, in this article, I try to explain **How to Debug a Multi-threaded Application in C#** with Examples. I hope you enjoy this How to Debug a Multi-threaded Application in C# with Examples article.